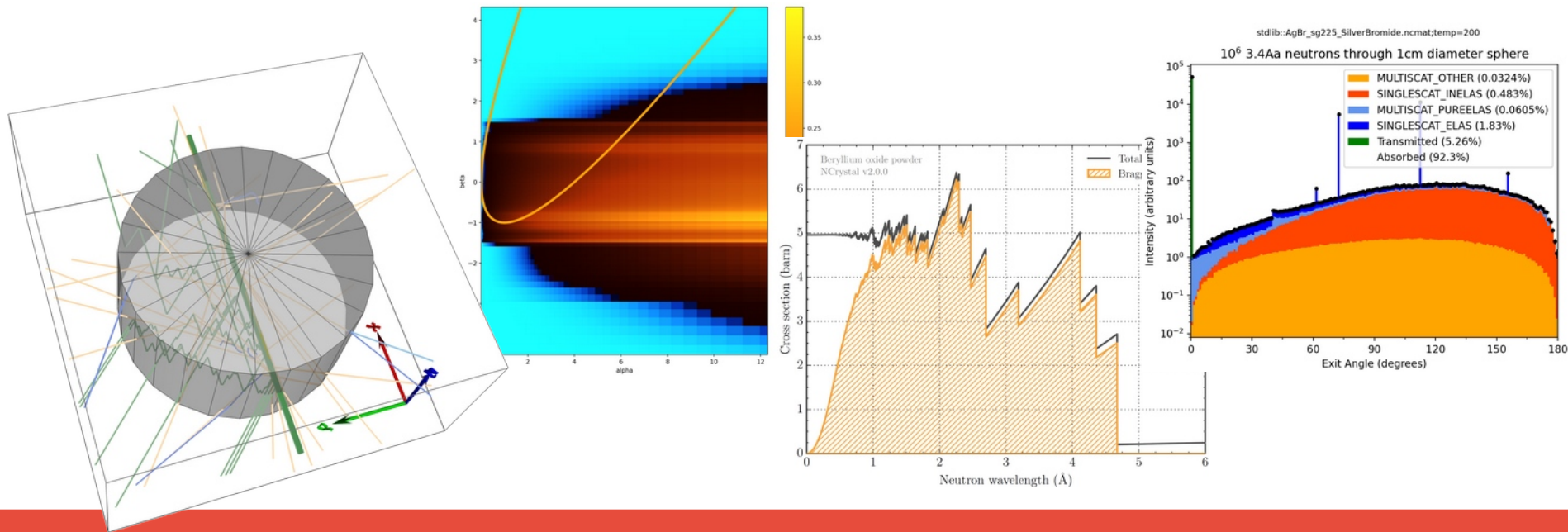




NCrystal : a library for thermal neutron transport



Thomas Kittelmann,
European Spallation Source ERIC
Data Management and Scientific Computing division



McStas training
ISIS, RAL, UK, Nov 4-6, 2025.



BrightNESS is funded by the EU Framework Program for Research and Innovation Horizon 2020, under grant agreement 676548
HighNESS is funded by the EU Framework Program for Research and Innovation Horizon 2020, under grant agreement 951782



EUROPEAN
SPALLATION
SOURCE



HighNESS
brightness

NCrystal for McStas users



NCrystal provides McStas users with
new options for material modelling.

Aim of this presentation:

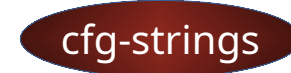
- 1) Explain **why/how/when** to use NCrystal in McStas.
- 2) General **Introduction to NCrystal project.**
Discuss tools, resources, capabilities.

Outline

- NCrystal: Background and introduction



- Core concepts and general usage



- Using NCrystal as a backend engine



- Material formats and data library



- Physics algorithm highlights



- *Quick demos at various points*



NCrystal: Introduction



The NCrystal project: Background

<https://mctools.github.io/ncrystal/>

NCrystal

Thermal Neutron Transport

Original motivation back in ~2014 (X.X. Cai and T. Kittelmann):

Augment Geant4 with proper modeling of thermal neutrons in crystalline materials



Detector frames, vessels, supports



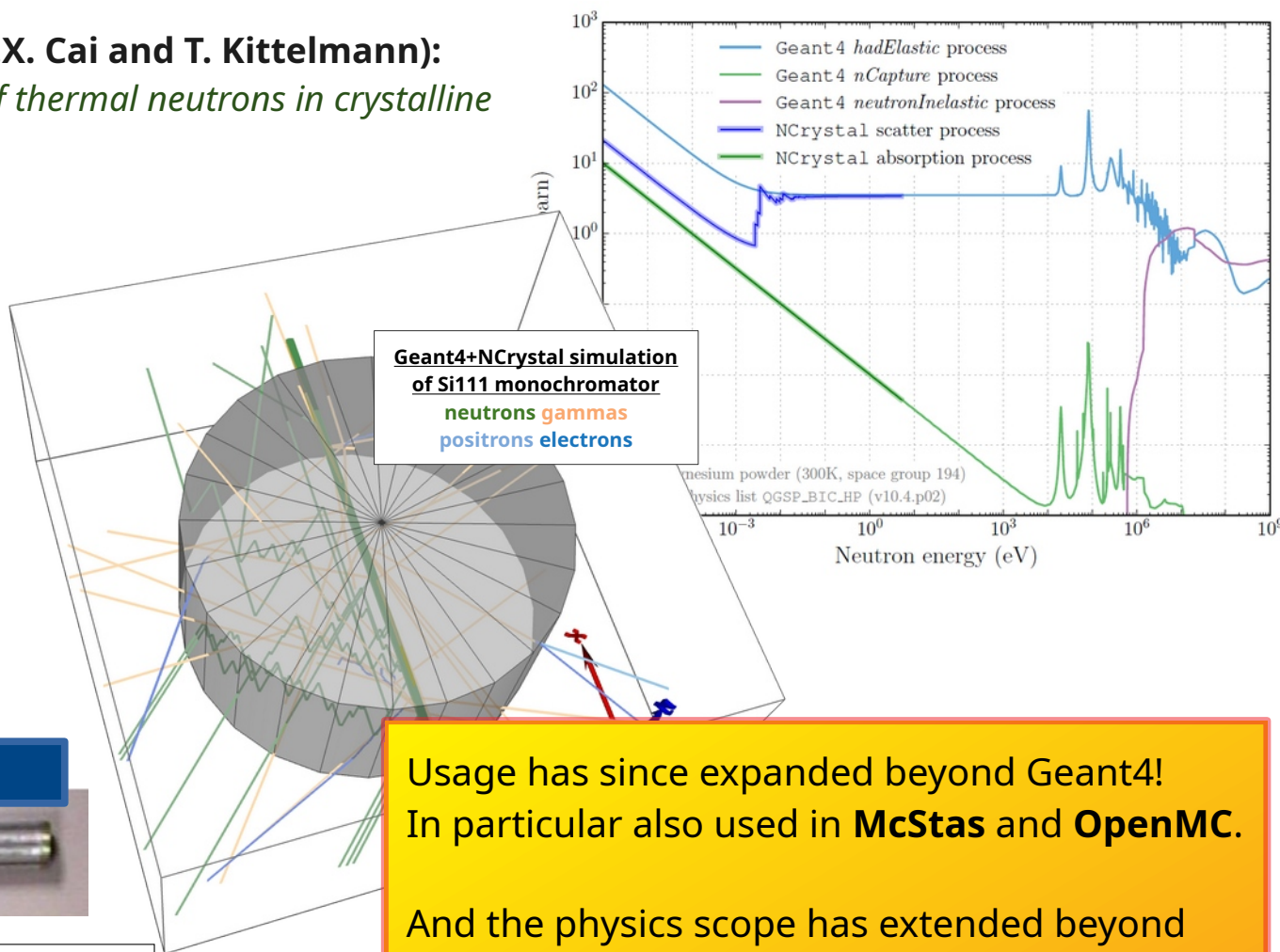
Filters



Monochromators,
analysers
(single crystals,
pyrolytic graphite)



Samples



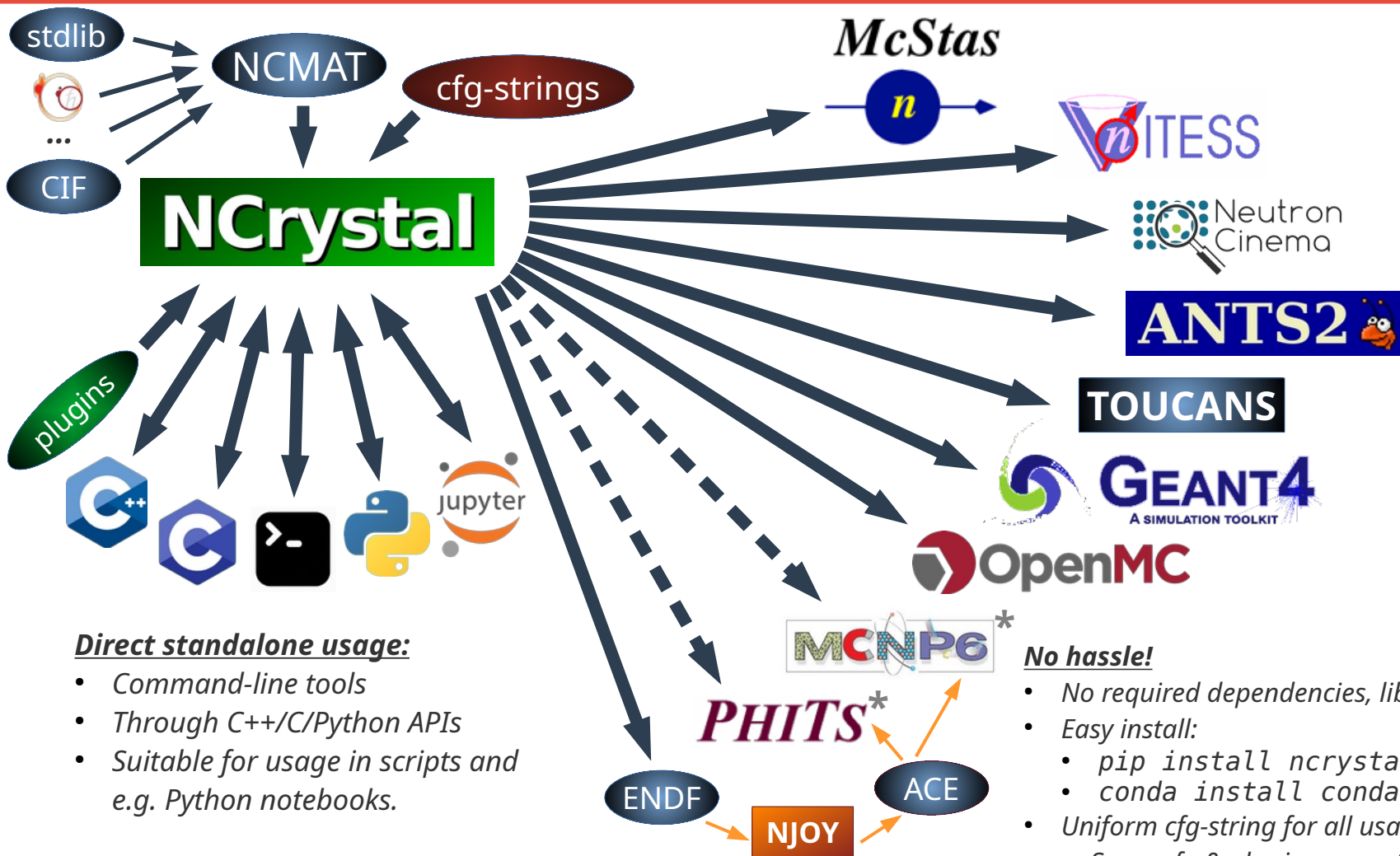
Usage has since expanded beyond Geant4!
In particular also used in **McStas** and **OpenMC**.

And the physics scope has extended beyond
crystals and beyond Bragg diffraction!

Advanced earlier efforts in older “NXSG4” plugin
- T. Kittelmann & M Boin 2015 *Comput. Phys. Commun.* **189**, 114-118

NCrystal: Open Source backend providing thermal neutron scattering to MC codes

NCrystal
Thermal Neutron Transport



Direct standalone usage:

- Command-line tools
- Through C++/C/Python APIs
- Suitable for usage in scripts and e.g. Python notebooks.

No hassle!

- No required dependencies, liberal license (Apache 2)
- Easy install:
 - `pip install ncrystal`
 - `conda install conda-forge::ncrystal`
- Uniform *cfg-string* for all usage
→ Same *cfg* & physics no matter usage context
- Multi-thread safe, support Linux/macOS/BSD/Windows

*: Can not share bindings for projects that are not open-source

Physics models in NCrystal

Crystals, liquids, amorphous solids, gasses, multiphase materials...

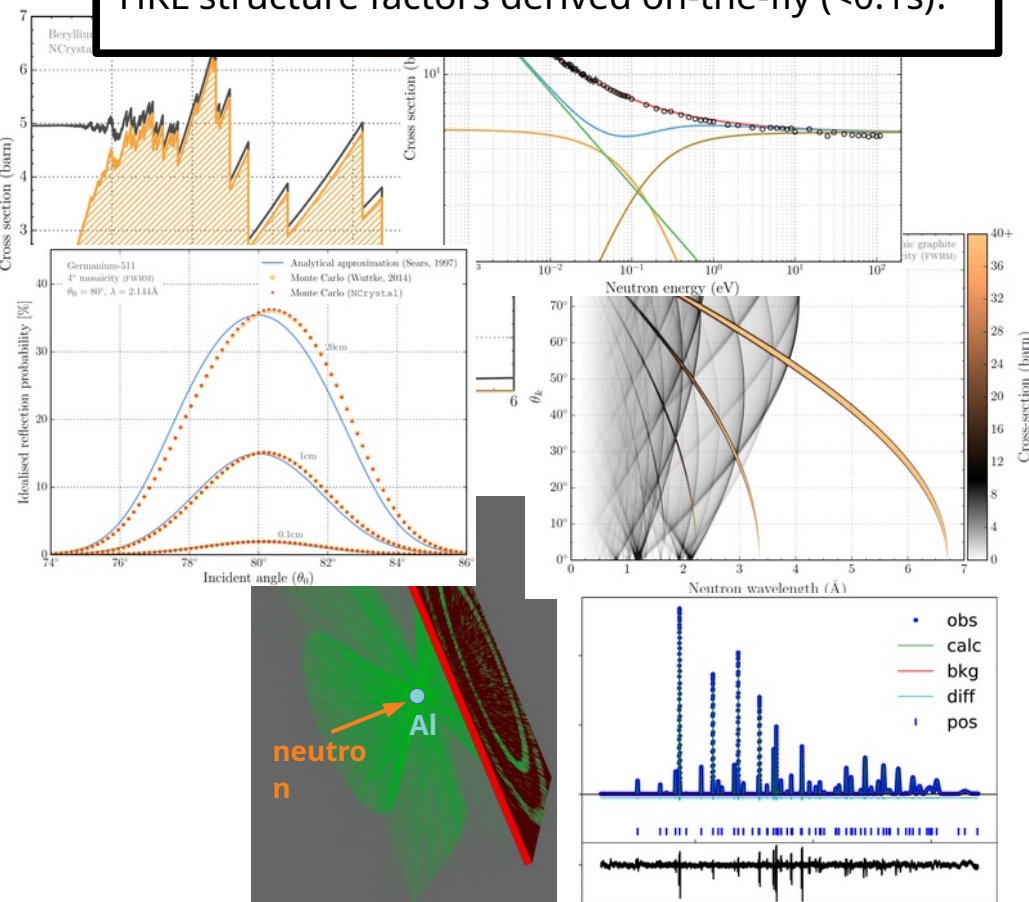
NCrystal

Thermal Neutron Transport

Elastic ($\Delta E=0$) components

Bragg diffraction, incoherent, single crystals, isotropic materials (powders), HOPG.

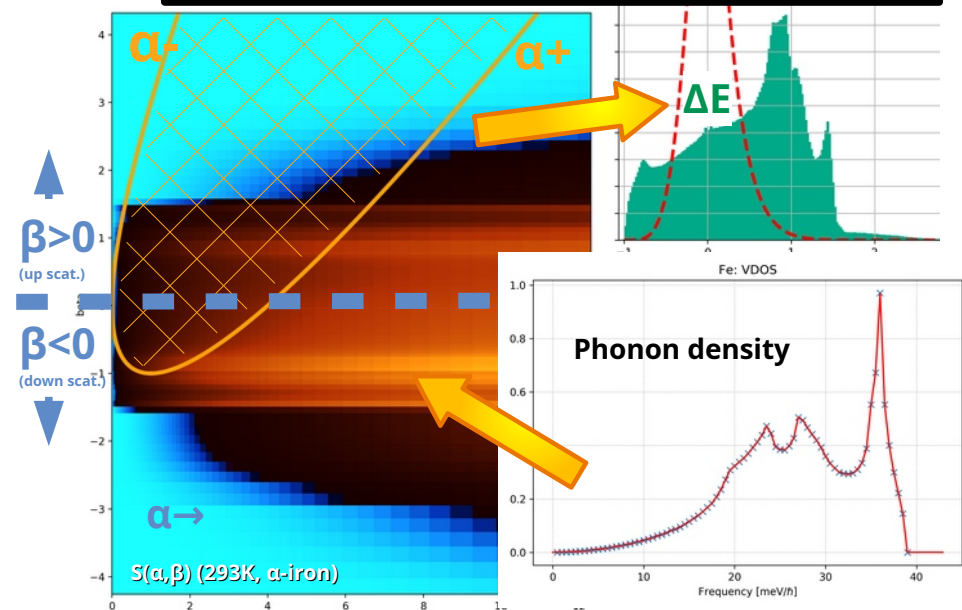
HKL structure factors derived on-the-fly ($<0.1s$).



Inelastic ($\Delta E \neq 0$) components

Scattering kernel based:

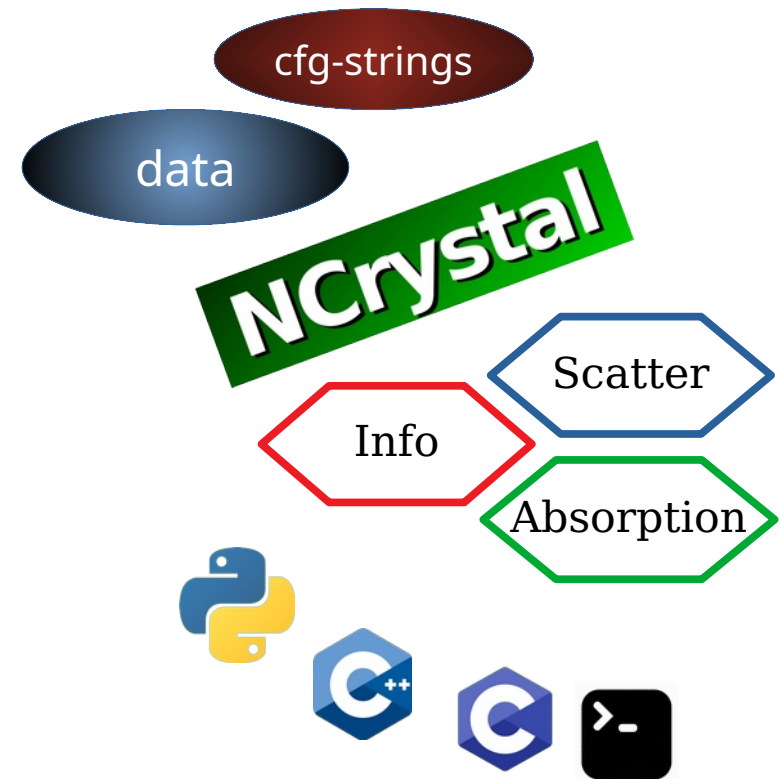
- Initialise from external kernel
- Or from phonon density curve ($\sim 0.1s$).
- Using incoherent approximation.



Other physics via plugins

- SANS, texture, extinction, coherent phonons, magnetic scattering, nanodiamonds, ...
- Intention to move features of wider interest to core project once matured.

NCrystal: Core concepts and general usage



Core NCrystal concepts

All other features, tools, hooks, etc. are built upon these.

NCrystal

Thermal Neutron Transport

Inputs

cfg-strings

```
"Al_sg225.ncmat"  
"Y2SiO5_sg15_YSO.ncmat;dcutoff=0.5Aa"  
"gasmix::0.7xCO2+0.3xAr/1.5atm/250K"  
""phases<0.1*PbS_sg225_LeadSulfide.ncmat  
&0.9*Epoxy_Araldite506_C18H2003.ncmat>""  
"Rubber_C5H8.ncmat;temp=200;vdoslux=4"  
"LiquidHeavyWaterD2O_T293.6K.ncmat"
```

data

Preferred native format is the NCMAT (.ncmat) format.

Can reside in actual files, or as content in memory.

Can be hand-crafted, auto-converted, or generated on-demand by plugins.

Outputs

Info
(object)

Densities, compositions
Phases
Scattering lengths
Crystal structures, HKL factors
Dynamics info like phonon DOS

Scatter
process
(object)

Provides integrated scattering cross sections when queried with neutron state.
Can perform Monte Carlo sampling of outgoing states.

Absorption
process
(object)

Provides integrated absorption cross sections when queried with neutron state.
Presently only implements simple $1/v$ model.

Apply after "filename" part, separated with semicolons:
`"Al_sg225.ncmat;density=2.6gcm3;temp=250K;comp=elas"`

- **Temperature:**

- `temp=100`, `temp=100K`, `temp=-10C`
- Default is usually room temperature (293.15K).

- **Density**

- `density=2.0gcm3`, `density=3.4kgm3`, `density=0.9x`.
- Only needed if you wish to override. Last example scales by factor of 0.9.

- **Scattering component toggling**

- `comp=inelas,sans` (only inelastic and SANS)
- `inelas=0` (without inelastic)
- `comp=coh_elas` (only coherent elastic)

- **Modify atomic compositions**

- `atomdb=H is D`
- `atomdb=Al:is:0.9:Al:0.1:Cr`
- `atomdb=Si29:28.97649466525u:4.7fm:0.001b:0.101b, ...`

- **Single crystal parameters**

- Single crystal models and orientations are primarily controlled by the parameters `mos`, `dir1`, `dir2`, and `dirtol`. Refer to the documentation linked above, and see examples in the notebooks.

Whitespace in cfg-strings is allowed (and ignored).

Docs built in: `$> nctool --doc --doc`

```
#Plot beryllium-oxide cross sections
import NCrystal as NC
import matplotlib.pyplot as plt
import numpy
scBeO = NC.createScatter('BeO_sg186.ncmat')
scBeO_nobragg = NC.createScatter('BeO_sg186.ncmat;bragg=0')
wls = numpy.linspace(0.0,7.0,1000)
plt.plot( wls, scBeO.xsect(wl=wls), label='All scattering' )
plt.plot( wls, scBeO_nobragg.xsect(wl=wls), label='No Bragg' )
plt.xlabel('Neutron wavelength (Å)')
plt.ylabel('cross section per atom (barn)')
plt.legend()
plt.show()
```

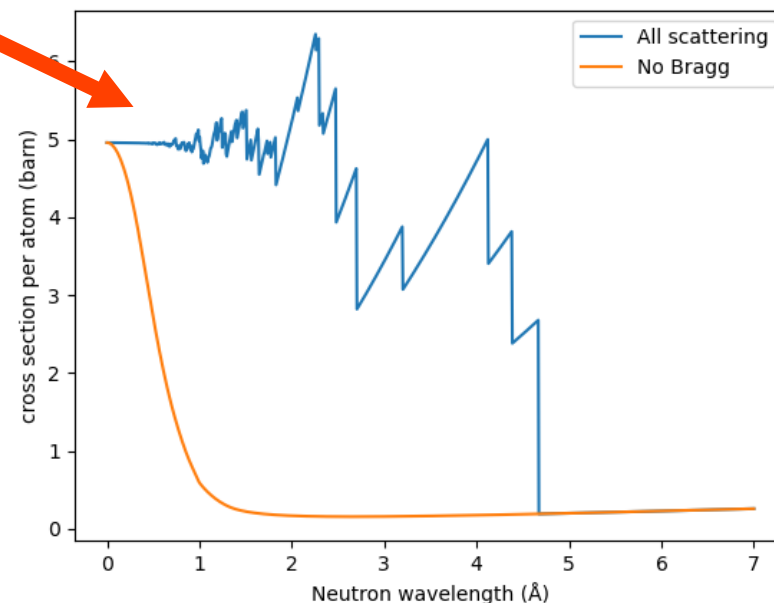
Simply use NCrystal cfg-strings
when creating materials



Jupyter tutorials available at:

<https://github.com/mctools/ncrystal-notebooks/>

**conda install -c conda-forge ncrystal
or pip install ncrystal**



Python API

Access detailed info



Jupyter tutorials available at:
<https://github.com/mctools/ncrystal-notebooks/>

NCrystal

Thermal Neutron Transport

Rich Python API provides many other features, in particular:

- Provides access to material information.
- Provides utilities for composing or understanding materials.

Small example showing how to access information about Bragg reflections (normals, d-spacings, structure factors):

```
import NCrystal as NC
#Can also extract more detailed info:
info_BeO = NC.createInfo('BeO_sg186.ncmat')
info_BeO.dump()
print('Density [g/cm3]: %g'%info_BeO.density)
print('Number density [atoms/Å³]: %g'%info_BeO.numberdensity)
for fraction,atom in info_BeO.composition:
    print(f'Has {fraction*100}% {atom}')
for hkl in [ hkl for hkl in info_BeO.hklObjects()
             if ( 1.1 < hkl.d < 1.17) ]:
    print(f'N={hkl.mult} d={hkl.d:g}Å F²={hkl.f2:g}barn',
          list(zip(hkl.h,hkl.k,hkl.l)))
```



```
Density [g/cm3]: 3.00836
Number density [atoms/Å³]: 0.144868
Has 50.0% Be=Be(cohSL=7.79fm cohXS=7.62579barn incXS=0.0018barn absXS=0.0076barn
Has 50.0% O=O(cohSL=5.803fm cohXS=4.2317barn incXS=0.0008barn absXS=0.00019barn m
N=6 d=1.16827Å F²=1.66027barn [(2, 0, 0), (2, -2, 0), (0, 2, 0)]
N=12 d=1.14858Å F²=3.49424barn [(2, -1, 2), (2, -1, -2), (1, 1, 2), (1, 1, -2), (
N=12 d=1.1288Å F²=0.772761barn [(2, 0, 1), (2, 0, -1), (2, -2, 1), (2, -2, -1), (
```

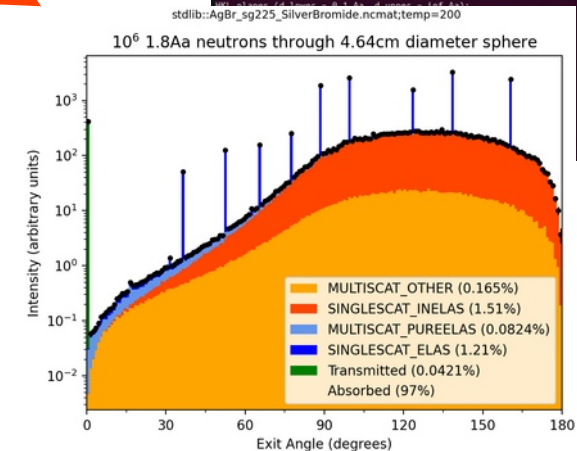
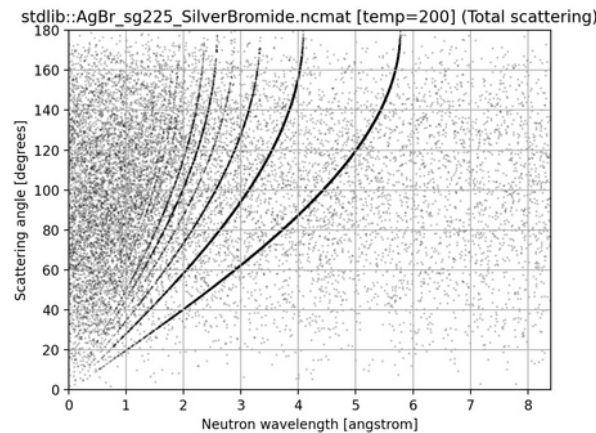
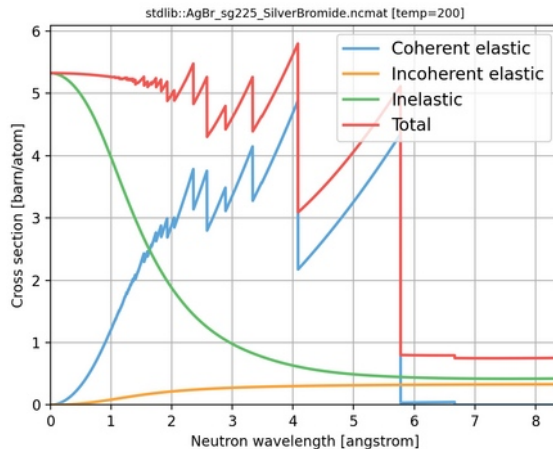
Command-line interface (CLI)

The `nctool` command allows inspection of physics resulting from a given `cfg-string`, browsing of data files, and more.

```
$> nctool --dump "AgBr_sg225_SilverBromide.ncmat;temp=200K"
```

```
$> nctool "AgBr_sg225_SilverBromide.ncmat;temp=200K"
```

```
..... NCrystal Material Info .....
Data source: stdlib::AgBr_sg225_SilverBromide.ncmat
.....
Density : 6.47734 g/cm3, 0.0415477 atoms/Aa^3
.....
Composition (by mole): 50% Br 50% Ag
Composition (by mass): 42.5535% Br 57.4465% Ag
.....
Atom data:
Br = Br(cohSL=6.795fm cohXS=5.8025barn incXS=0.1barn absXS=6.9barn mass=79.9035u Z=35)
Ag = Ag(cohSL=5.922fm cohXS=4.4878barn incXS=0.5barn absXS=63.3barn mass=107.868u Z=47)
.....
Averaged quantities:
Atomic mass : 93.8859u
Absorption XS at 2200m/s : 35.1 barn
Free scattering XS : 5.32549 barn
Scattering length density : 2.64181 10^-6/Aa^2
.....
State of matter: Solid (crystalline)
.....
Space group number : 225
Lattice spacings [Aa] : 5.7745 5.7745 5.7745
Lattice angles [deg] : 90 90 90
Unit cell volume [Aa^3] : 192.55
Atoms / unit cell : 8
.....
Atoms in unit cell (total 8):
4 Br atoms [T_Debye=123.347K, MSD=0.0241933Aa^2]
4 Ag atoms [T_Debye=183.372K, MSD=0.0254376Aa^2]
.....
Atomic coordinates:
Br      0      0      1/2
Br      0      1/2     0
Br      1/2     0      0
Br      1/2     1/2    1/2
Ag      0      0      0
Ag      0      1/2    1/2
Ag      1/2     0      1/2
Ag      1/2    1/2     0
.....
Dynamic info for Br (58K):
Type: S(alpha,beta) [From VDOS]
VDOS Source: 2030 points
VDOS E_max: 18.0877 meV
Dynamic info for Ag (58K):
Type: S(alpha,beta) [From VDOS]
VDOS Source: 2030 points
VDOS E_max: 18.0877 meV
.....
HKL info type: SymEqGroup
.....
```



Other scripts provided, mostly for conversions:
`ncrystal_cif2ncmat`, `ncrystal_endf2ncmat`,
`ncrystal_ncmat2hkl`, `ncrystal_hfg2ncmat`,
`ncrystal-config`, ...

The Python API provides everything provided by the cmdline scripts, but sometimes a quick command in the terminal is convenient.

Using NCrystal as a backend engine



NCrystal in McStas



Standard McStas example (.instr)

```
COMPONENT mysample = NCrystal_sample(cfg="Al203_sg167_Corundum.ncmat",
                                     radius=0.01, yheight=0.05)
AT (0, 0, 0) RELATIVE PREVIOUS
```

McStasScript + Union example (Python)

```
import mcstasscript.tools.ncrystal_union as ncunion
ncunion.add_ncrystal_union_material(instr,
                                   name="myAl",
                                   cfgstr="Al_sg225.ncmat;temp=10C")

#... usual mcstasscript code for creating volume "myvol" here
myvol.set_parameters(radius=0.01,
                    yheight=0.01,
                    material_string=' "myAl" ',
                    priority=1)
```

Simply use NCrystal cfg-strings
when creating materials

To use Union+NCrystal directly in .instr files, the
ncrystal_mcstasunion script easily generates
the required code (can be used with .instr
SHELL keyword for automated execution).

NOTE: This is admittedly a bit "hacky",
and we plan to improve this!!

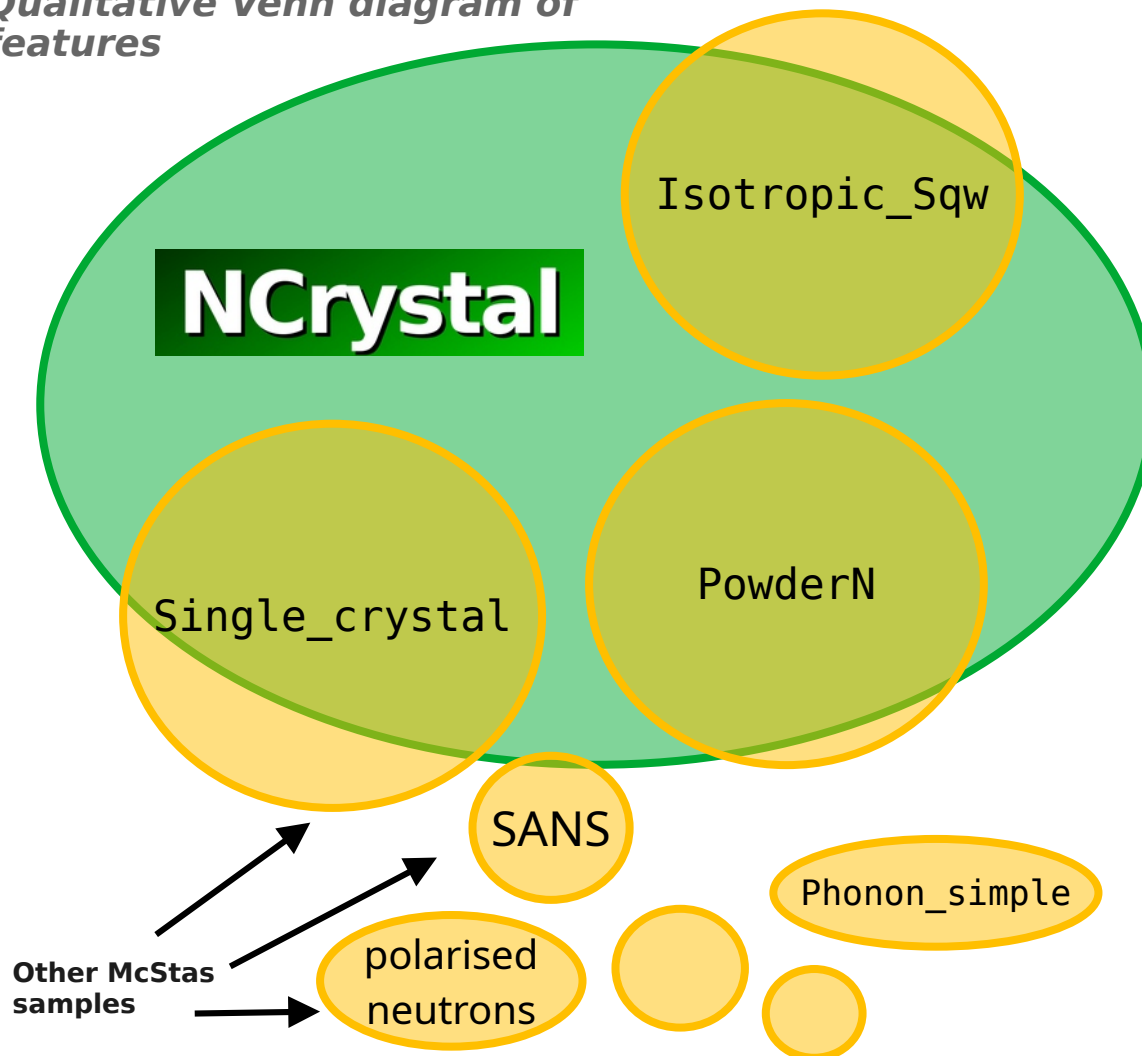
```
$> ncrystal_mcstasunion MyAl "Al_sg225.ncmat;temp=25C"
COMPONENT MyAl_ncrystal_proc = NCrystal_process(
  cfg = "Al_sg225.ncmat;temp=25C"
)
AT (0,0,0) ABSOLUTE

COMPONENT MyAl = Union_make_material(
  process_string = "MyAl_ncrystal_proc",
  my_absorption = 1.39136803716641
)
AT (0,0,0) ABSOLUTE
```


NCrystal versus other McStas “sample” components



*Qualitative Venn diagram of
features*



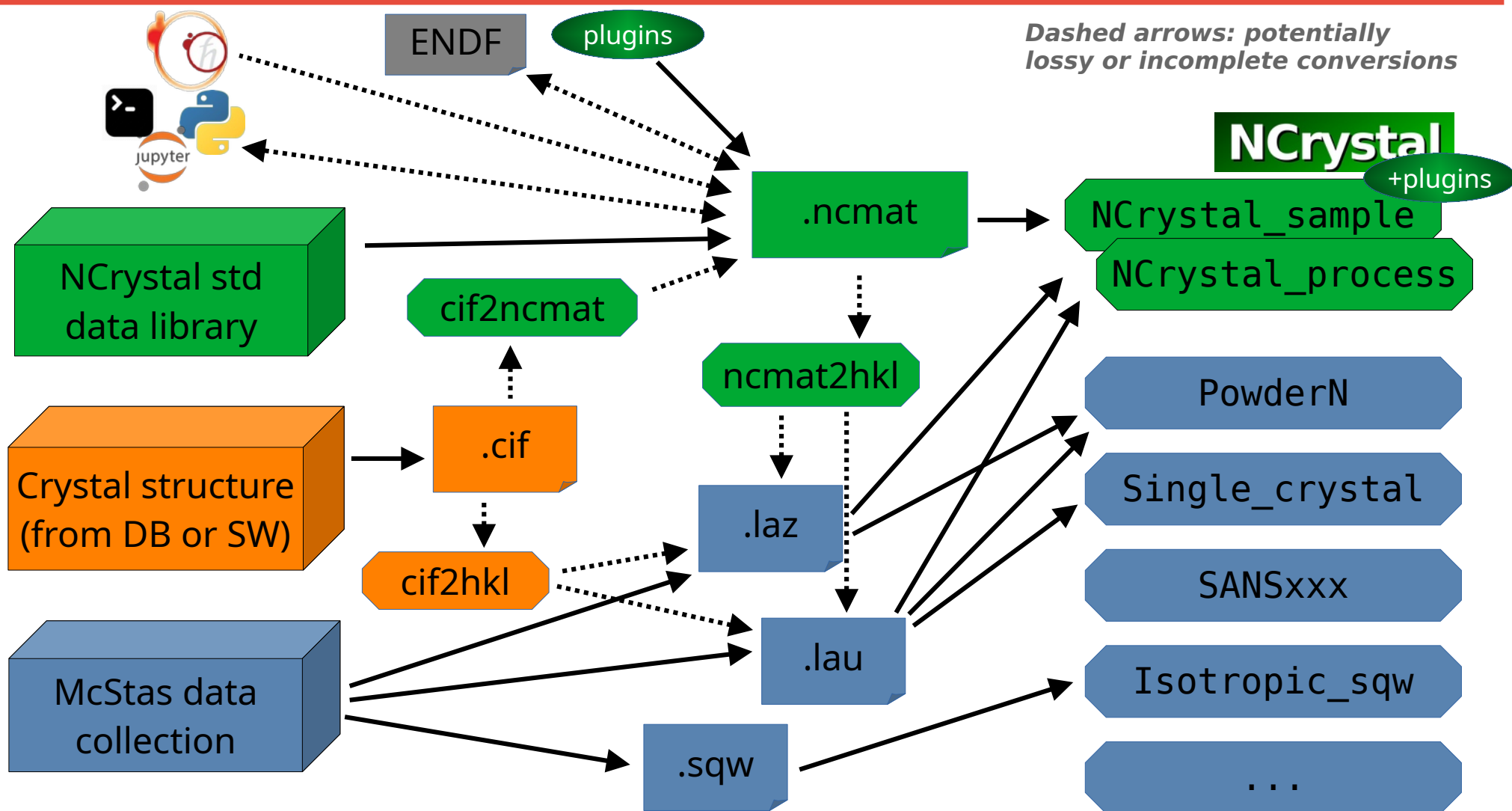
- **Some reasons to use NCrystal:**

- Standalone features for debugging, configuring, or creating materials.
- Aims to be a standard used across multiple codes (e.g. Vitess, OpenMC, Geant4, ...).
- Large pre-existing material library.
- Highly configurable materials (multi-phase support, easy to play with temperatures, enrichments, etc.).
- Inclusive total cross sections (always accounting for both inelastic/elastic and coherent/incoherent, multiple scattering) → More realistic transmission.
- In some cases NCrystal is only option for the physics models in McStas: HOPG, extinction, etc.
- ...

- **Missing in NCrystal:**

- Focusing options.
- GPU support (runs in McStas funnel mode)
- Surface physics
- Huge component library of McStas contains many physics models without analogues in NCrystal.
- ...

Materials in McStas eco-system

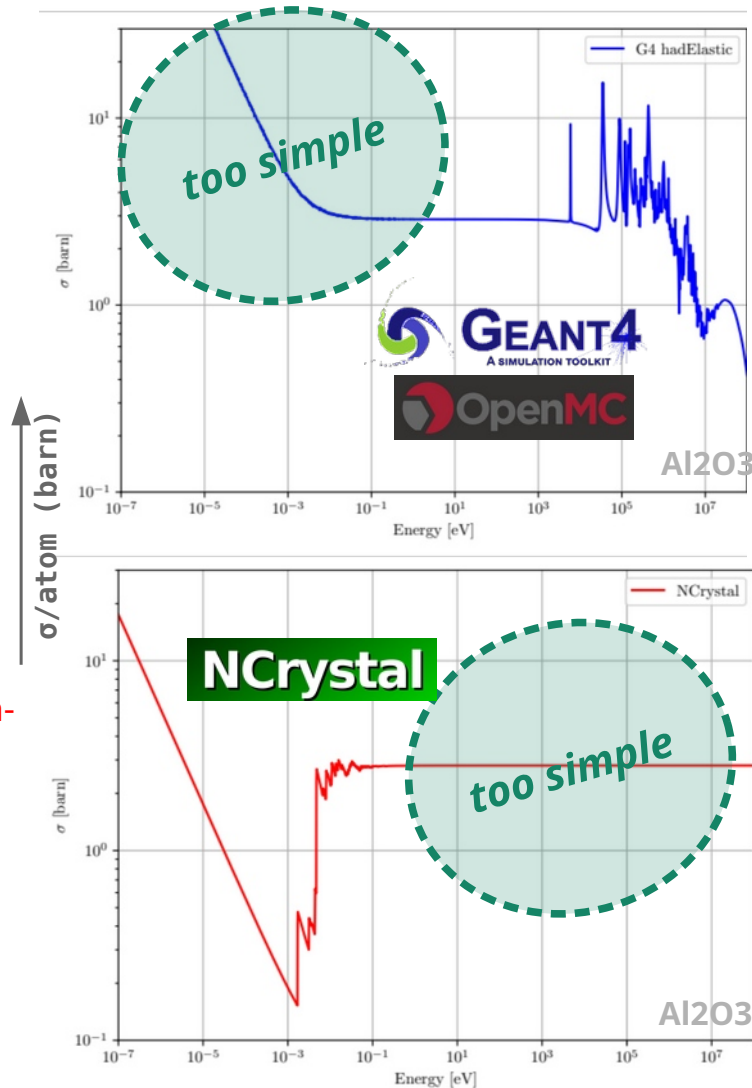


NCrystal + OpenMC/Geant4

Combining two regimes of neutron scattering

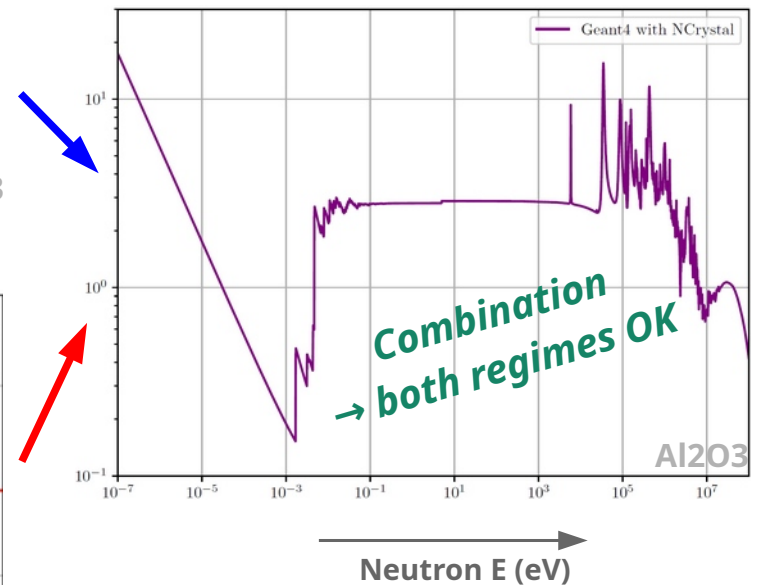
Higher energy ($\gg eV$):

- Complex neutron-nuclei interactions with energy dependent strength.
- Not sensitive to material structure.



Low energy ($\ll eV$):

- Simple (point-like) neutron-nuclei interactions with constant strength.
- Very sensitive to material structure.



NCrystal in OpenMC



```
import openmc
# Materials
openmc_mat = openmc.Material.from_ncrystal('Polyethylene_CH2.ncmat;temp=50C')
# Geometry
s1 = openmc.Sphere(r=10, boundary_type='vacuum')
c1 = openmc.Cell(region=-s1, fill=openmc_mat)
geometry = openmc.Geometry([c1])
# Execution settings
settings = openmc.Settings()
settings.source = openmc.Source(energy=openmc.stats.Discrete(x=[10.0], p=[1.0]))
settings.run_mode = 'fixed source'
settings.batches = 10
settings.particles = 10000
# Write xml files
model = openmc.model.Model(geometry=geometry, settings=settings)
model.export_to_xml()
#Now launch openmc
```

Simply use NCrystal cfg-strings
when creating materials

Currently needs OpenMC built with `-DOPENMC_USE_NCRYSTAL=0n`
Plan is to enable this in OpenMC conda packages.



NCrystal in Geant4



```
//Include the relevant header:  
#include "G4NCrystal/G4NCrystal.hh"
```

```
//...
```

```
//Create materials directly from cfg-strings:  
G4Material * mat_aluminium = G4NCrystal::createMaterial("Al_sg225.ncmat");
```

```
//...
```

```
//Currently the NCrystal process must then be injected in the following way:
```

```
g4runManager->Initialize();  
G4NCrystal::installOnDemand();  
g4runManager->BeamOn(1000)
```

Simply use NCrystal cfg-strings
when creating materials



Somewhat unusual initialisation
(alternative exist using the Geant4
biasing framework, supports MT mode)



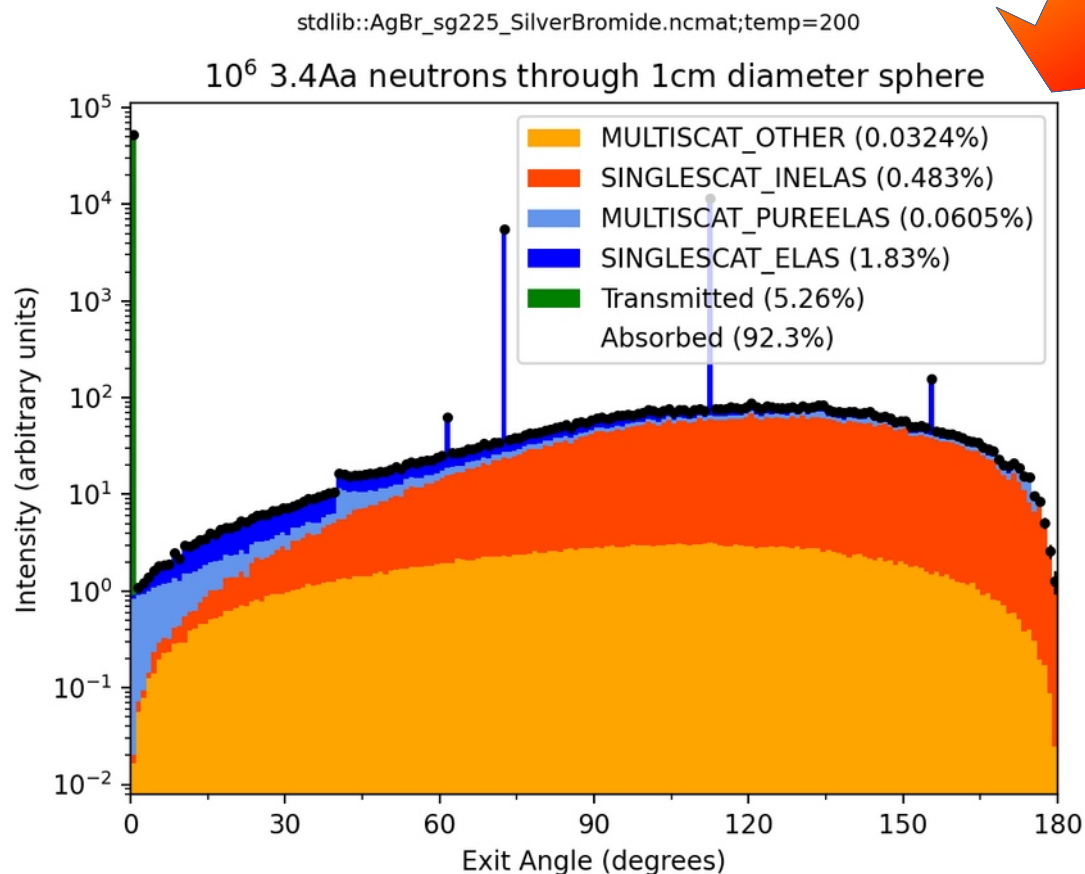
NB: Needs ncrystal-geant4 package installed.

Built-in “Mini MC” for scatter patterns with multiple-scattering effects (Sep 2024)

NCrystal

Thermal Neutron Transport

```
$> nctool --mc 3.4Aa 1cm 'stdlib::AgBr_sg225_SilverBromide.ncmat;temp=200K'
```



Runs multithreaded and vectorised, for fast results (ultimately intended for possible usage in diffraction analysis)

Time for this particular simulation on my laptop:

Load material: 140ms

Run simulation: 170ms

Total: 310ms

Intended usage:

- 1) Quickly debugging material models and cfg-strings.
- 2) Usage in data analysis loops.

Note: 10^6 src neutrons $\rightarrow >10^7$ tally entries due to some variance reduction tricks.

NCrystal material formats and data library

The NCMAT data format for materials

<https://github.com/mctools/ncrystal/wiki/NCMAT-format>

NCrystal

Thermal Neutron Transport

Sources:

- NCrystal data library
- Converted from (or to) other formats: ENDF, CIF, .laz, .lau, ...
- NCMATComposer
- Generated on-the-fly by NCrystal plugins
- Written by tools
- Written by users

```
NCMAT v7 — Ge_sg227.ncmat
#Comments here (not shown)
@CELL
  cubic 5.65735
@SPACEGROUP
  227
@ATOMPOSITIONS
  Ge 1/2 1/2 1/2
  Ge 1/4 1/4 3/4
  Ge 1/2 0 0
  Ge 1/4 3/4 1/4
  Ge 0 1/2 0
  Ge 3/4 1/4 1/4
  Ge 0 0 1/2
  Ge 3/4 3/4 3/4
@DYNINFO
  element Ge
  fraction 1
  type vdos
  vdos_egrid .0037709 .0377897
  vdos_density .0064216 .0067183 .0070150
  .0074164 .0078883 .0078611 .0078339
  # (~50 lines here not shown)
  .032778 .02264 .012923 .002484 0
```

Native NCrystal
format (static or
dynamic)

- NCrystal supports multiple input formats
- But NCMAT (.ncmat) is the native one.
- Can load from on-disk file, memory buffers, data generated by plugins, ...

Optional formats
added for McStas
community

.ncmat loader

.nxs loader

.laz/.lau loader

Info

inelastic

Bragg
(powder)

Bragg
(single crystal)

Incoherent
elastic

Absorption

NCrystal data library

Browse with performance plots at: <https://github.com/mctools/ncrystal/wiki/Data-library>

NCrystal

Thermal Neutron Transport

AcrylicGlass_C502H8.ncmat
AgBr_sg225_SilverBromide.ncmat
Ag_sg225.ncmat
Al2O3_sg167_Corundum.ncmat
Al4C3_sg166_AluminiumCarbide.ncmat
AlN_sg186_AluminumNitride.ncmat
Al_sg225.ncmat
Ar_Gas_STP.ncmat
Au_sg225.ncmat
BaF2_sg225_BariumFluoride.ncmat
BaO_sg225_BariumOxide.ncmat
Ba_sg229.ncmat
Be3N2_sg206_BerylliumNitride.ncmat
BeF2_sg152_Beryllium_Fluoride.ncmat
BeO_sg186.ncmat
Be_sg194.ncmat
Bi_sg166.ncmat
CaCO3_sg62_Aragonite.ncmat
CaF2_sg225_CalciumFluoride.ncmat
CaH2_sg62_CalciumHydride.ncmat
CaOH2_sg164_CalciumHydroxide.ncmat
CaO_sg225_CalciumOxide.ncmat
Ca_sg225.ncmat
Ca_sg229_Calcium-gamma.ncmat
CaSiO3_sg2_Wollastonite.ncmat
CeO2_sg225_CeriumOxide.ncmat
Cr_sg229.ncmat
C_sg194_pyrolytic_graphite.ncmat
C_sg227_Diamond.ncmat
Cu2O_sg224_Cuprite.ncmat
Cu_sg225.ncmat
Dy2O3_sg206_DysprosiumOxide.ncmat
Epoxy_Araldite506_C18H20O3.ncmat
Fe_sg225_Iron-gamma.ncmat
Fe_sg229_Iron-alpha.ncmat
GaN_sg186_GalliumNitride.ncmat
GaSe_sg194_GalliumSelenide.ncmat
Ge3Bi4O12_sg220_BismuthGermanate.ncmat
Ge_sg227.ncmat
He_Gas_STP.ncmat
HfO2_sg14_HafniumOxide.ncmat
Ho2O3_sg206_HolmiumOxide.ncmat
Kapton_C22H10N2O5.ncmat
KBr_sg225_PotassiumBromide.ncmat
KF_sg225_PotassiumFluoride.ncmat
KOH_sg4_PotassiumHydroxide.ncmat
Kr_Gas_STP.ncmat
K_sg229.ncmat
LaBr3_sg176_LanthanumBromide.ncmat
Li2O_sg225_LithiumOxide.ncmat
Li3N_sg191_LithiumNitride.ncmat
LiF_sg225_LithiumFluoride.ncmat
LiH_sg225_LithiumHydride.ncmat
LiquidHeavyWaterD2O_T293.6K.ncmat
LiquidWaterH2O_T293.6K.ncmat
Lu2O3_sg206_LutetiumOxide.ncmat
Lu2SiO5_sg15.ncmat
Mg2SiO4_sg62_MagnesiumSilicate.ncmat
MgAl2O4_sg227_MAS.ncmat
MgCO3_sg167_MagnesiumCarbonate.ncmat
MgO2_sg136_MagnesiumDeuteride.ncmat
MgF2_sg136_MagnesiumFluoride.ncmat
MgH2_sg136_MagnesiumHydride.ncmat
MgOH2_sg164_MagnesiumHydroxide.ncmat
MgO_sg225_Periclase.ncmat
Mg_sg194.ncmat
Mo_sg229.ncmat
Na4Si3Al3O12Cl_sg218_Sodalite.ncmat
NaBr_sg225_SodiumBromide.ncmat
NaCl_sg225_SodiumChloride.ncmat
NaF_sg225_SodiumFluoride.ncmat
NaI_sg225_SodiumIodide.ncmat
Na_sg229.ncmat
Nb_sg229.ncmat
Ne_Gas_STP.ncmat
Ni_sg225.ncmat
Nylon11_C11H21NO.ncmat
Nylon12_C12H23NO.ncmat
Nylon610_C16H30N2O2.ncmat
Nylon66or6_C12H22N2O2.ncmat
PbF2-beta_sg225_BetaLeadFluoride.ncmat
PbO-alpha_sg129_Litharge.ncmat
PbO-beta_sg57_Massicot.ncmat
Pb_sg225.ncmat
PbS_sg225_LeadSulfide.ncmat
Pd_sg225.ncmat
PEEK_C19H12O3.ncmat
Polycarbonate_C16O3H14.ncmat
Polyester_C10H8O4.ncmat
Polyethylene_CH2.ncmat
Polylactide_C3H4O2.ncmat
Polypropylene_C3H6.ncmat
Polystyrene_C8H8.ncmat
Pt_sg225.ncmat
PVC_C2H3Cl.ncmat
Rb_sg229.ncmat
Rubber_C5H8.ncmat
Sc_sg194.ncmat
SiC-beta_sg216_BetaSiliconCarbide.ncmat

SiO2-alpha_sg154_AlphaQuartz.ncmat
SiO2-beta_sg180_BetaQuartz.ncmat
Si_sg227.ncmat
Sn_sg141.ncmat
SrF2_sg225_StrontiumFluoride.ncmat
SrH2_sg62_StrontiumHydride.ncmat
Sr_sg225.ncmat
Th3N4_sg166_ThoriumNitride.ncmat
ThO2_sg225_ThoriumDioxide.ncmat
Th_sg225.ncmat
TiO2_sg136_Rutile.ncmat
TiO2_sg141_Anatase.ncmat
Ti_sg194.ncmat
TlBr_sg221_ThalliumBromide.ncmat
Tm2O3_sg206_ThuliumOxide.ncmat
UF6_sg62_UraniumHexafluoride.ncmat
UO2_sg225_UraniumDioxide.ncmat
void.ncmat
V_sg229.ncmat
W_sg229.ncmat
Xe_Gas_STP.ncmat
Y2O3_sg206_YttriumOxide.ncmat
Y2SiO5_sg15_YSO.ncmat
Y3Al5O12_sg230_YAG.ncmat
Y_sg194.ncmat
ZnF2_sg136_ZincFluoride.ncmat
ZnO_sg186_ZincOxide.ncmat
Zn_sg194.ncmat
ZnS_sg216_Sphalerite.ncmat
ZrF4-beta_sg84.ncmat
ZrO2_sg137_Zirconia.ncmat
ZrO2_sg14_Zirconia.ncmat
Zr_sg194.ncmat

134 materials (v4.2.10):
Crystals (110), amorphous
solids (16), liquids, gasses, ...

Easy universal cfg:
"Al_sg225.ncmat;temp=250K"
"Rubber_C5H8.ncmat;comp=inelas"
- Same physics in all applications!
- Cfg variables documented at:
github.com/mctools/ncrystal/wiki/CfgRefDoc

Small (few kB) file sizes:
- Entire collection normally embedded
in NCrystal shared library.

Easy to create more:

- Hand-write NCMAT file (human readable ASCII, format well-defined & versioned) or use NCMATComposer
- Convert from ENDF, CIF, online crystal DB carbohydrate chemical formula, Quantum Espresso output, ...
- Request help on github.com/mctools/ncrystal

Can be converted to other formats:

- To .laz/.lau for McStas (via ncmat2hkl)
- To ENDF (via ncmat2endf)
- But limited by target format physics capabilities!

Quick one-liner materials

Especially useful for gas mixtures and highly absorbing materials

- Some materials are for many purposes adequately defined without an actual data file, and allows users to simply provide a suitable short string instead of a filename.
- Behind the scenes NCrystal (built-in) plugins analyse the string and constructs intermediate NCMAT data accordingly.
- Examples:

```
"gasmix::0.72xC02+0.28xAr/massfractions/1.5atm/250K"
```

```
"gasmix::0.7xC02+0.3xAr/0.001relhumidity"
```

```
"gasmix::0.7xC02+0.3xAr/1.5atm/250K"
```

```
"gasmix::BF3/2atm/25C/B_is_0.95_B10_0.05_B11"
```

```
"gasmix::C02"
```

```
"gasmix::He/1.64kgm3"
```

```
"gasmix::He/10bar"
```

```
"gasmix::He/He_is_He3/10bar"
```

```
"gasmix::air"
```

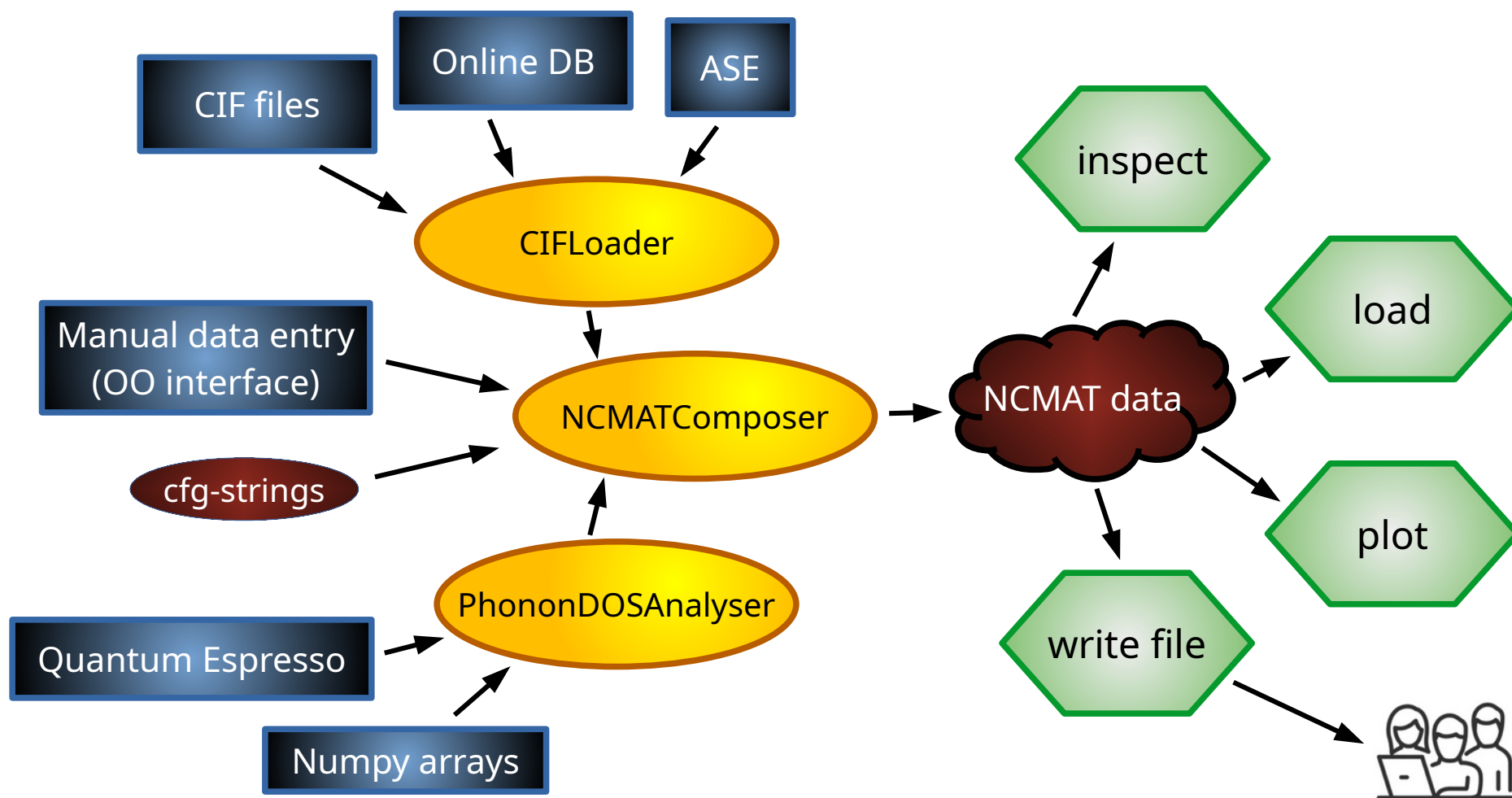
```
"gasmix::air/-10C/0.8atm/0.30relhumidity"
```

```
"solid::Gd203/7.07gcm3/Gd_is_0.9_Gd157_0.1_Gd155"
```

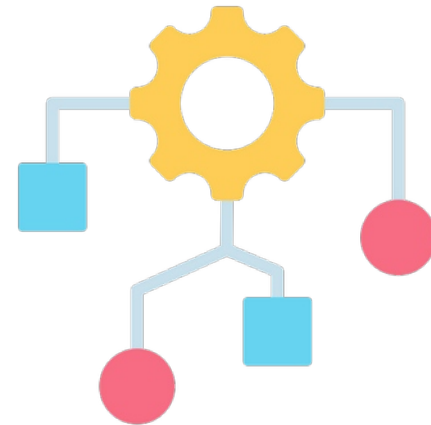
Python tools for material composition

Note Jupyter notebooks at <https://github.com/mctools/ncrystal-notebooks>

- NCrystal includes Python tools for creating new materials.

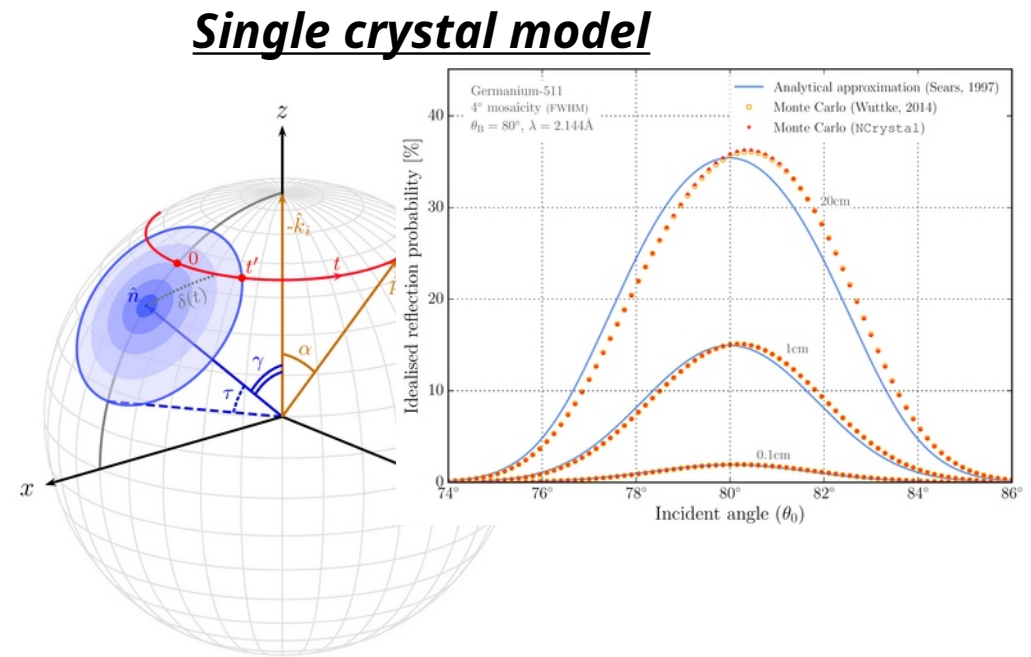
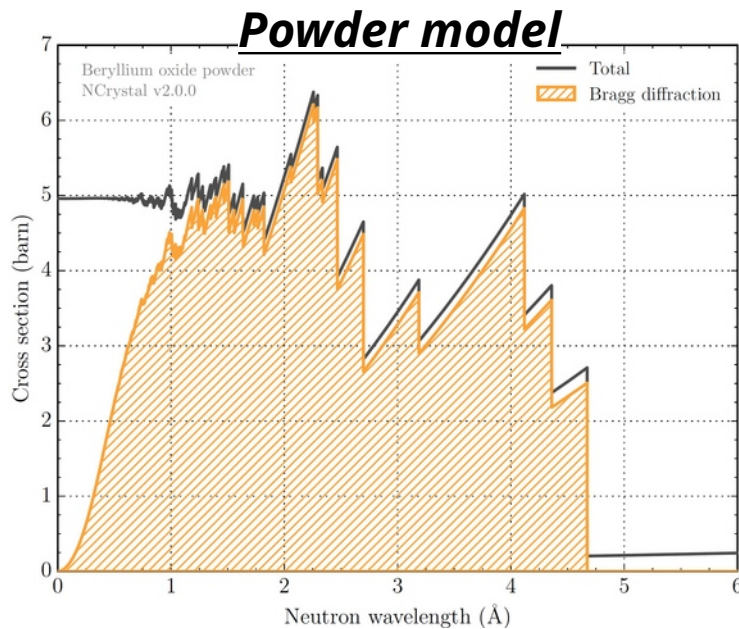


NCrystal physics algorithms (highlights)



Bragg diffraction

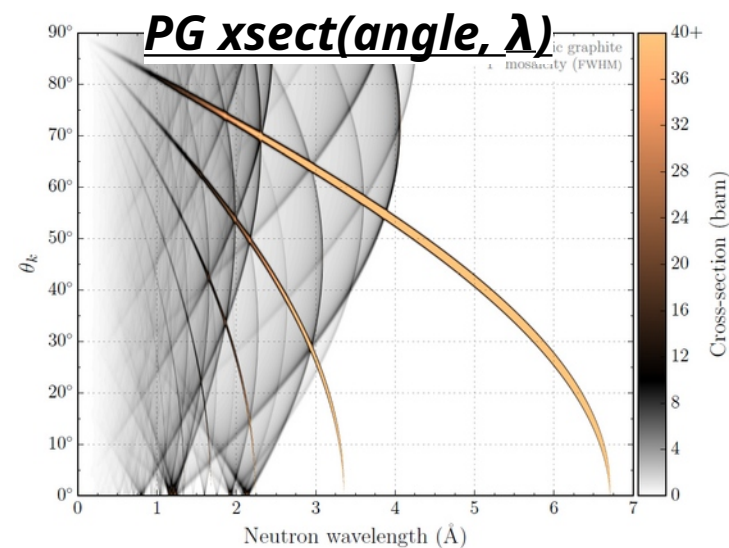
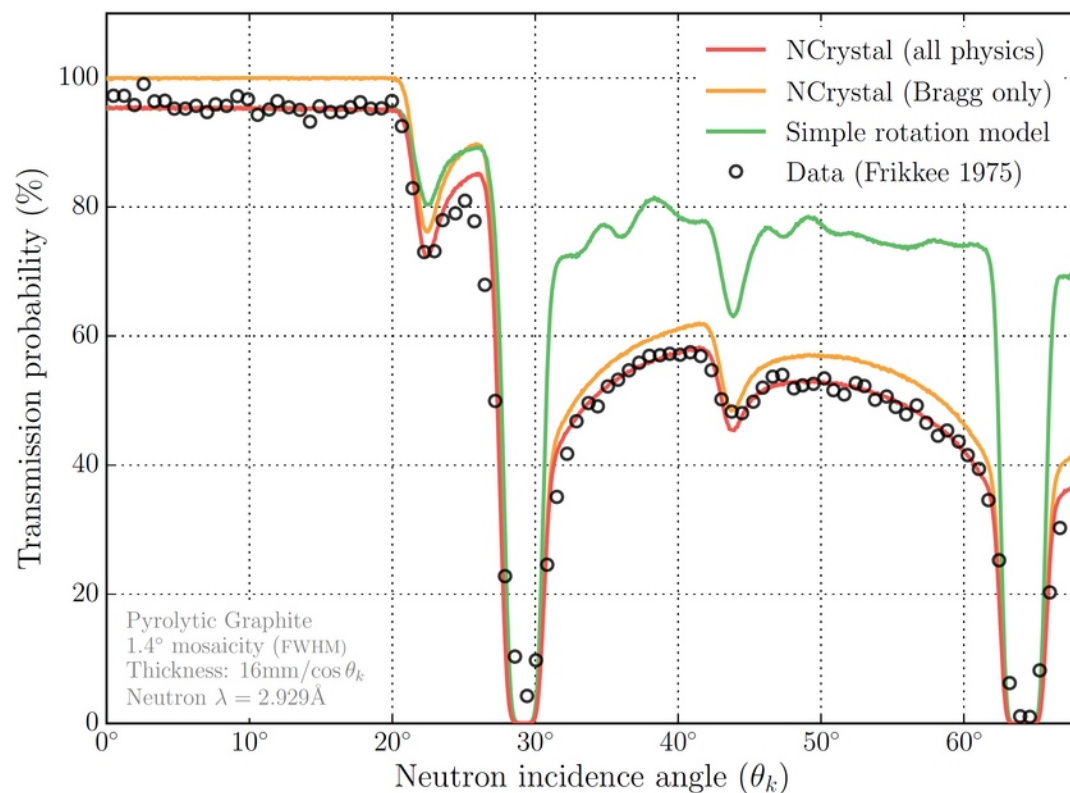
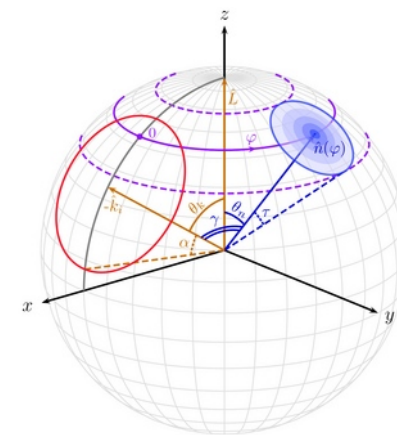
- HKL factors **initialised on-the-fly** from unit cell (<50ms).
 - Includes enough low-d-spacing planes to ensure correct transmission at shorter-wavelengths.
- Contains **power model**, **Gaussian mosaic single crystal model**, and **single crystal model for pyrolytic graphite**.
 - Single crystal models support back-scattering and any size of mosaic spread.
- Plugins for **texture** and **primary extinction** effects exist (to be adopted for NCrystal core).





Special single crystal model for pyrolytic graphite

Tricky model to implement, but
can actually reproduce
transmission data!



Inelastic physics

For now: isotropic materials + incoherent approximation.

NCrystal

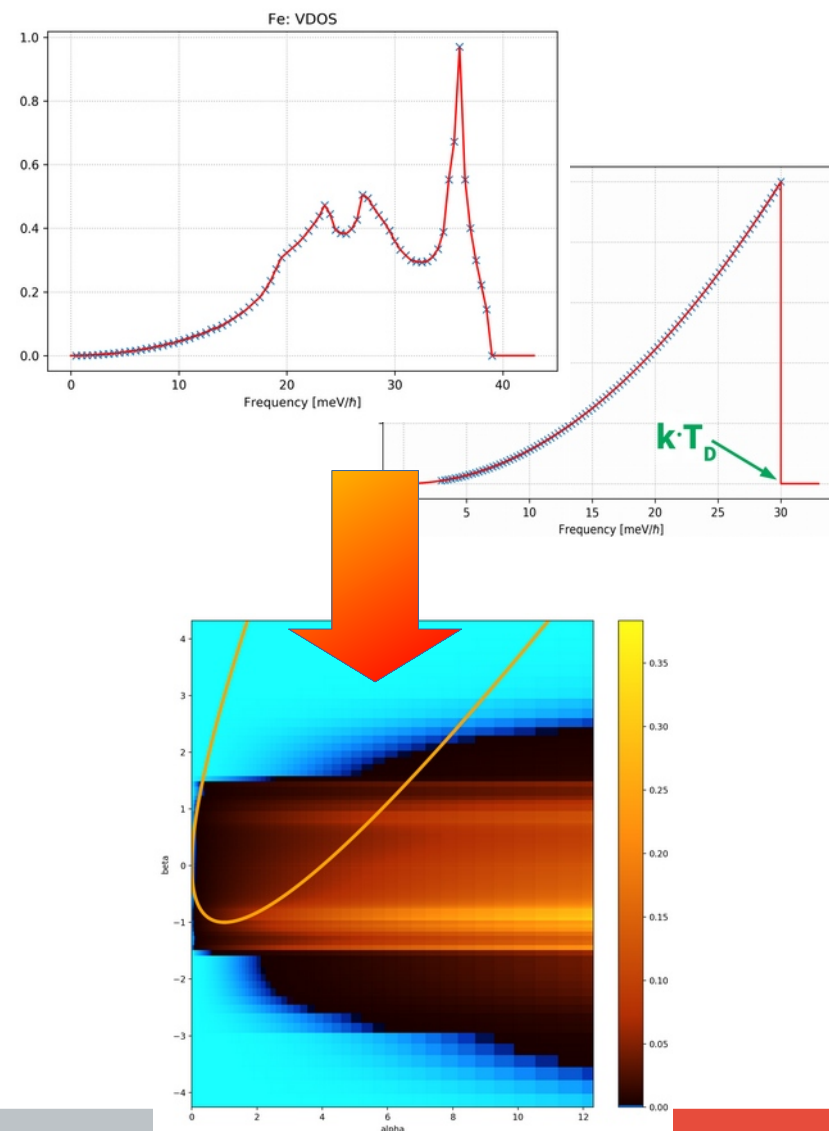
Thermal Neutron Transport

Based on phonon spectrums and scattering kernels

- Input is usually 1D phonon spectrums (VDOS). Taken from Debye Model if precise spectrum unknown.
- Precomputed kernels can be used, to support additional physics (liquids, coherent phonons, ...)

On-demand expansion of VDOS to 2D kernel

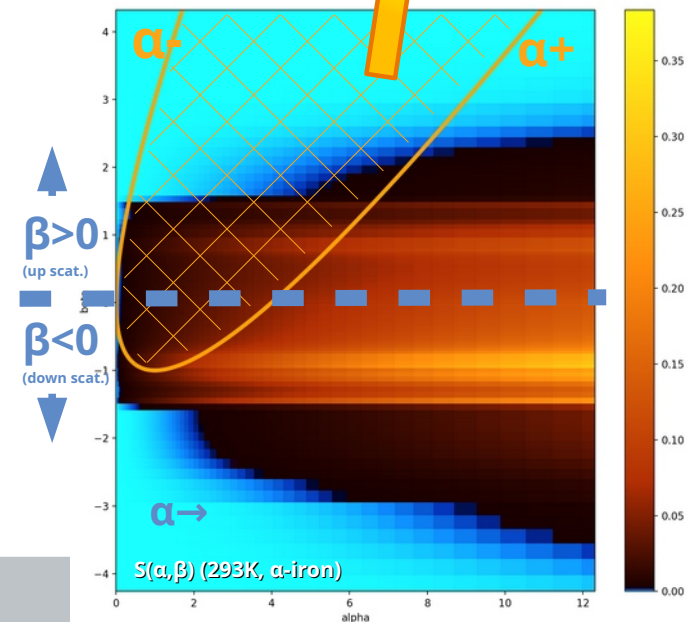
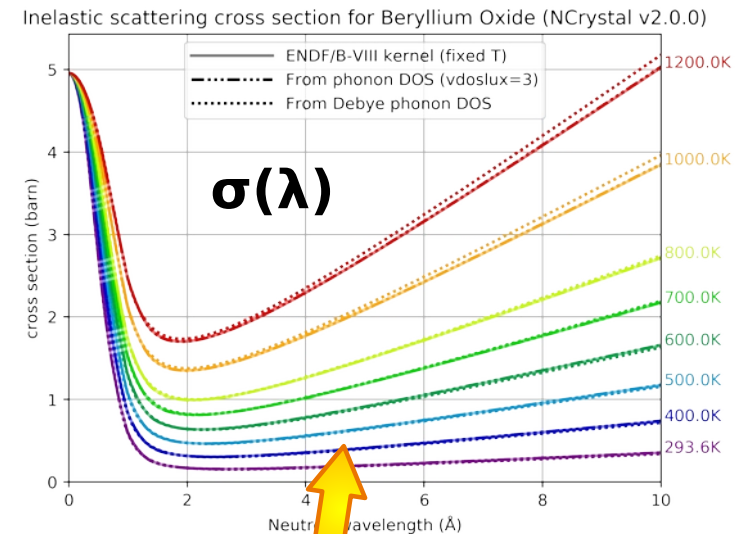
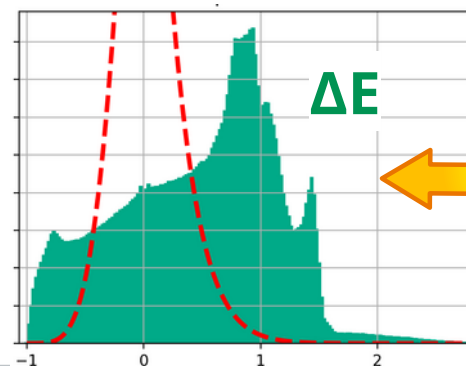
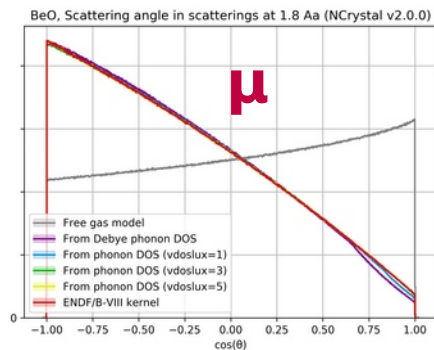
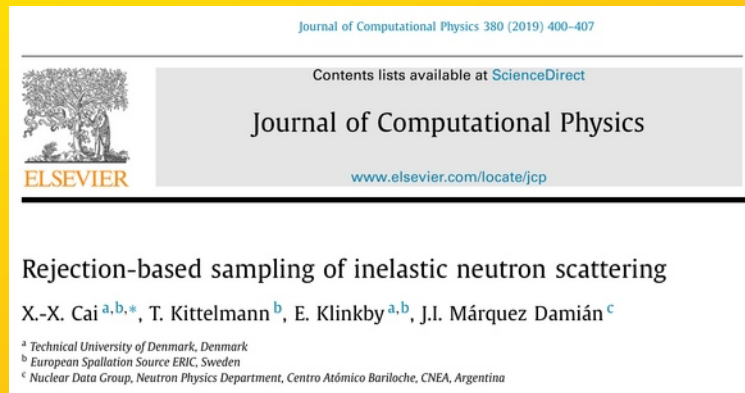
- Custom implementation of Sjölander 1958 method for multi-phonon effects.
- Expansion is temperature dependent.
- Similar to LEAPR, but our custom algorithms runs on-demand in $O(<100\text{ms})$, adapts grid to spectrum features, and covers multi-eV range.
- A single few-kilobyte NCMAT file can support any temperature, no need for megabyte files with fixed temperature selection



Inelastic physics: Using the 2D scattering kernels

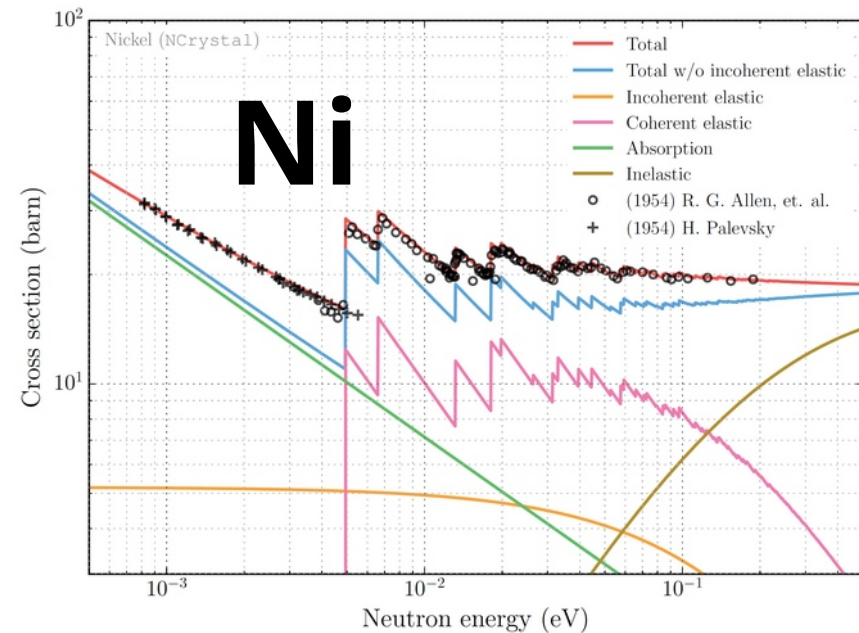
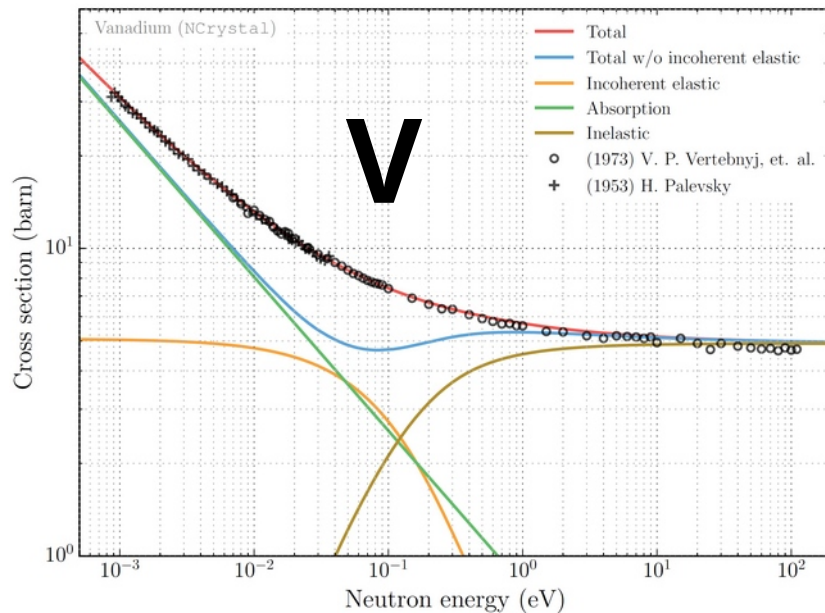
Custom algorithms for using the 2D kernels

- neutron total cross sections $\sigma(k_f)$
- precise MC sampling of (q, ω) for $k_i \rightarrow k_f$



Incoherent-elastic model

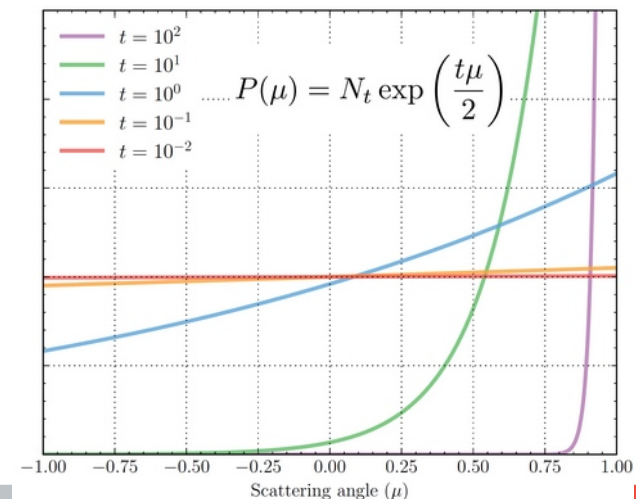
Simple if you know the Debye-Waller factors



- Physics described by **sum of Debye-Waller factors**.
- Governed by ratio between neutron wavelength and atomic displacements:

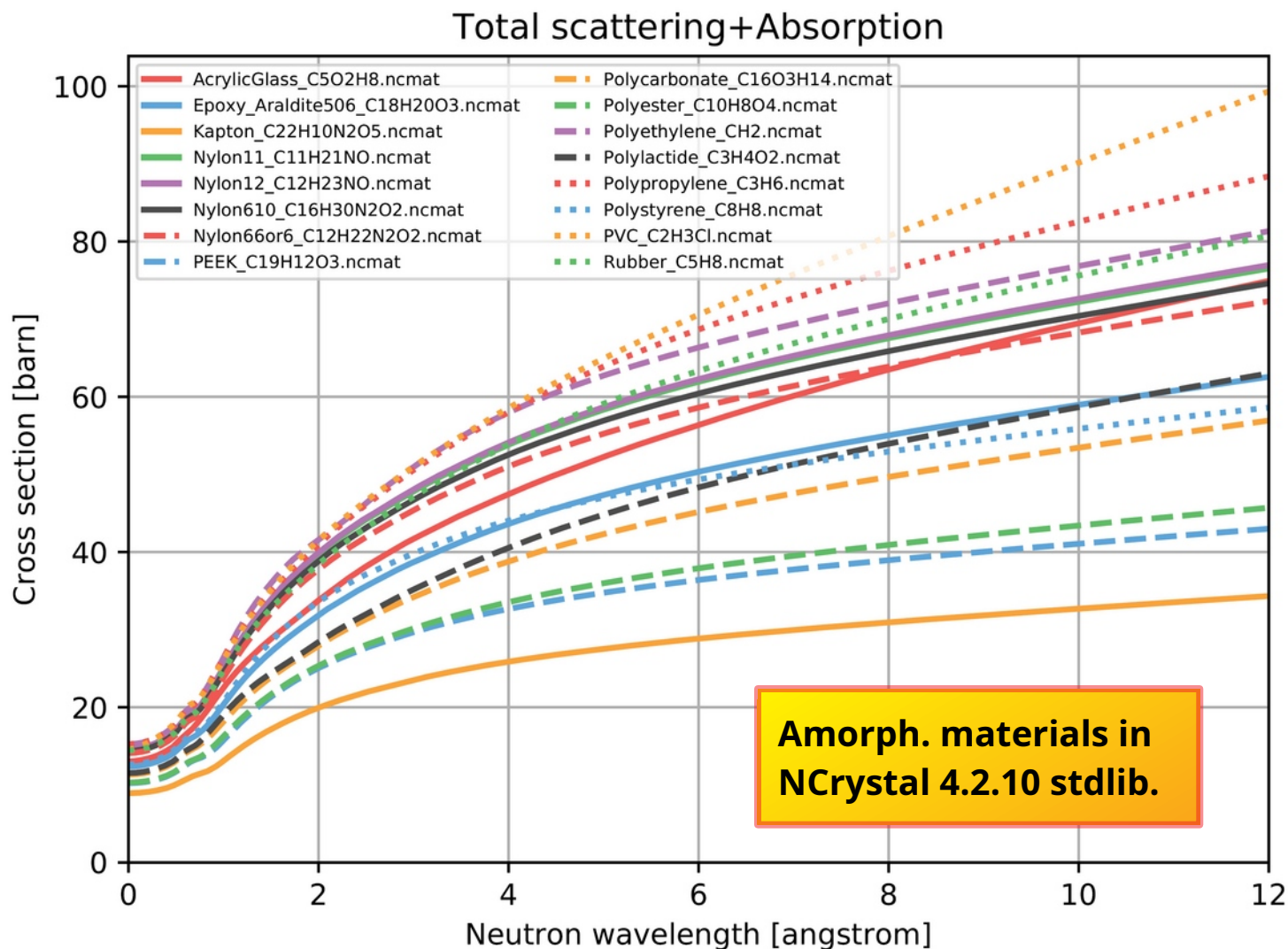
$$t \equiv (2k\delta)^2 = \left(\frac{4\pi\delta}{\lambda}\right)^2$$

$$\sigma^{\text{inc,el}}(k) = \sigma_{\text{inc}} \frac{1 - \exp(-t)}{t}$$



Amorphous materials

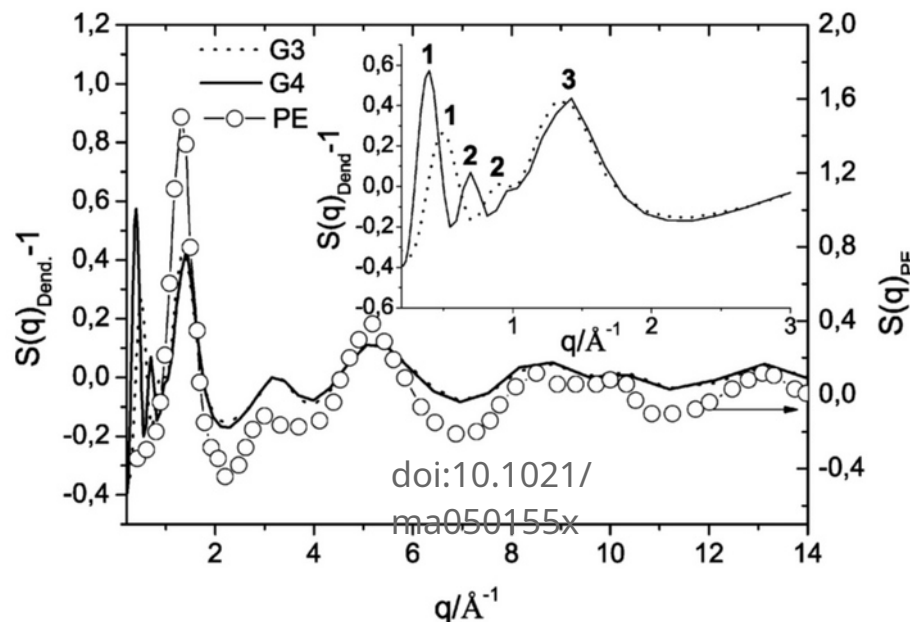
Modelled under incoherent approximation, suitable for hydrogen-rich materials.



- Polyethylene and AcrylicGlass (a.k.a. Plexiglass/Lucite) based on VDOS from other sources.
- 14 others based on ncrystal_hfg2ncmat script (AFGA).
- Let us know if we are missing something useful!

WIP: Coherent effects in amorphous materials (static structure factors)

- We plan to eventually also optionally include static structure factors $S(q)$ in our treatment of amorphous solids, but for now they are modelled under the *incoherent approximation*.
- The *incoherent approximation* is very good for esp. hydrogen-rich materials, but amorphous materials without strong incoherent cross sections might suffer in realism currently.



NCrystal development at ESS was recently chosen to be supported by the APRENDE EURATOM grant. This will hopefully allow us to include these effects for increased realism.

The tricky part is to include them along with the current inelastic kernel, without ending up with double-counting (likely we will use Sköld's method).

Easily model any hydrogen-rich amorphous solids (CLI tool)

- DFT/MD modelling of amorphous materials can be difficult and time consuming.
- Recent paper (Romanelli et. al., 2021) provides trustworthy and cheap alternative for hydrogen-rich materials.
- Relies on universality of hydrogen vibrations in different materials: Overall hydrogen VDOS can be composed from list of hydrogen bindings.
- We provide script for setting up NCMAT files with this.

Journal of Physics: Condensed Matter

PAPER

Thermal neutron cross sections of amino acids from average contributions of functional groups

Giovanni Romanelli¹ , Dalila Onorati^{2,3} , Pierfrancesco Ulpiani³ , Stephanie Cancelli⁴, Enrico Perelli-Cippo⁴, José Ignacio Márquez Damián⁵, Silvia C Capelli¹, Gabriele Croci^{4,6}, Andrea Muraro⁶, Marco Tardocchi⁶ [Show full author list](#)

Published 31 May 2021 • © 2021 IOP Publishing Ltd

[Journal of Physics: Condensed Matter](#), Volume 33, Number 28

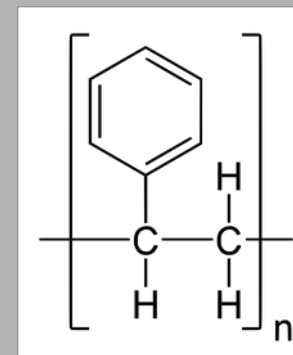
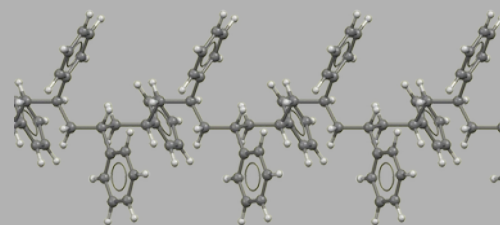
Citation Giovanni Romanelli et al 2021 *J. Phys.: Condens. Matter* **33** 285901

DOI 10.1088/1361-648X/abfc13

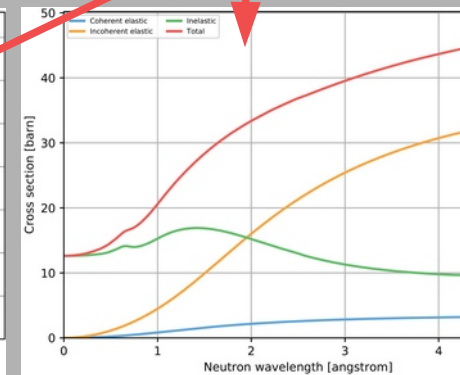
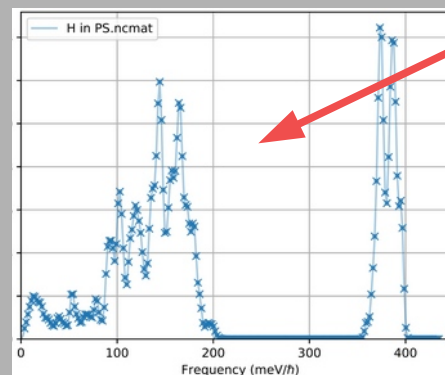
[References](#) ▼

Example (polystyrene):

- 1 aromatic ring with 5 H
- 1 CH₂ group
- 1 aliphatic CH binding

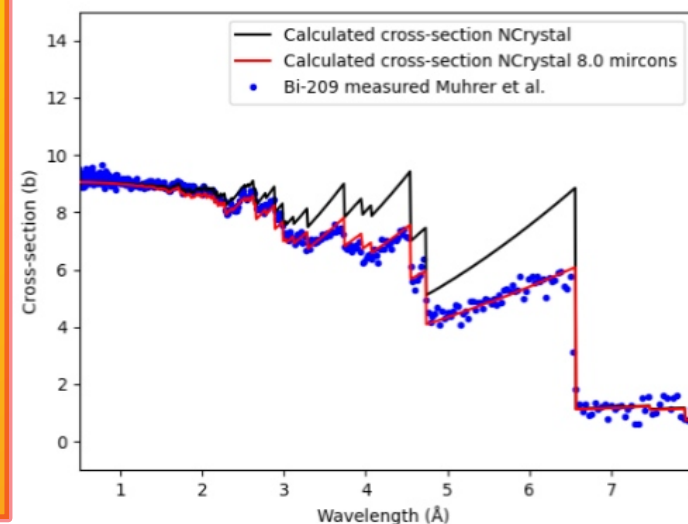


```
$> ncrystal_hfg2ncmat --formula C8H8 \  
--spec 5xCharo+1xCHali+1xCH2 \  
--density 0.99 -o PS.ncmat
```



WIP: Extinction models

- Large mosaic blocks can lead to breakdown of kinematic diffraction model due interference between scattered and direct neutron wave.
- Affects both powders and mosaic single crystals.
- Lots of work ongoing in NCrystal team (in particular by D. DiJulio, S. Xu, and recently T. Kittelmann), to investigate.
- Some models available as (CrysExtn) plugin, but plan is to integrate as built-in NCrystal model ~this year.
- At minimum first release will support primary + secondary extinction models by Becker-Coppens (1974) and Sabine (2006).



Impact of extinction effects on neutron transmission in solid beryllium metal

S. Xu,^{a*} D. D. DiJulio,^{a*} J. I. Márquez Damian,^a S. C. Vogel,^b A. M. Long,^b T. Y.
Hirsh,^c T. Kittelmann,^a V. Kuksenko^d and G. Muhrer^a

^aEuropean Spallation Source ERIC, Partikelgatan 5, 224 84 Lund, Sweden, ^bMaterials Science and Technology Division,
Los Alamos National Laboratory, Los Alamos, NM 87545, USA, ^cSoreq Nuclear Research Center, 81800, Yavne, Israel,
and ^dUK Atomic Energy Authority, Culham Science Centre, Abingdon, Oxon OX14 3DB, UK. *Correspondence e-mail:
shuqi.xu@ess.eu, douglas.dijulio@ess.eu

Time-of-flight neutron transmission and diffraction measurements were performed on four solid beryllium metal grades using the HIPPO neutron time-of-flight diffractometer equipped with an energy-resolved neutron imaging detector at Los Alamos National Laboratory. Analysis of the extracted total neutron cross sections from the transmission measurements revealed a reduction in the coherent elastic scattering component compared with the values predicted for an ideal polycrystalline material. This reduction could be explained by a lattice-plane-dependent extinction model, which underscores the

1 Revisiting Becker-Coppens (1974): Updated Recipes 2 for Estimating Extinction Factors in Spherical 3 Crystallites

4 Thomas Kittelmann^a, Douglas D. DiJulio^b, Shuqi Xu^b, and J. I. Márquez
5 Damián^b

6 ^aESS Data Management and Scientific Computing Division, Lyngby, Denmark
7 ^bEuropean Spallation Source ERIC, Lund, Sweden

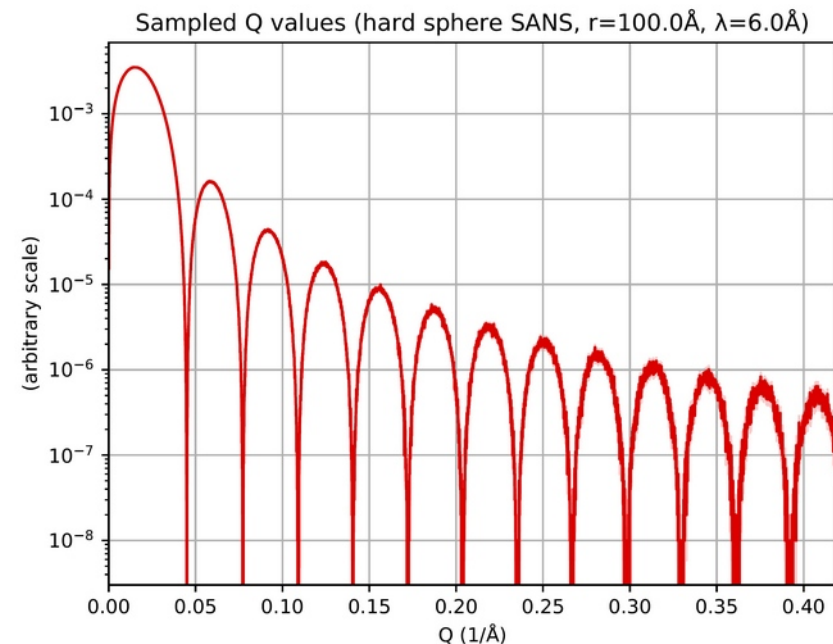
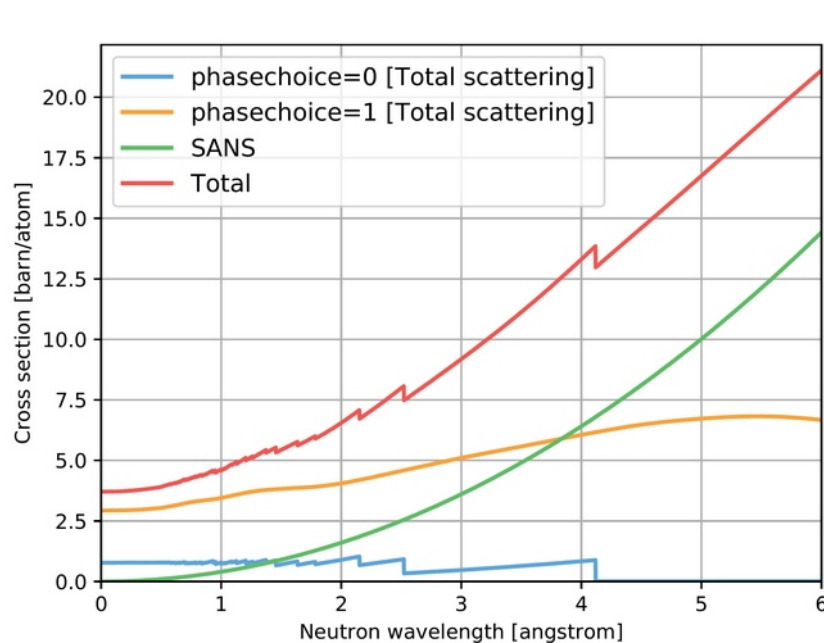
8 Synopsis

9 A technical update of the Becker-Coppens (1974) recipe for estimating extinction factors
10 in spherical crystallites is provided, addressing limitations in scenarios with higher
11 Bragg angles or stronger extinction effects, as well as providing improved precision in
12 general. The work utilises modern computing capabilities and improved recipes are
13 provided for easy community adoption.

SANS (WIP except for hard-spheres)

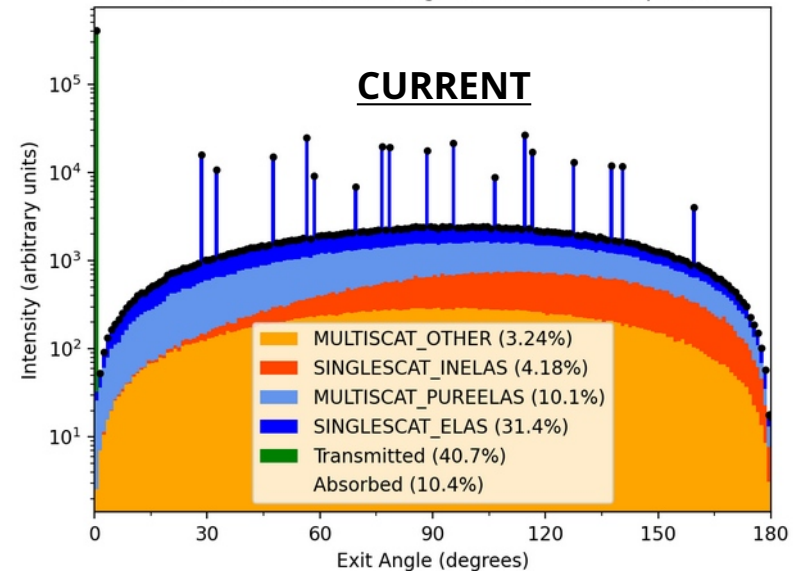
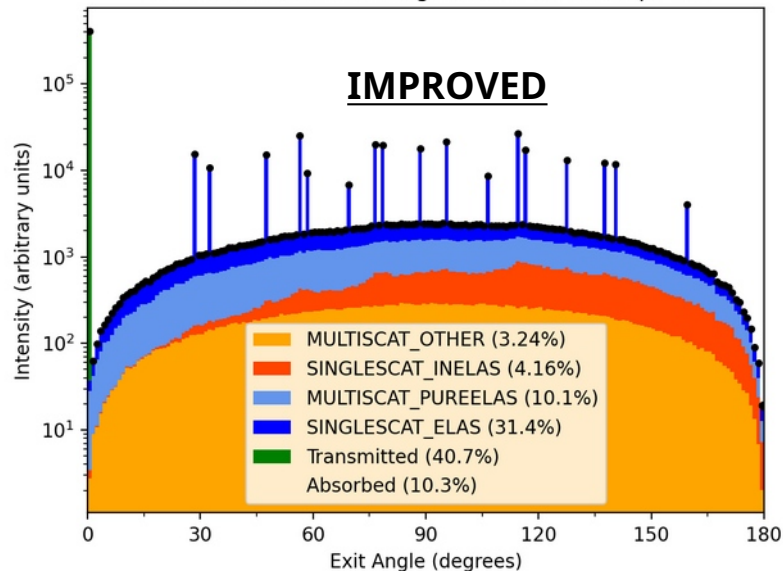
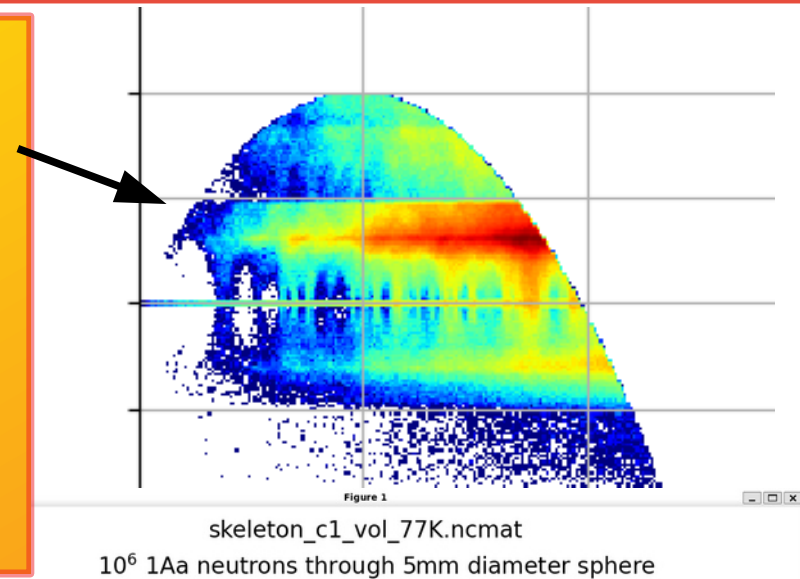
Small Angle Neutron Scattering

- Closely connected to multiphase support, NCrystal contains a framework for SANS physics (= phase interference).
- For now, only a basic hard-sphere SANS model can be enabled, as proof of concept.
- Planning to at least add this and general $I(Q)$ support properly to the NCMAT format, and possibly make tools for importing models from SasView.



WIP: Coherent inelastic effects

- CSNS group (Xiao Xiao Cai, et al.) working on tools for providing high quality coherent-inelastic kernels for NCrystal.
- A proof-of-concept plugin for NCrystal for using those already exists.
- Such files would be temperature-specific and much larger, but for many use-cases this would be very interesting.
- Group also looking at extending this to non-isotropic materials (single crystals).



Extend NCrystal with 3rd-party plugins

<https://github.com/mctools/ncrystal/wiki/Plugins>

NCrystal includes a plugin mechanism, making it possible add custom physics models

- This can help people with their specific simulation use-case, and (in an ideal world) high quality models can eventually be adopted into the main NCrystal code.
- Extending NCrystal will naturally require C++ knowledge. Can develop separately.
- More details on: <https://github.com/mctools/ncrystal/wiki/Plugins>

Flexible, current plugins provide:

- New physics models
- More .ncmat datafiles
- Support for more data formats

Developed in 3rd party repos

- Room for more experimental or less tested features.
- Can be created by anyone with the adequate skills

NCrystal's built-in capabilities use same C++ infrastructure as plugins

Curated list of plugins

<https://github.com/mctools/ncrystal/wiki/CuratedPlugins>

NCrystal

Thermal Neutron Transport

Curated list of plugins

On this page we maintain a curated list of plugins, known to the NCrystal developers and gauged to be of sufficient interest for the wider community. If we are missing something, please [contact](#) us to add another plugin to the list (or simply open a PR adding your plugin to the list we maintain in [this file](#)).

For how to use plugins in general, consult the [Plugins](#) page.

GraphiticData

NCMAT data files for graphite oxides and fullerite (C60).

- **Install with:** `pip install ncrystal-plugin-GraphiticData`
- **PyPI URL:** <https://pypi.org/project/ncrystal-plugin-GraphiticData>
- **Source URL:** <https://github.com/highness-eu/ncmat-graphitic>

UraniumOxideData

Extra NCMAT data files for Uranium Oxide (UO₂), with phonon densities optimised for various temperatures. Phonon density curves in the files are taken from: G. Noguere, et al., 2020 (<https://doi.org/10.1103/PhysRevB.102.134312>).

- **Install with:** `pip install ncrystal-plugin-UraniumOxideData`
- **PyPI URL:** <https://pypi.org/project/ncrystal-plugin-UraniumOxideData>
- **Source URL:** <https://github.com/mctools/ncrystal-extra/tree/HEAD/pypkgs/ncrystal-plugin-UraniumOxideData>

WaterData

Extra NCMAT data files for light and heavy water, covering a wider range of temperatures compared to what is found in the NCrystal standard library. The data files were based on ENDF/B-VIII.0 CAB scattering kernels by J.I. Márquez Damián, et al. (<https://doi.org/10.1016/j.anucene.2013.11.014>).

- **Install with:** `pip install ncrystal-plugin-WaterData`
- **PyPI URL:** <https://pypi.org/project/ncrystal-plugin-WaterData>
- **Source URL:** <https://github.com/mctools/ncrystal-extra/tree/HEAD/pypkgs/ncrystal-plugin-WaterData>

BzScope

A plugin allowing coherent inelastic kernels for isotropic materials, thus foregoing the incoherent approximation. The actual upstream location of this plugin is <https://code.ihep.ac.cn/cinema-developers/ncplugin-bzscope> but we also keep a mirror at the provided GitHub location in order to reduce network latency issues.

- **Install with:** `pip install "git+https://github.com/mctools/ncrystal-plugin-BzScope"`
- **Source URL:** <https://github.com/mctools/ncrystal-plugin-BzScope>

ClathrateHydrateData

NCMAT data files of various clathrate hydrates, including paramagnetic effects of scattering on enclathrated O₂ via the MagScat plugin.

- **Install with:** `pip install "git+https://github.com/highness-eu/ncmat-clathrates"`
- **Source URL:** <https://github.com/highness-eu/ncmat-clathrates>

CrysExtn

A plugin providing extinction models for polycrystalline materials. Both Sabine's and Becker and Coppens' (BC) extinction models are implemented.

- **Install with:** `pip install "git+https://github.com/xushuqi7/ncplugin-CrysExtn"`
- **Source URL:** <https://github.com/xushuqi7/ncplugin-CrysExtn>

CrysText

A plugin providing texture models. Note that this plugin is slow for shorter wavelengths.

- **Install with:** `pip install "git+https://github.com/highness-eu/ncplugin-CrysText"`
- **Source URL:** <https://github.com/highness-eu/ncplugin-CrysText>

LiquidHeliumData

A plugin providing super critical helium near 1K, based on <https://doi.org/10.1016/j.nima.2023.168284> (for details refer to comments on <https://github.com/mctools/ncrystal/issues/279> and <https://github.com/ddijulio/ncrystal-plugin-LiquidHelium>).

- **Install with:** `pip install "git+https://github.com/ddijulio/ncrystal-plugin-LiquidHelium"`
- **Source URL:** <https://github.com/ddijulio/ncrystal-plugin-LiquidHelium>

MagScat

A plugin providing paramagnetic scattering in O₂-clathrates, based on formulas by O. Zimmer (<https://doi.org/10.1103/PhysRevC.93.035503>).

- **Install with:** `pip install "git+https://github.com/highness-eu/ncplugin-MagScat"`
- **Source URL:** <https://github.com/highness-eu/ncplugin-MagScat>

SANSND

A plugin to provide small angle neutron scattering for nanodiamonds. Models and benchmarking are described in N. Rizzi, et al. (<https://doi.org/10.1080/00295639.2023.2196926>).

- **Install with:** `pip install "git+https://github.com/highness-eu/ncplugin-SANSND"`
- **Source URL:** <https://github.com/highness-eu/ncplugin-SANSND>

nxslib

NCrystal plugin for loading .nxs files. Also includes a small database of .nxs

- **Install with:** `pip install "git+https://github.com/mctools/ncplugin-nxslib"`
- **Source URL:** <https://github.com/mctools/ncplugin-nxslib>

Tested in NCrystal CI
Ensure compatibility with
new NCrystal releases

Easy installation
Just "pip install ..."

More info about NCrystal

NCrystal
Thermal Neutron Transport

**Forum for questions/discussions,
issue tracker, wiki, data library page at:**
<https://github.com/mctools/ncrystal>

General info:
DOI 10.1016/j.cpc.2019.07.015

Details about elastic models:
DOI 10.1016/j.cpc.2021.108082

Embedded documentation:
Entire Python API has doc-strings.
Command-line tools have -h / --help flags

Details about inelastic sampling:
DOI 10.1016/j.jcp.2018.11.043



Jupyter tutorials at:
<https://github.com/mctools/ncrystal-notebooks/>

**Better website to come
(hopefully in 2026)**



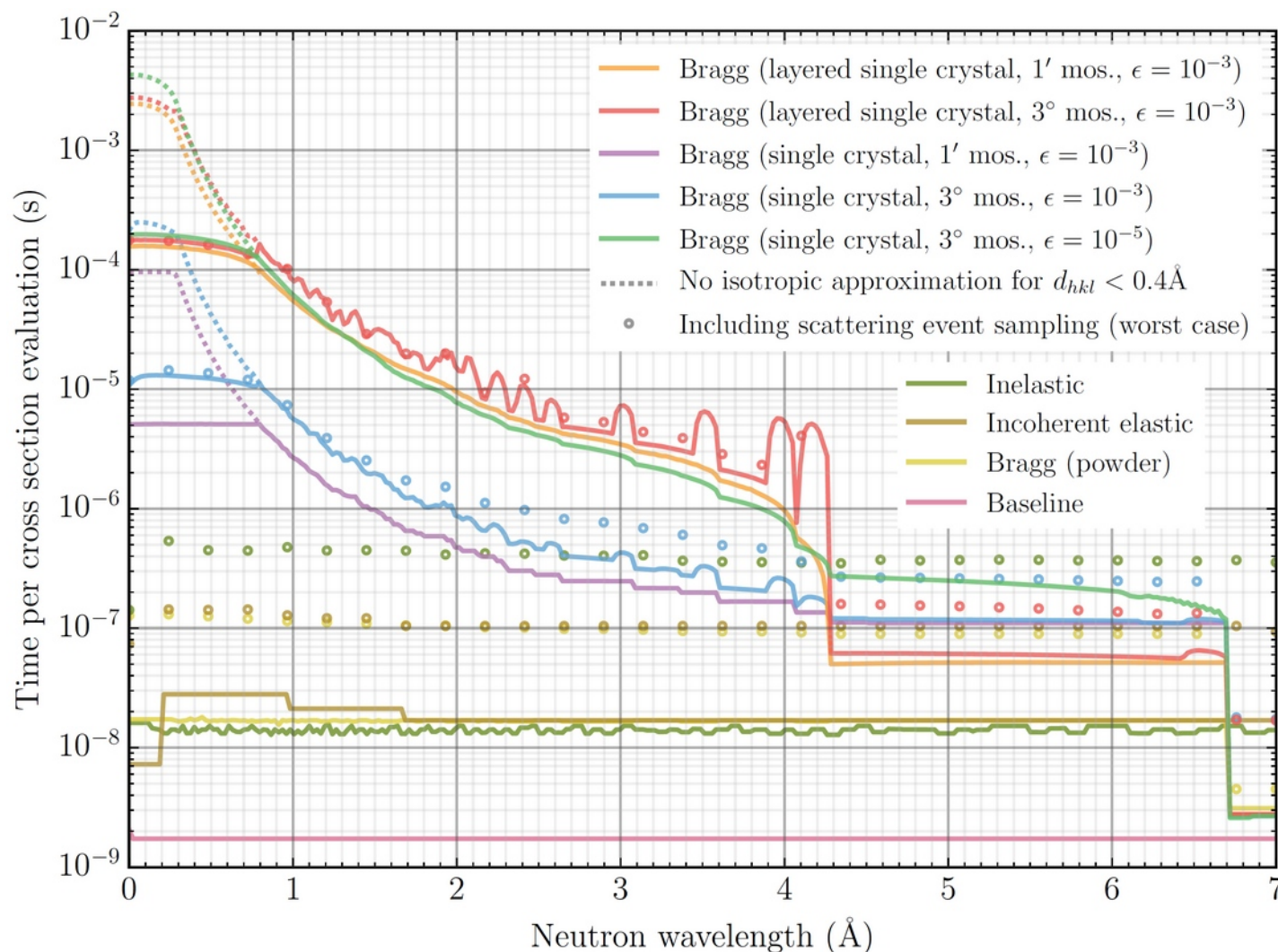
Backup slides



Miscellaneous



Strong focus on computational speed



Rough conclusions for MC simulations with thin samples:

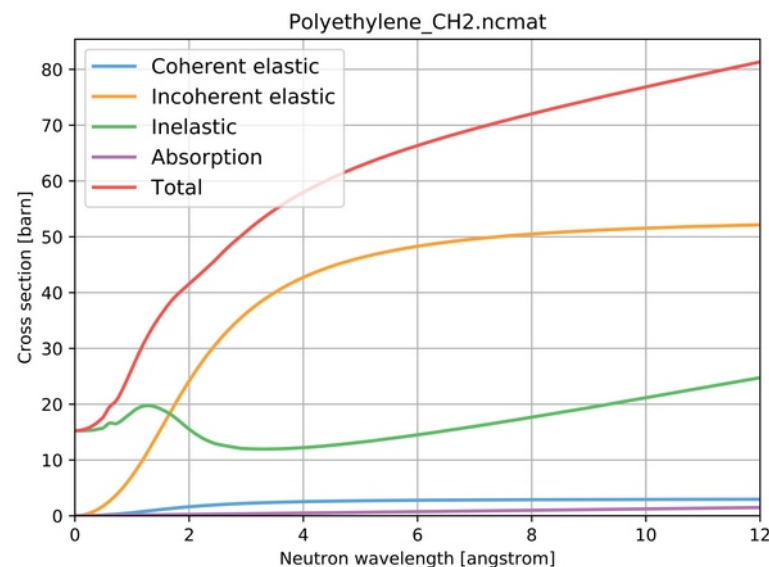
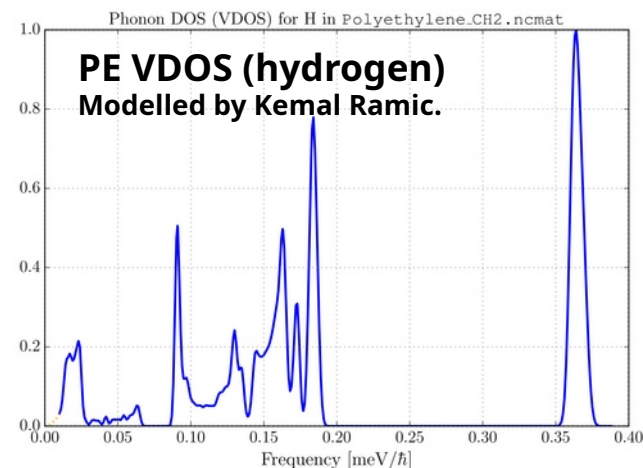
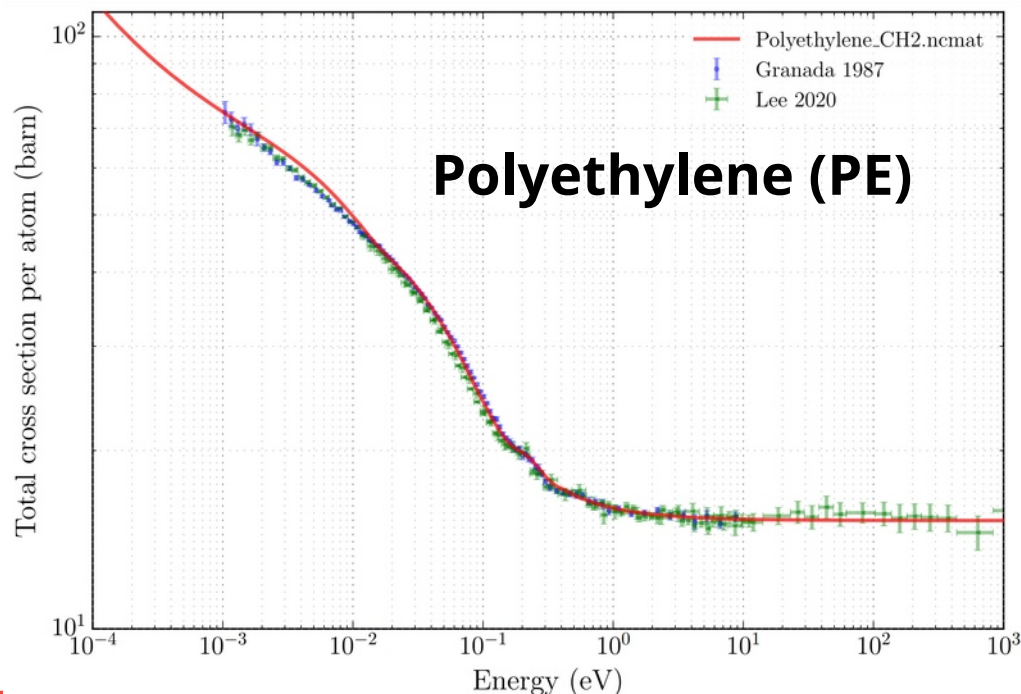
- **O(1-100MHz) neutrons per thread in powder**
- **O(0.1-10MHz) neutrons per thread in single crystal (depends on λ)**

Material *initialisation* time also very strong focus.

- **V3.9.0 introduced several speedups and the option for multithreaded initialisation.**
- **Most materials initialise in <200ms (many <10ms)**

Amorphous solids

- Uses same inelastic and incoherent-elastic approach as for crystalline solids.
- Coherent-elastic scattering via incoherent approximation for now, best for materials with lots of incoherent scattering like H-rich materials (but see next slide).



Flexible atomic (re)definitions

NCrystal supports atoms which are not just natural elements!

- Ships with database of 80+ natural elements and 261+ isotopes.
- Possibility to customise:
 - In NCMAT data
 - In cfg-string parameter
 - With the NCMATComposer

NCMAT v3

@ATOMDB

#Override data for whatever reason:

H 1.008u -3.7fm 80.3b 0.3b

#Provide absent data:

Rn222 222.017u 123fm 0.456b 789b

#Enrich Boron:

B mix 0.95 B10 0.05 B11

#Add dopants on Al positions:

Al mix 0.99 Al 0.01 Cr

#Alternatively use "variable names"

\$(for usage elsewhere in the file):

X mix 0.2 Al 0.4 Cr 0.4 Th

#Or simply assign:

B is B10

```
auto sc = NCrystal::createScatter("Al2O3_sg167_Corundum.ncmat;atomdb=Al:mix:0.99:Al:0.01:Cr");
```

Multiphase materials

- Multiphase materials can be defined inside NCMAT data (i.e. with the NCMATComposer), or in a cfg-string:
 - `"phases<FRAC1*CFGSTR1&...&FRACN*CFGSTRN>;COMMONCFG"`
- Example (enriched B4C pellets in epoxy):
 - `"phases<0.01*solid::B4C/2.52gcm3/B_is_0.95_B10_0.05_B11
&0.99*Epoxy_Araldite506_C18H2003.ncmat>;temp=250K"`
- NB: Using volume fractions, not mass fractions (for now).
- NB: Syntax designed so you can always append e.g. `";temp=250K"` to a cfg-string, and have it work.

NCrystal data library wiki page

<https://mctools.github.io/ncrystal/> → wiki → Data-library

NCrystal

Thermal Neutron Transport

Data library

Thomas Kittelmann edited this page 1 minute ago · 29 revisions

This page last updated with NCrystal release v3.9.2

NB: Please also check [here](#) for the actual version-controlled data files.

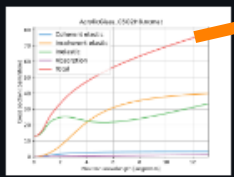
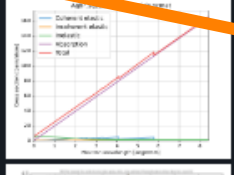
If you do not find a given material below you can [request help creating such a file](#).

Note that for gas-mixtures, you can simply use a cfg-string like "gasmix::0.72xCO2+0.28xAr/massfractions/3bar/250K" or "gasmix::air/-10C/0.8atm/0.38rehumidity", and the relevant material will be generated on-the-fly. See [this page](#) for more examples and syntax.

You might also find some additional (both validated and unvalidated/incomplete) materials in the [ncrystal-extra](#) repository.

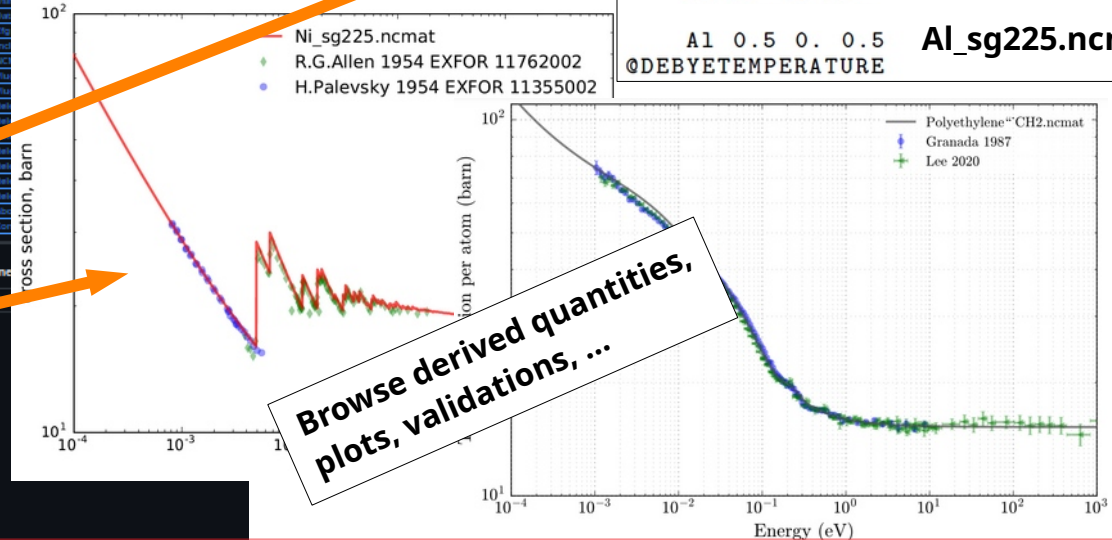
You can also consult the "cookbook" of dgcode, which provides many recipes for NCrystal materials:

<https://mctools.github.io/simplebuild-dgcode/matcookbook.html>

Data file (.ncmat)	Properties	Plot validation
AcrylicGlass_C502H8	State: Solid (amorphous) $\rho = 1.186 \text{ g/cm}^3$ $\rho_a = 0.1065 \text{ atoms/\AA}^3$ Inelastic models: 53.3% H: VDOS 33.3% C: Debye model 13.3% O: Debye model Full info...	
AgBr_sg225_SilverBromide	State: Solid (crystalline) Structure: Fm-3m (225) $\rho = 6.477 \text{ g/cm}^3$ $\rho_a = 0.04155 \text{ atoms/\AA}^3$ Inelastic models: 50% Br: VDOS 50% Ag: VDOS	

```
NCMAT v1
#Some comment here...
@CELL
  lengths 4.04958 4.04958 4.04958
  angles 90. 90. 90.
@SPACEGROUP
  225
@ATOMPOSITIONS
  Al 0. 0.5 0.5
  Al 0. 0. 0.

  Al 0.5 0. 0.5
Al_sg225.ncmat
@DEBYETEMPERATURE
```



Browse derived quantities,
plots, validations, ...

To use in Geant4/McStas/OpenMC/Python/..., simply supply name of material file, along with relevant parameters like temperature, orientation of single crystal, etc.

Alternatively browse locally with:

nctool -browse

And then:

nctool [-dump|--extract] SomeFile.ncmat

NOTE: M. Klausz is working on a much improved less static datalib website.

In-memory data

With or without a "filename"

NCrystal

Thermal Neutron Transport

Register as virtual file with filename:

```
import NCrystal as NC
content="""NCMAT v3
@CELL
lengths 4.04958 4.04958 4.04958
angles 90 90 90
@SPACEGROUP
225
@ATOMPOSITIONS
Al 0 1/2 1/2
Al 0 0 0
Al 1/2 1/2 0
Al 1/2 0 1/2
@DEBYETEMPERATURE
Al 410.4
"""
```

```
NC.registerInMemoryFileData( "MyAl.ncmat",content)
sc = NC.createScatter("MyAl.ncmat")
```

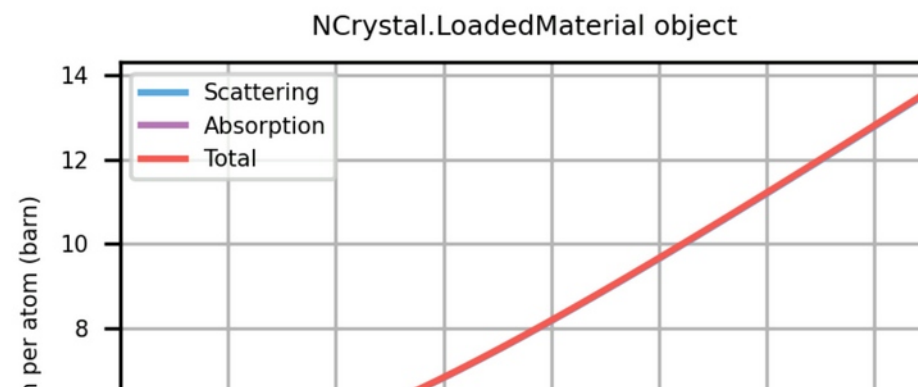
Accessing data is of course possible:

```
nctool -extract Al_sg225.ncmat
NC.createTextData('Al_sg225.ncmat')
```

NCrystal conda/pip packages use this to embed standard data lib files into libNCrystal.
(using CMake -DNCRYSTAL_ENABLE_DATA=EMBED)

Load NCMAT data directly:

```
a_string_with_ncmat_data="""NCMAT v7
#Don't use this material for anything
@DENSITY
1.2345 g_per_cm3
@DYNINFO
element C
fraction 1
type freegas
"""
NC.load(a_string_with_ncmat_data).plot()
```



A brief history of NCrystal

(no, this will not be on the test)



- (prehistoric: NXSG4 in 2014 by T. Kittelmann + M. Boin)
- ~2015: T. Kittelmann + X. X. Cai of the ESS Detector Group and DTU NuTech join separate efforts aimed at adding thermal neutron physics to Geant4. Quickly decide to make it a standalone project and add support for McStas as well. Both join the Geant4 collaboration to get NCrystal integrated, and also work with P. Willendrup on McStas integration.
- 2019 (v1.0): After years of development and writing the first publication, release 1.0, focusing on Bragg diffraction and crystalline materials. HKL structure factors are calculated on the fly at startup. Contains minimal C++/C/Python API and hooks for Geant4 and McStas.
- 2020 (v2.0): with inelastic physics: scattering kernels and a unique capability to expand phonon spectra to scattering kernels on the fly.
- 2020 (fall) (v2.1-2.4): Support atomic/isotopic mixtures, virtual files, and a system for third-party plugins with specific new physics.
- March 2021 (v2.5): Major C++11 rewrite for safe modern C++. Becomes multi-thread safe and gets a flexible data-source structure.
- April 2021 (v2.6): K. Ramic, J. I. Marquez Damian, and D. DiJulio join the efforts, and most files in the data library gets phonon DOS curves added. We also begin to estimate atomic displacements from such curves.
- May 2021 (v2.7): The data library grows enormously (now arguably world leading!) thanks again to collaboration with the same people. Add support for amorphous materials, add cmdline tools for adding new materials.
- April 2022 (v3.0): Large update with multi-phase materials, support for SANS physics, new cfg-parameters like "density", "one-liner materials", etc.
- June 2022 (v3.1): Focus on UCN (ultra cold neutron) production in inelastic collisions, ensuring artifact-free modelling and possibilities for biasing.
- August 2022 (v3.2): Support for easily configuring gas-mixtures.
- Aug-Dec 2022 (v3.3-3.5): Improve CMake layer and introduce ncrystal-config command. Improve integrations with McStas (with P. Willendrup and M. Bertelsen) and Geant4. Bindings for OpenMC in OpenMC 13.3 release. NCrystal appears on conda-forge. CI tests improves with help of M. Klausz.
- May 2023 (v3.6): Large improvement of python API, many utilities for creating new materials from a variety of sources. Publish Jupyter tutorials.
- July 2023 (v3.7): NCrystal appears on PyPI, "pip install ncrystal" now works! ESS DMSC supports NCrystal.
- December 2024 (v3.8): NCrystal components move into McStas itself, NCrystal+McStas integration improves via conda (work with P. Willendrup).
- August 2024 (v3.9): NCrystal ships with "MiniMC" for fast simple diffraction patterns.
- October 2024 - : NCrystal work supported by APRENDE EURATOM grant (T.K. with J. I. Marquez Damian, and D. DiJulio).
- Along the way, supported by: two EU projects, EURATOM, got physics in external plugins (Thanks N. Rizzi, S. Xu, CSNS), and were supported in various fashions by many people not mentioned already, in particular: T. R. Nielsen, V. Santoro, R.H.Wilton, K.Kanaki, A. Morozov, E. Klinkby, E. Knudsen, ...
- Many ongoing ideas and developments at ESS (in various groups), and CSNS in particular.

Not such an old project, but a lot of activity and many people contributing in many ways!
Always looking for more contributions!

More details on NCrystal elastic physics algorithms

$S_{\text{coh,el}}(\vec{q}, \omega)$ in crystals: Bragg diffraction!

Microscopic scatter function:

$$S_{\text{coh}}^{\text{el}}(\vec{Q}, \omega) = \frac{(2\pi)^3 \delta(\hbar\omega)}{V_{\text{uc}}} \sum_{hkl} \delta(\vec{Q} - \vec{\tau}_{hkl}) |F(\vec{\tau}_{hkl})|^2$$

V_{uc} : unit cell volume
 $\vec{\tau}_{hkl}$: plane normal with magnitude $2\pi/d_{hkl}$
 $F(\vec{Q}) \equiv \sum_i \bar{b}_i e^{-W_i(\vec{Q})} e^{i\vec{Q} \cdot \vec{p}_i}$
 $F(\vec{Q})$: unit cell structure factor depends on layout of atoms in unit cell
 $2W = \delta^2 q^2$, δ : atomic displacement

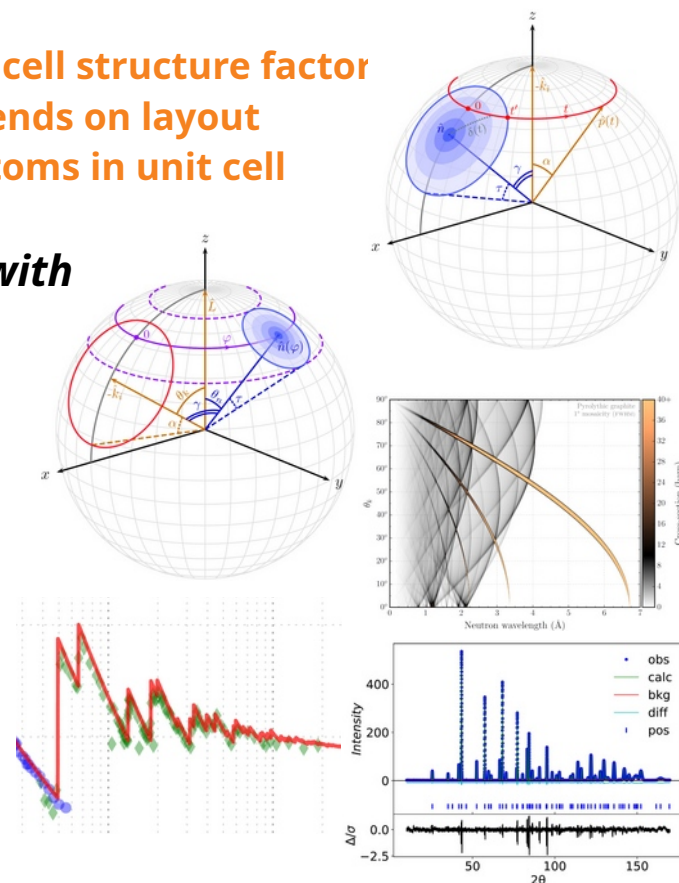
Macroscopic values are found from convoluting microscopic values with crystal grain distributions → geometrical problem!

Supported geometries in NCrystal:

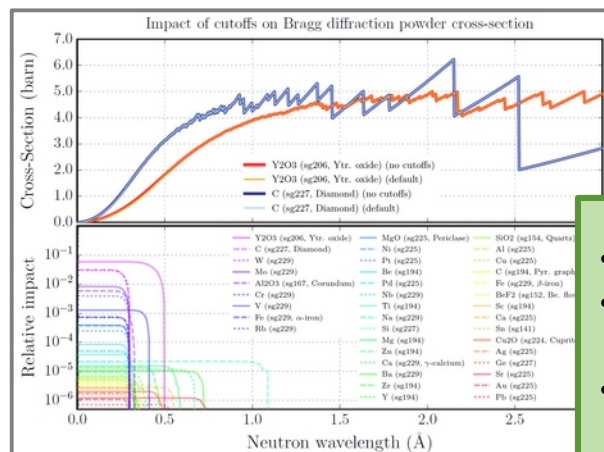
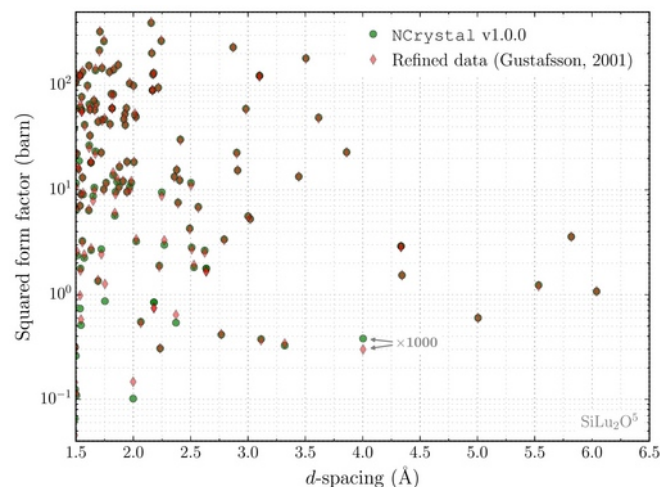
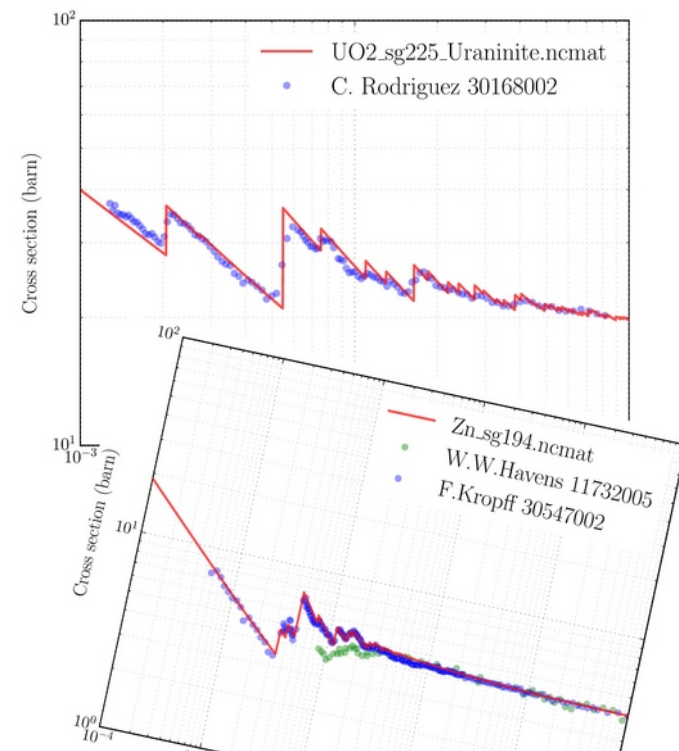
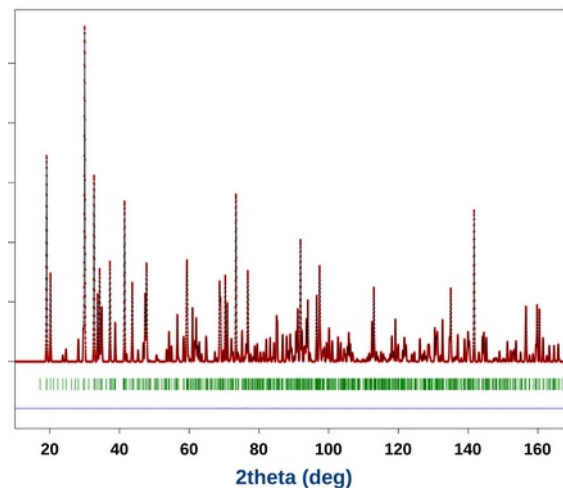
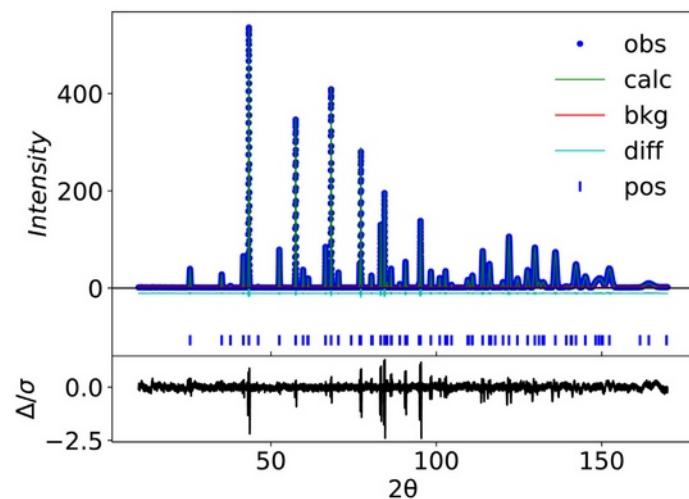
- Completely disoriented layout ("powder approximation")
- Gaussian deviations from completely oriented ("Mosaic single crystals")
- Layered crystals ("rotated mosaic single crystals", pyrolytic graphite)

Not supported yet:

- Bent/deformed crystals, corrections for very small grain sizes.
- Textured crystals (as in most metals/polycrystals). But powder approximation OK for many use-cases! People have expressed interest in improving this though, get in touch if you want to contribute :-)



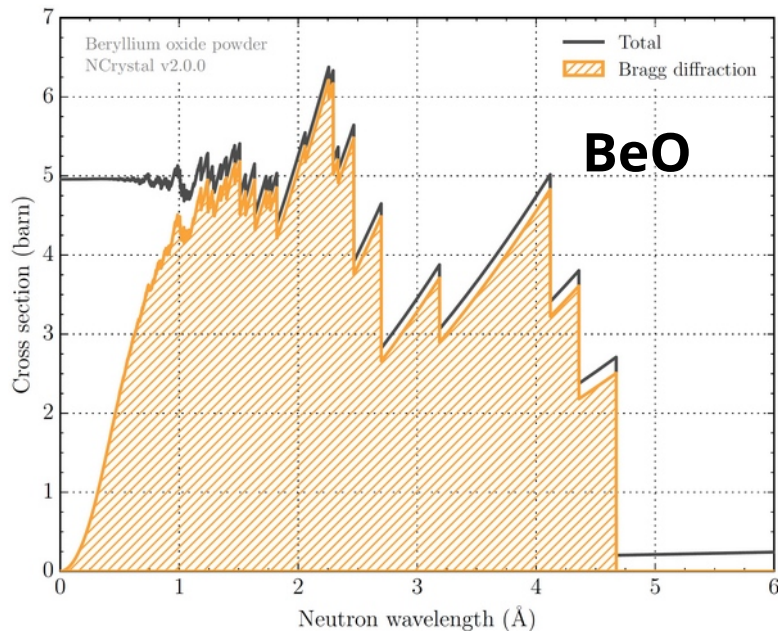
HKL structure factors initialised on-the-fly (validated thoroughly)



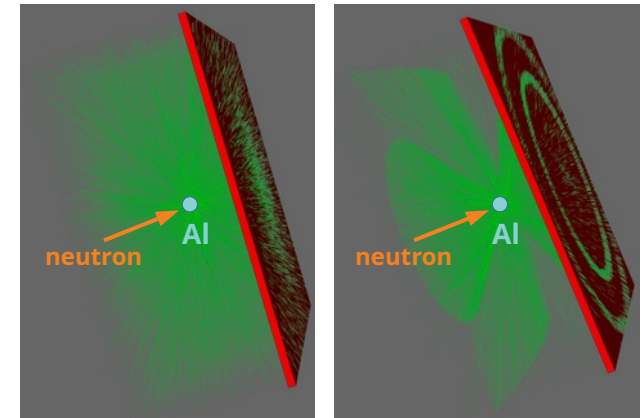
- Fast (few ms) init. of **all** relevant planes.
- Comparisons to measured structure factors and total cross sections.
- NCrystal+McStas simulated scattering patterns analysed with GSAS-II/Fullprof (recovering input crystal parameters).
- Comparison with NXS library predictions.

Bragg diffraction in powders (and texture-free polycrystals)

Based on provided HKL planes with d-spacings and structure factors, the implementation is straight-forward.
Care is taken to be extremely fast $O(10\text{ns}/\text{call})$, even in case of huge number of planes.
Currently no built-in texture/grain-size effects.

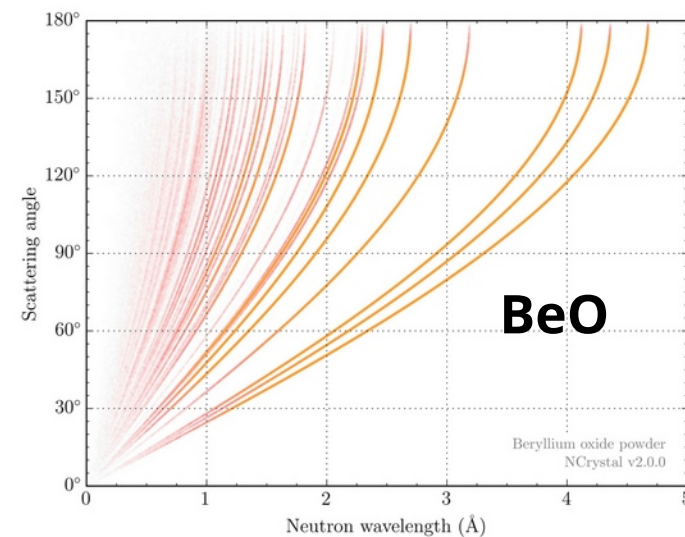


WIP: adding support for primary extinction models (relevant for large mosaic blocks, e.g. in Be, Bi, Pb, ...).



Geant4 free-gas model
(wrong MFP, wrong scatter)

Geant4 with NCrystal
⇒ Debye-Scherrer cones



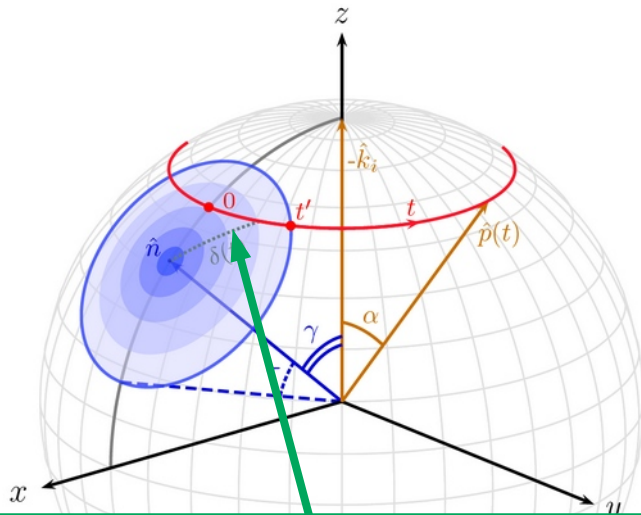
A plugin exist for March-Dollase texture (S. Xu, ESS)

Single Crystals with Gaussian mosaicity

Can model monochromators, analysers, filters, samples
Handles also large mosaicities and backscattering!

NCrystal

Thermal Neutron Transport



The tricky part* is the integration of mosaic density along circle of Bragg condition.

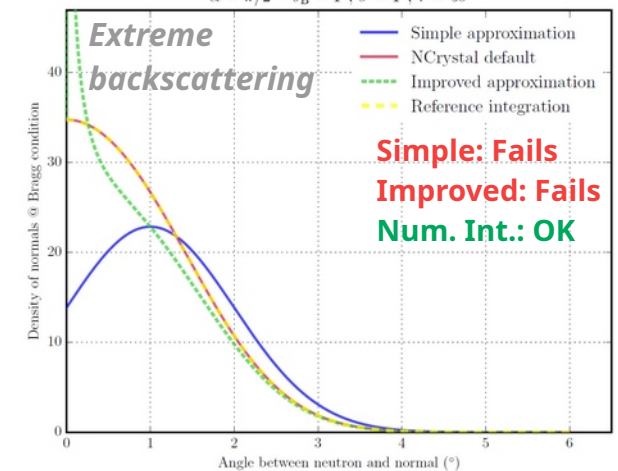
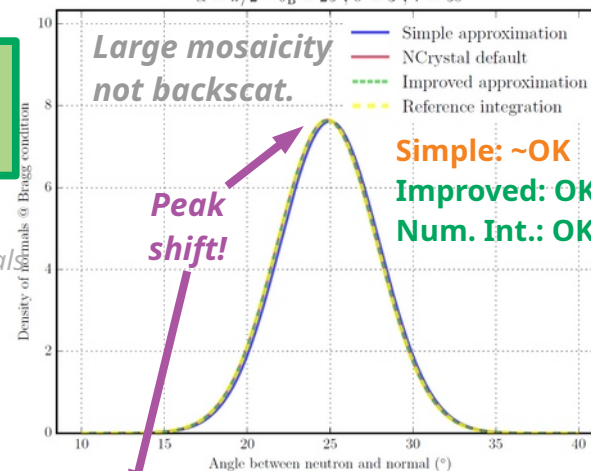
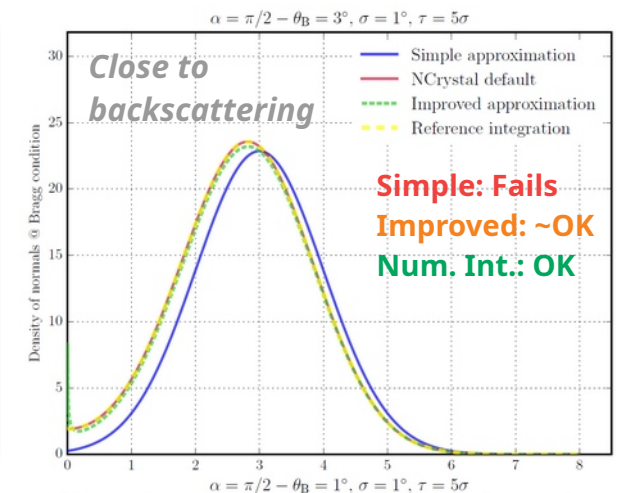
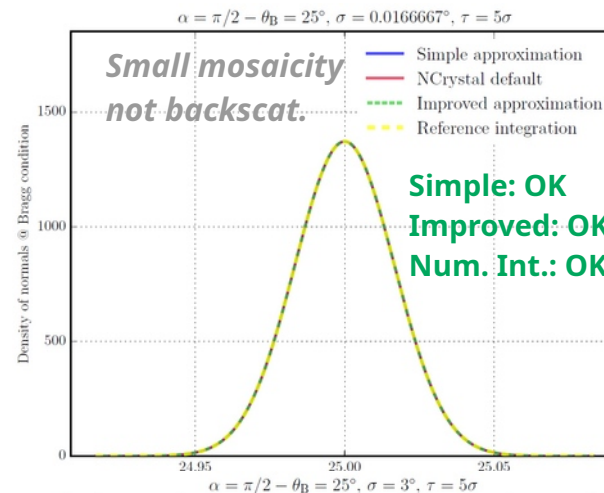
*: Once contributing normals
Have been identified.

Simple closed-form approx.
valid for small mosaicity
(and not backscattering):

$$\sigma_{\text{Bragg}}(\alpha, \gamma) = Q \times \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\frac{\delta_0^2}{\sigma^2}\right] \times \text{erf}\left[\sqrt{\frac{\tau^2 - \delta_0^2}{2\sigma^2}}\right] \times \sqrt{\frac{\sin \alpha}{\sin \gamma}} \times \frac{N}{1/(2\pi\sigma^2)}$$

$$\delta_0 = |\alpha - \gamma|$$

Our improved form extends validity
to much larger mosaicities



Code picks appropriate method from:

- Our closed form approximation
- Full numerical integration

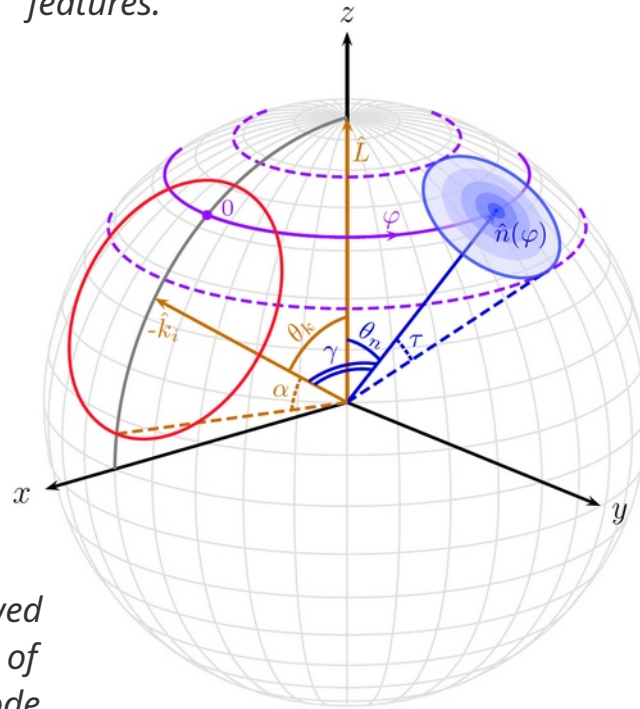
Special anisotropic model for Pyrolytic Graphite

PG often used as filters, monochromator, analyser



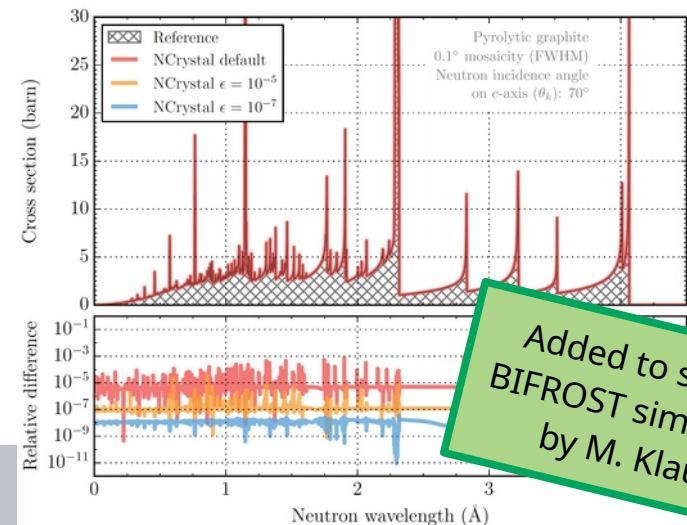
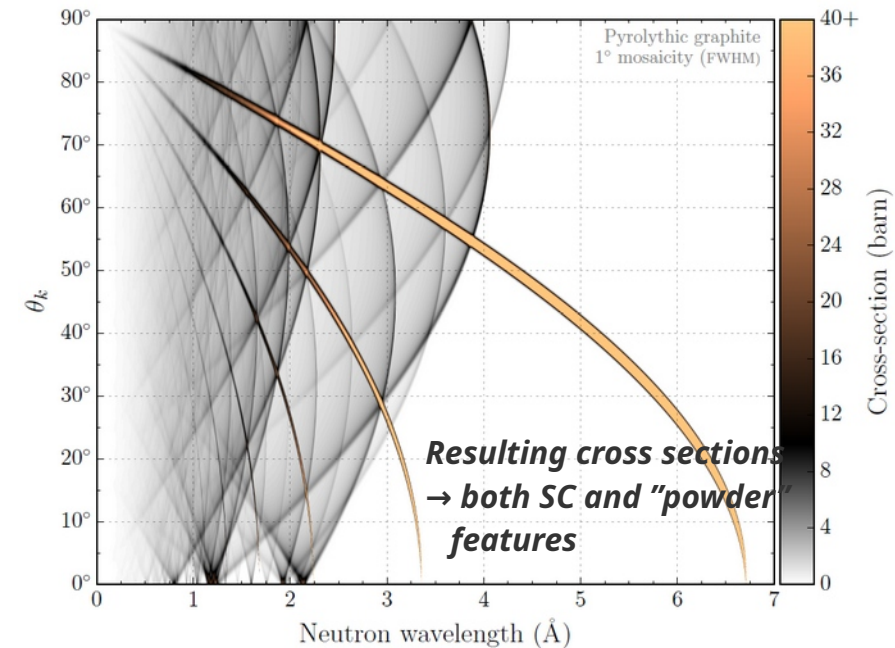
Layered crystal model:

- Usual Gaussian mosaic distribution is "smeared out" by rotation
- Exhibits both single-crystal and powder features.



Features:

- Cross-sections determined by efficient pre-search followed by fast Romberg integration of non-layered single crystal code.
- Features realistic transmission probabilities and multiple-scattering effects (incl. "zig-zag walk")

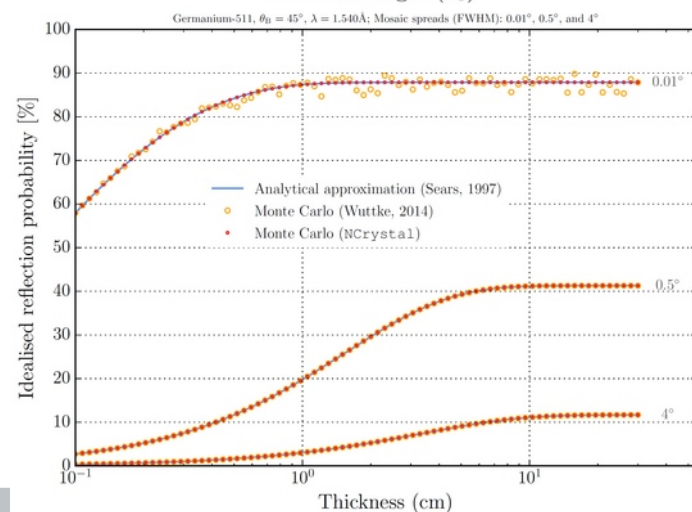
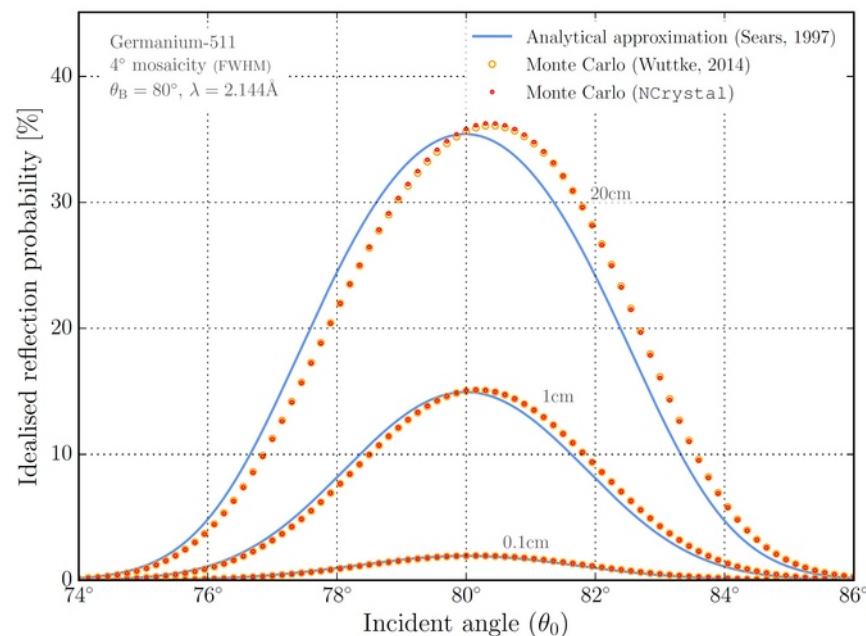
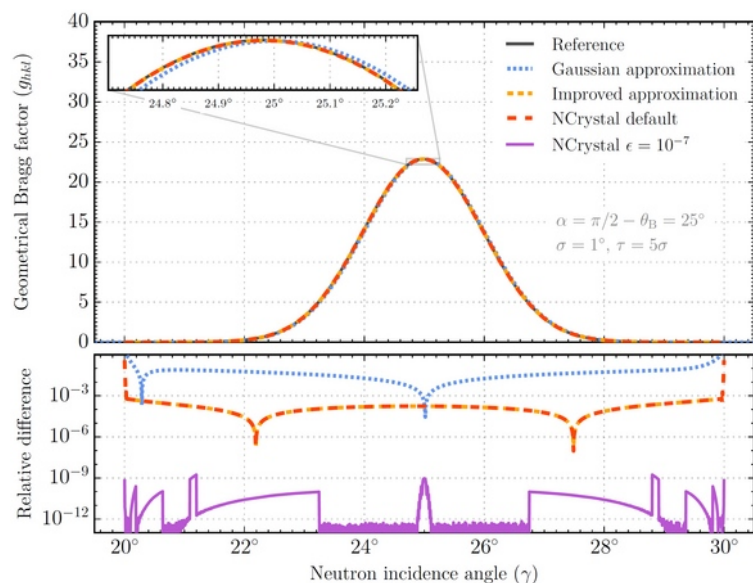


Added to support BIFROST simulations by M. Klausz.

Single crystal model validated

Validation includes:

- Against existing codes (Wuttke2014) or analytical results (Sears1997) in their domains of validity.
- Against (very) slow but simple+precise implementation (using mpmath high-precision math module)
- Technical validations (zig-zag, "powdered")

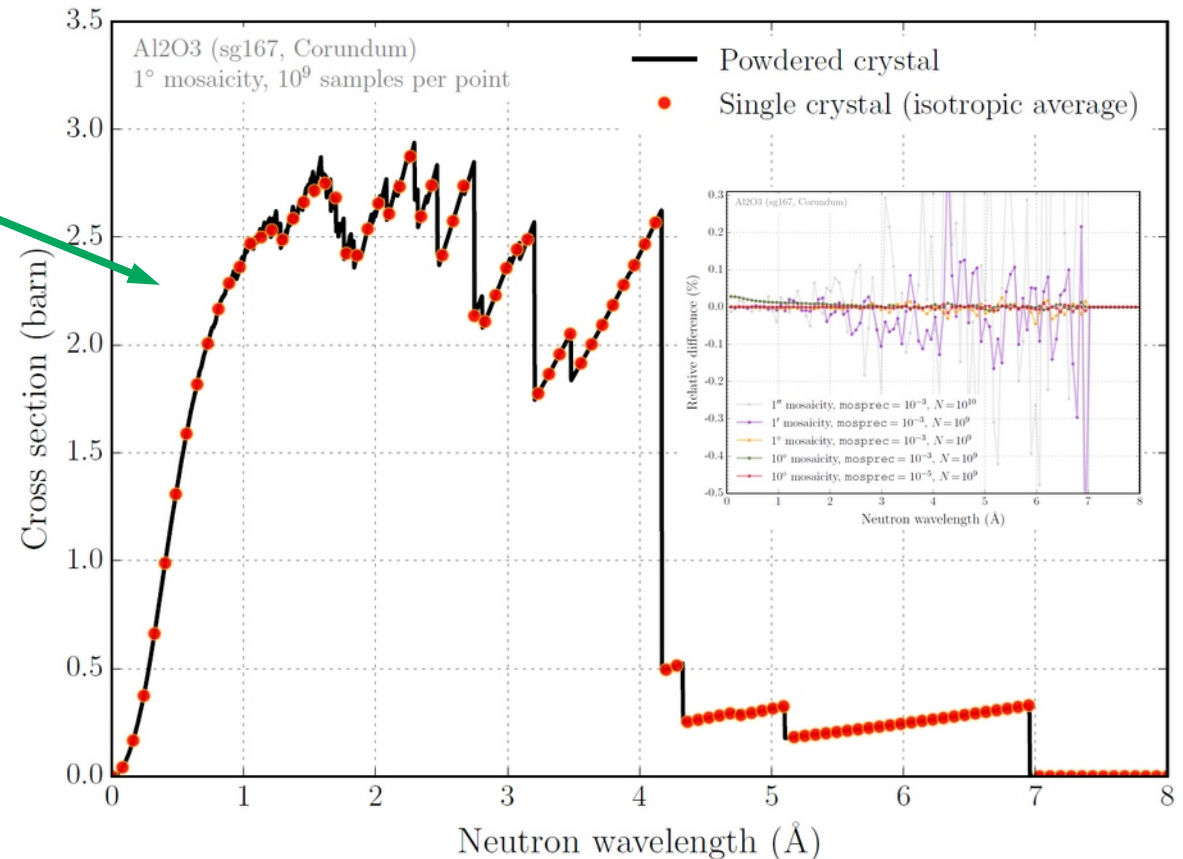


Technical checks for consistency of single crystal codes (should also hold for texture models)

In principle an isotropically illuminated single crystal should on average give powder-like cross-sections.

In practice, a lot of edge-cases and details have to be treated correctly in the SC code before this happens!

Another check is that consistent SC codes should provide “zig-zag walk”



Thanks to the DMSC cluster for help with this brute force validation.

Single crystal with Gaussian mosaicity uses cheap pre-check to speed up

```
double NC::GaussMos::calcCrossSections( InteractionPars& ip,
                                       const NC::Vector& indir,
                                       const std::vector<NC::Vector>& deminormals,
                                       std::vector<NC::GaussMos::ScatCache>& cache,
                                       VectD& xs_commul ) const
{
    nc_assert(ip.isValid()&&ip.m_wl>0);
    nc_assert(indir.isUnitVector());
    std::vector<Vector>::const_iterator it(deminormals.begin()), itE(deminormals.end());
    double xsoffset = xs_commul.empty() ? 0.0 : xs_commul.back();
    double xssum(0.0);
    const double cptsq = ip.m_cos_perfect_theta_sq;
    const double cta = m_gos.getCosTruncangle();
    for(; it!=itE; ++it) {
        const Vector& normal = *it;
        const double dot = normal.dot(indir);
        double sdotcptsq = (1.0 - dot * dot)*cptsq;
        double ds = dot * ip.m_sin_perfect_theta;

        //First a combined check, which usually allows us to skip both normal and
        //anti-normal:
        double A0 = ncmax( 0.0, cta - ncabs(ds) );
        if ( sdotcptsq <= A0*A0 )
            continue;

        //At least one of the two normals should contribute, so deal with them:
        double Am = ncmax( 0.0, cta - ds );
        if ( sdotcptsq > Am*Am ) {
            //anti-normal is within truncated Gauss
            double xs = calcRawCrossSectionValue(ip, dot );
            if (xs) {
                xs_commul.push_back(xsoffset + (xssum += xs));
                cache.emplace_back(-normal, ip.m_inv2dsp);
            }
        }
        double Ap = ncmax( 0.0, cta + ds );
        if ( sdotcptsq > Ap*Ap ) {
            //normal is within truncated Gauss
            double xs = calcRawCrossSectionValue(ip, -dot );
            if (xs) {
                xs_commul.push_back(xsoffset + (xssum += xs));
                cache.emplace_back(normal, ip.m_inv2dsp);
            }
        }
    }
    return xssum;
}
```

This function gives xsect for all planes with a given d-spacing (only called if d-spacing < $\lambda/2$)

Must check all normals in group.

Precheck allows most planes to be skipped since they do not contribute.

Precise evaluation of the non-zero contribution is still where most time is spent, even when only very few planes survives the cheap pre-check!

Time to sample interactions is much less crucial, since it mostly deals with only a single plane.

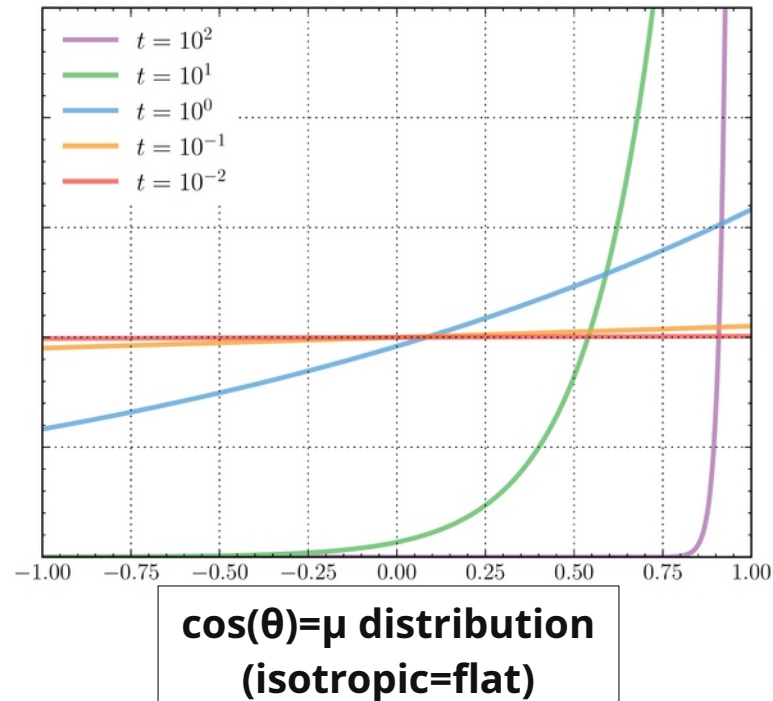
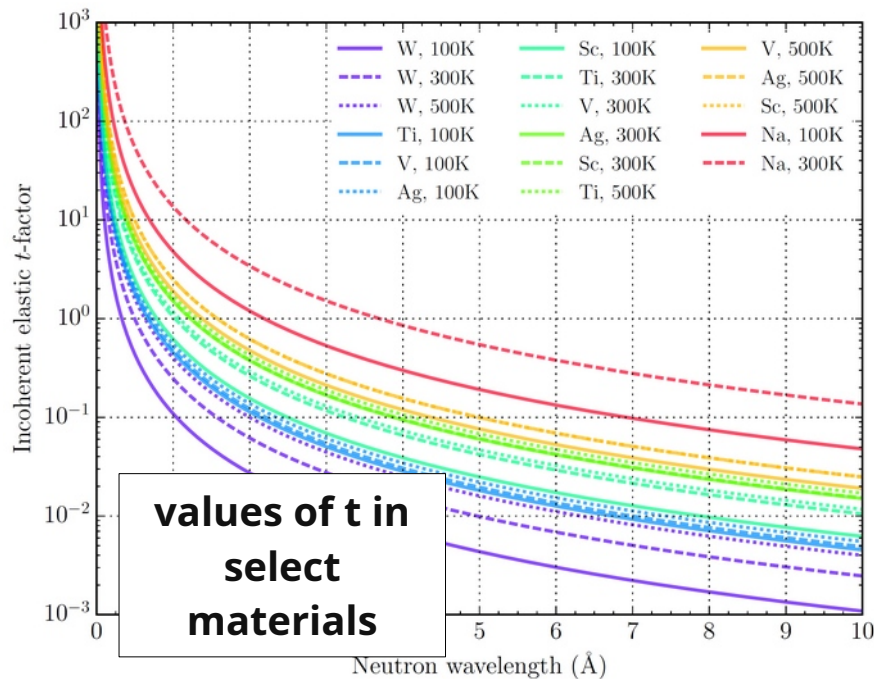
Incoherent-elastic scattering

$$\frac{d\sigma_{\vec{k}_i \rightarrow \vec{k}_f}^{\text{inc,el}}}{d\Omega_f} = \frac{1}{N} \sum_{j=1}^N \frac{\sigma_j^{\text{inc}}}{4\pi} e^{-2W_j(\vec{Q})} \longrightarrow \sigma^{\text{inc,el}}(k) = \sigma_{\text{inc}} \frac{1 - \exp(-t)}{t}$$

Get Debye-Waller factors (or δ^2)
from phonon DOS (or Debye temp.).

$$P(\mu) = N_t \exp\left(\frac{t\mu}{2}\right)$$

$$t \equiv (2k\delta)^2 = \left(\frac{4\pi\delta}{\lambda}\right)^2$$



→ Not
always
completely
isotropic!!

More details on NCrystal inelastic physics algorithms

Inelastic physics : Scattering kernels

$$\frac{d^2 \sigma_{\vec{k}_i \Rightarrow \vec{k}_f}}{d\Omega_f dE_f} = \frac{k_f}{k_i} S(\vec{Q}, \omega)$$

$$S(\vec{Q}, \omega) = S_{\text{coh}}(\vec{Q}, \omega) + S_{\text{inc}}(\vec{Q}, \omega)$$

$$S_{\text{coh}}(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j,j'=1}^N \bar{b}_j \cdot \bar{b}_{j'} \int_{-\infty}^{\infty} dt \langle j', j \rangle e^{-i\omega t}$$

$$S_{\text{inc}}(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j=1}^N \left(\bar{b}_j^2 - (\bar{b}_j)^2 \right) \int_{-\infty}^{\infty} dt \langle j, j \rangle e^{-i\omega t}$$

Under some assumptions $S(\vec{Q}, \omega)$ can be described with a single “smooth” 2D function (one per atom type):

- Elastic scattering is dealt with separately (as it is in NCrystal).
- *Isotropic material* ($\vec{Q}$ dependency becomes scalar Q)
- *Incoherent approximation*: Off-diagonal entries in S_{coh} wash out when integrating over isotropic grain distribution, so $\text{shape}(S_{\text{coh}}) \approx \text{shape}(S_{\text{inc}})$.

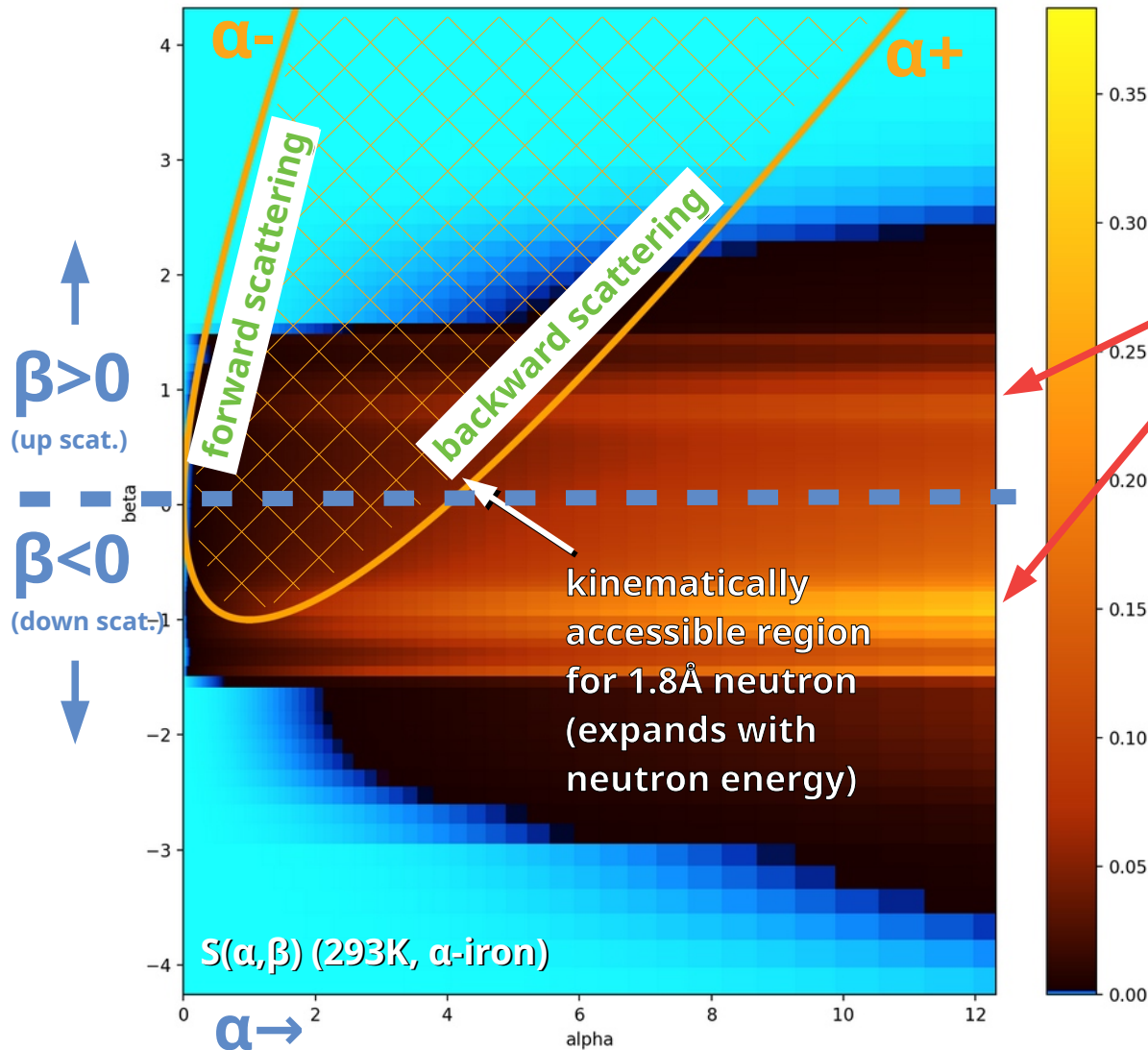
Tabulate this function on a grid → **scattering kernel**.

Scattering kernel and connection to neutron

$S(q, \omega) \sim S(\alpha, \beta)$; α : dimensionless q^2 , β : dimensionless ω ($\sim \Delta E$)

NCrystal

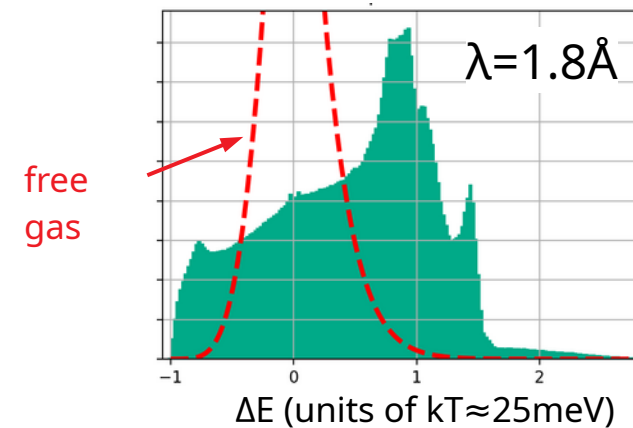
Thermal Neutron Transport



“Detailed balance”:

Boltzmann factor + identical strength of phonon absorption & emission → Symmetry around $\beta=0$, given by $S(\alpha, -\beta) = \exp(\beta) \cdot S(\alpha, \beta)$

β -projection of kinematically accessible region → energy transfer spectrum (green):



Formulation in dimensionless variables: α, β

(q, ω) preferred in neutron scattering community, (α, β) preferred in nuclear industry (incl. MCNP, ENDF, ...)

α : dimensionless q^2
 β : dimensionless ΔE

$$\beta = \frac{E_f - E}{kT} = -\frac{\hbar\omega}{kT}$$

$$\begin{aligned}\alpha &= \frac{\hbar^2}{2m_n kT} q^2 \\ &= \frac{E + E_f - 2\mu\sqrt{EE_f}}{kT} \quad \mu = \cos\theta_{\text{scat}} \\ &= \frac{2E}{kT} + \beta - 2\mu\sqrt{\frac{E}{kT} \left(\frac{E}{kT} + \beta\right)}\end{aligned}$$

Scattering lengths taken outside definition of S:

$$\frac{d^2\sigma}{dE_f d\Omega} = \sqrt{\frac{E_f}{E}} \frac{\sigma_b}{4\pi} \frac{S(\alpha, \beta)}{k_b T}$$

Total cross section, with explicit kinematic limits:

$$\sigma(E) = \frac{\sigma_b kT}{4E} \int_{-E/kT}^{\infty} \int_{\alpha_-(E, \beta)}^{\alpha_+(E, \beta)} S(\alpha, \beta) d\alpha d\beta$$

constant affects $\sigma(E)$,
but not (α, β)-sampling

neutron lose all its energy

$\mu=-1$, complete backwards scattering

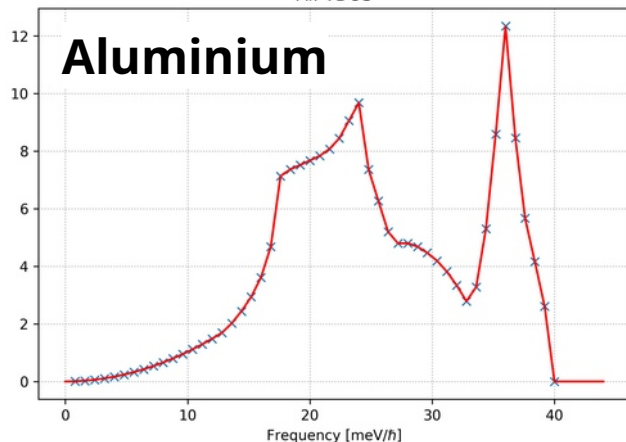
$\mu=+1$, complete forward scattering

$$\alpha_{\pm}(E, \beta) = \frac{2E}{kT} + \beta \pm 2\sqrt{\frac{E}{kT} \left(\frac{E}{kT} + \beta\right)}$$

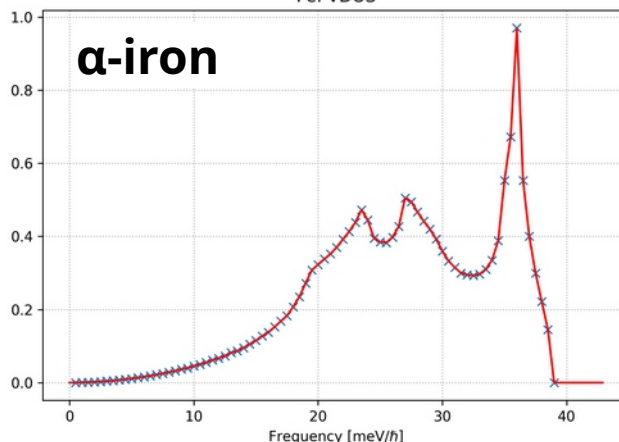
kinematically accessible
region is a parabola in
the (α, β)-plane

Solids: Scattering kernels are connected to phonon frequency spectrums (aka Vibrational Density Of States, VDOS)

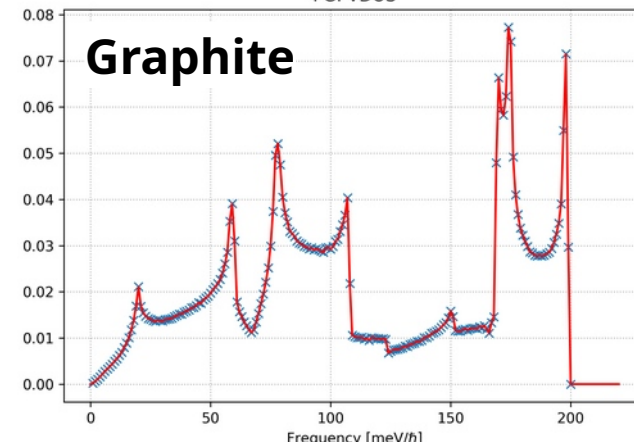
Al: VDOS



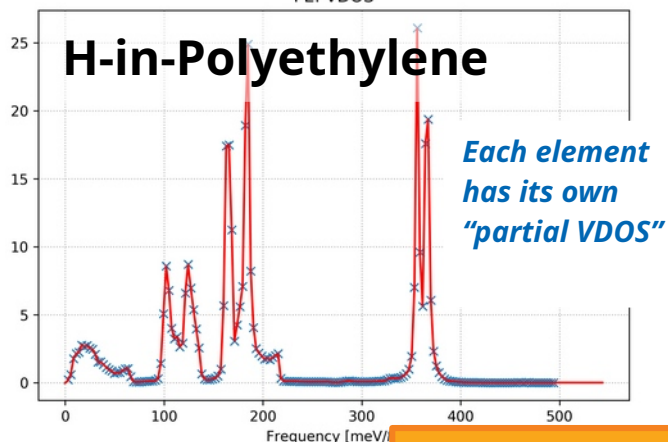
Fe: VDOS



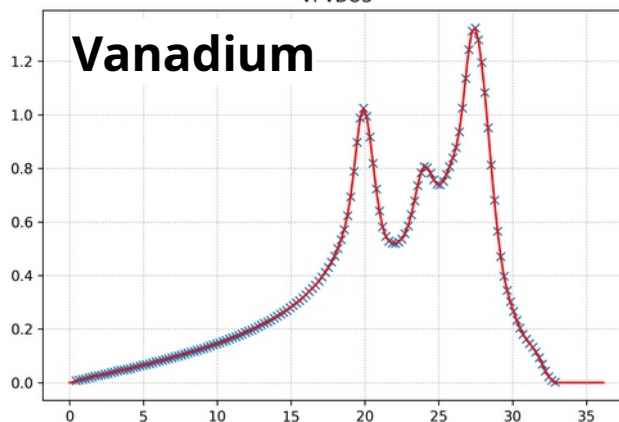
PG: VDOS



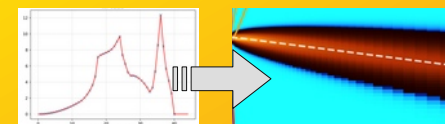
PE: VDOS



V: VDOS



NCrystal can expand VDOS to scattering kernels on-the-fly in O(100ms).



Captures material structure info relevant for *inelastic* neutron scatterings (isotropic materials, incoherent approximation)
+ gives displacements (Debye-Waller factors) needed for *elastic* scatterings.

Phonon spectrum (VDOS) sources

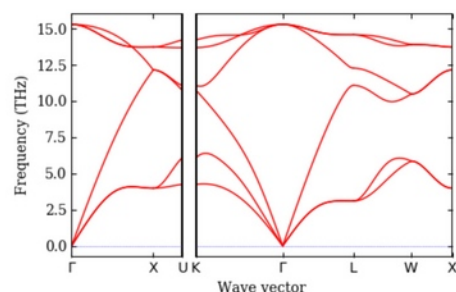
VDOS is not specific to neutrons!

→ Many resources exists

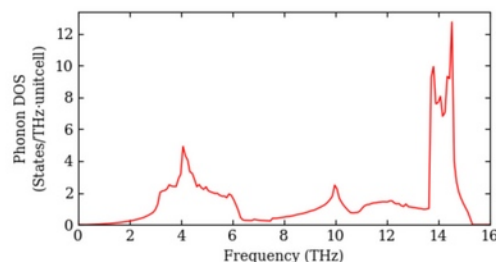
Materials id 149 / Si / Fd-3m (227)

- Date page updated: 2018-4-17
- Space group type: Fd-3m (227) / F 4d 2 3 -1d
- Number of formula units (Z): 8
- Phonon raw data: [mp-149-20180417.tar.gz](#)
- Link to Materials Project: <https://www.materialsproject.org/materials/mp-149/>

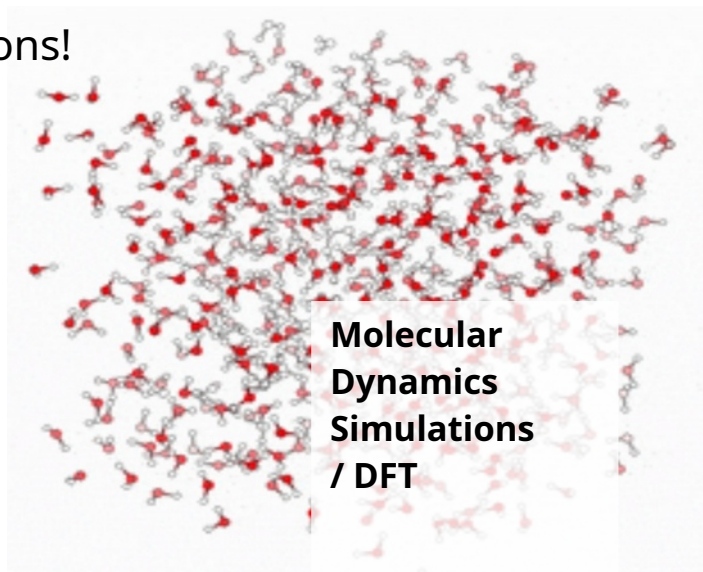
Phonon band structure



Phonon DOS



<http://phonondb.mtl.kyoto-u.ac.jp/>



Daide Campi's lectures at the 2023 HighNESS school contains more information:
<https://indico.ess.eu/event/3096/>



VDOS can also be measured experimentally...

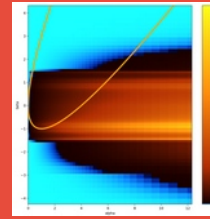
... or dug out from old research papers!

Or a combination, potentially using other SW (QuantumEspresso / VASP / phononpy / Oclimax / ...)

Many VDOS curves need a bit of processing and cleanup to be used.

This can be done in Python using the PhononDOSAnalyser class provided with NCrystal.

NCrystal has unique features for *using* scattering kernels



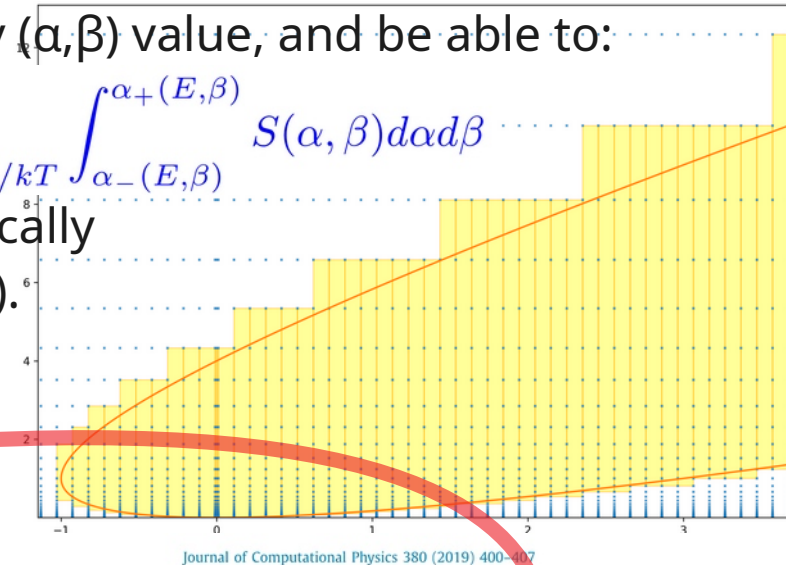
- Given $S(\alpha, \beta)$ values on a grid (α_i, β_i) and a neutron with energy, E , we must define suitable interpolation scheme to provide $S(\alpha, \beta)$ for any (α, β) value, and be able to:

1) Estimate scattering cross section:
$$\sigma(E) = \frac{\sigma_b kT}{4E} \int_{-E/kT}^{\infty} \int_{\alpha_-(E, \beta)}^{\alpha_+(E, \beta)} S(\alpha, \beta) d\alpha d\beta$$

2) Sample (α, β) values randomly within the kinematically accessible region, with density proportional to $S(\alpha, \beta)$.

- This must be done accurately and with reasonable computing resources! **Tricky part is sampling.**
- NCrystal has novel method for accurate+fast sampling, without ACE-like discretisation, with attention to near-endpoint sampling (crucial for ultra-cold neutron moderator studies).

- We have ideas to further improve this** (make it faster, remove remaining artifacts).



Contents lists available at ScienceDirect
Journal of Computational Physics

www.elsevier.com/locate/jcp

Rejection-based sampling of inelastic neutron scattering

X.-X. Cai^{a,b,*}, T. Kittelmann^b, E. Klinkby^{a,b}, J.I. Márquez Damián^c

^a Technical University of Denmark, Denmark

^b European Spallation Source ERIC, Sweden

^c Nuclear Data Group, Neutron Physics Department, Centro Atómico Bariloche, CNEA, Argentina



How we handle materials with no phonon DOS specified?

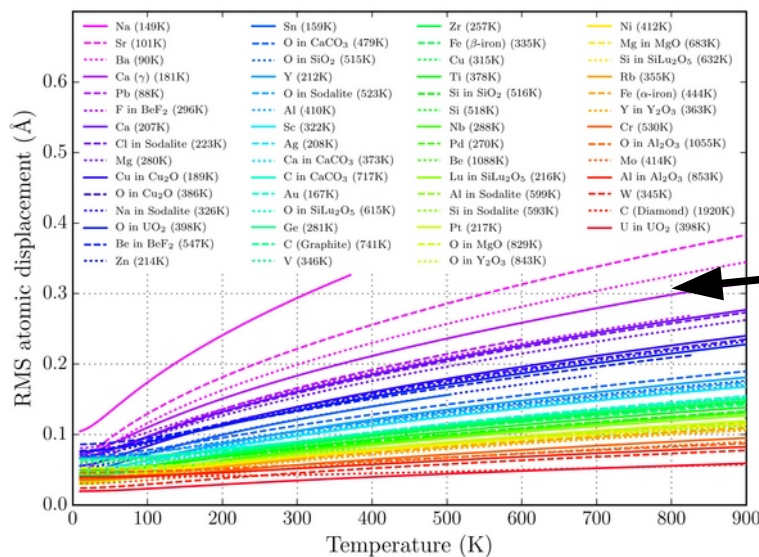
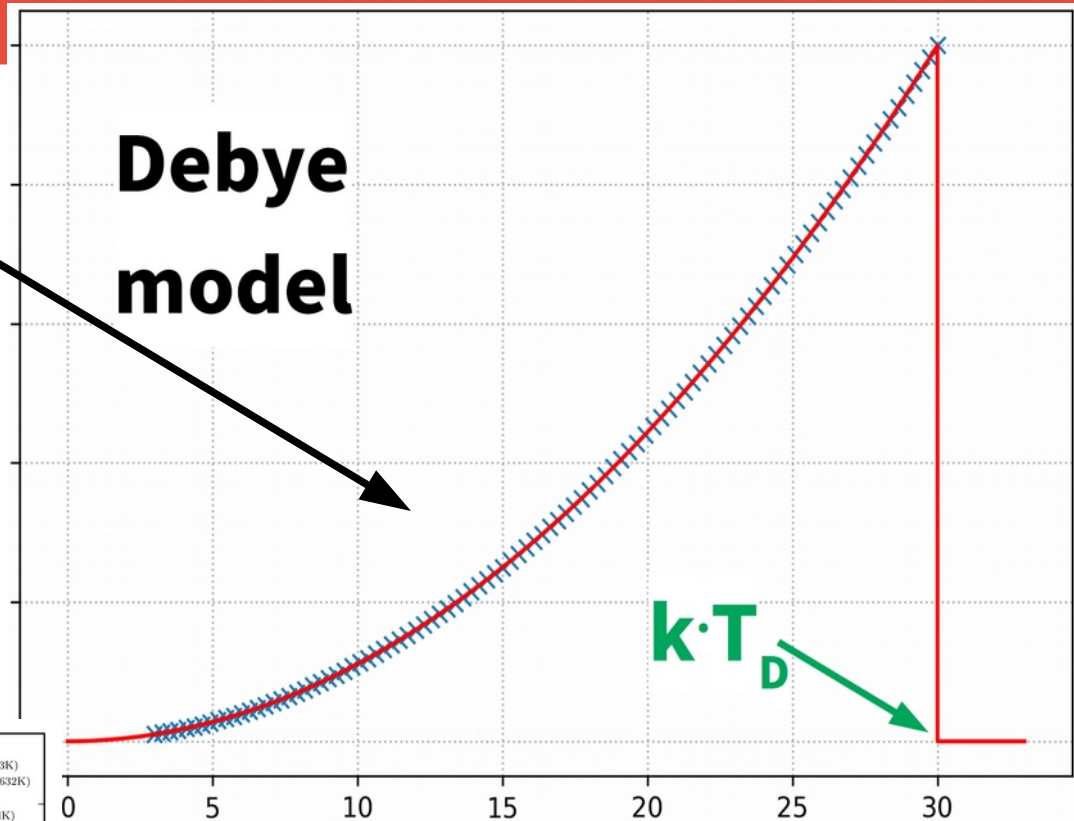
NCrystal

Thermal Neutron Transport

Idealised DOS (Debye Model) is constructed and fed into same infrastructure as any other DOS.

Lacks details of course, but gives consistent kinematics and handles multi-phonon physics ~OK.

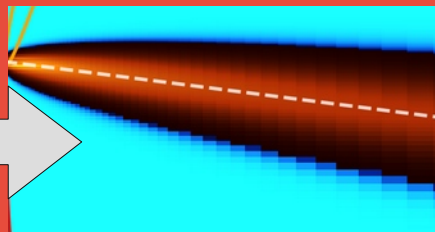
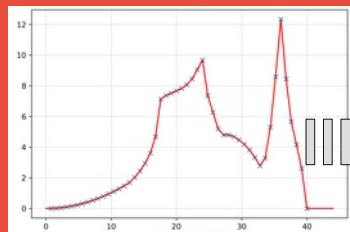
Debye model



T-dependent atomic displacements (δ),
from Debye temperature (T_D)

Of course, a real DOS gives more realistic δ .

VDOS $\rightarrow S(\alpha, \beta)$ [solids]

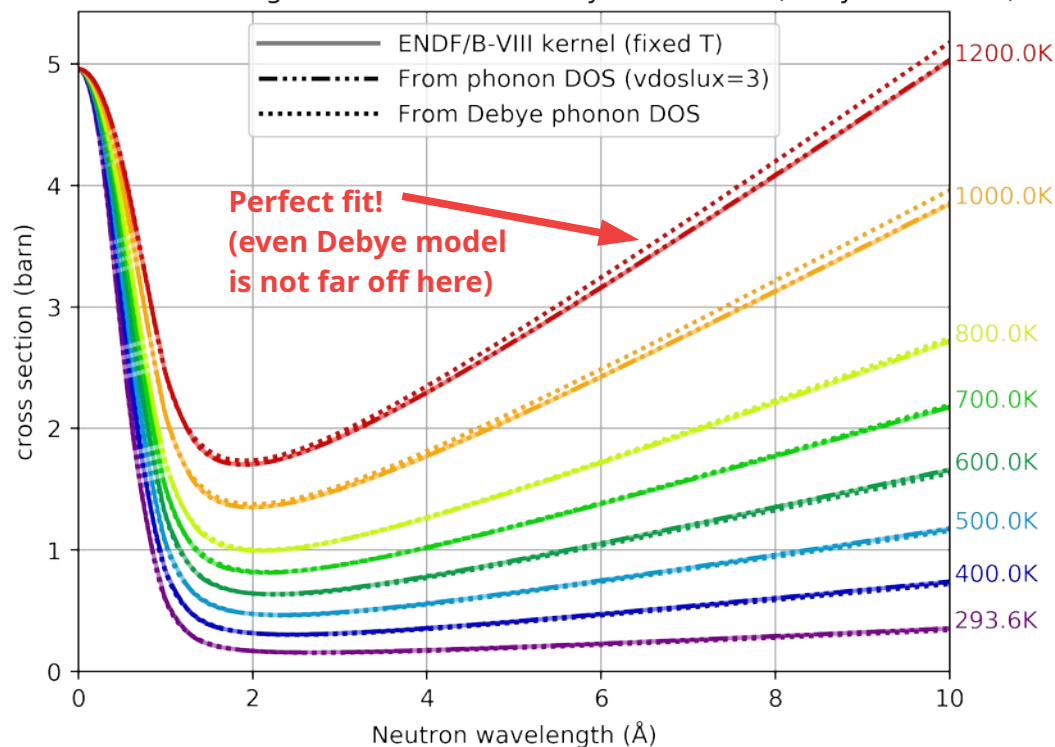


NCrystal
Thermal Neutron Transport

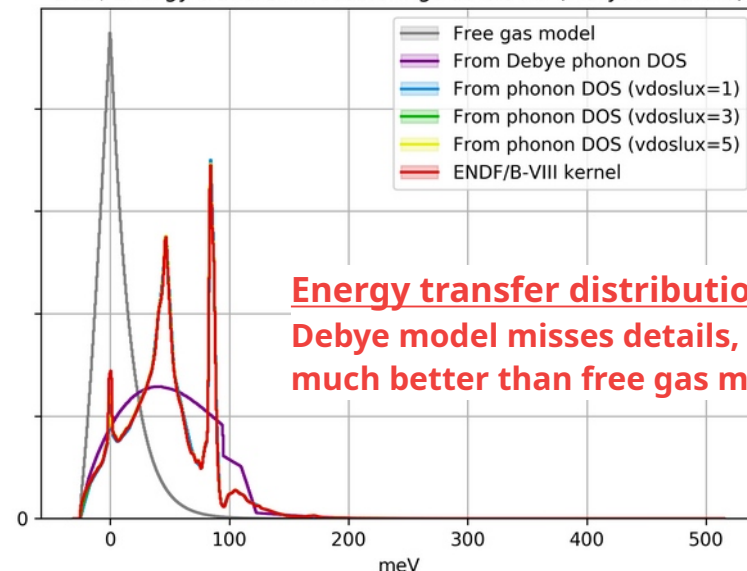
Identical results as when using LEAPR/ENDF kernels:

- Comparing at fixed temperature values from ENDF/B-VIII, using the same VDOS curve.
- NCrystal “luxury” level (cfg param “vdoslux”, default value 3) mostly affects scattering kernel grid size+granularity.

Inelastic scattering cross section for Beryllium Oxide (NCrystal v2.0.0)

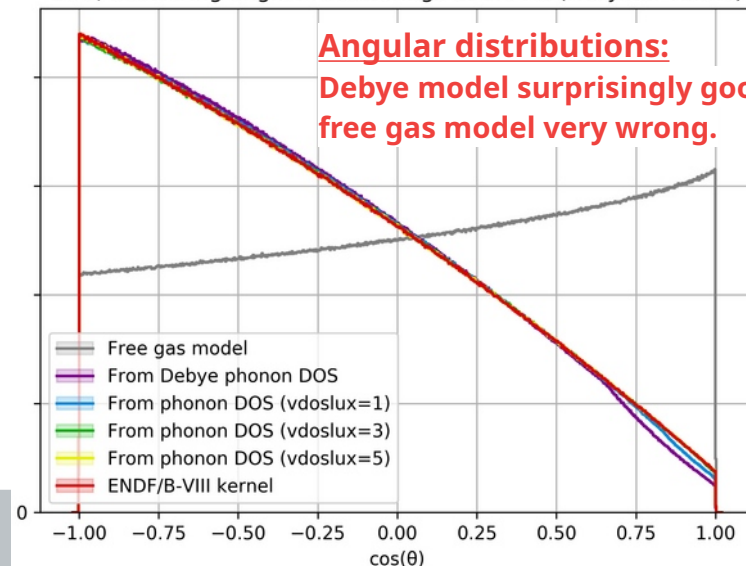


BeO, Energy transfer in scatterings at 1.8 Å (NCrystal v2.0.0)



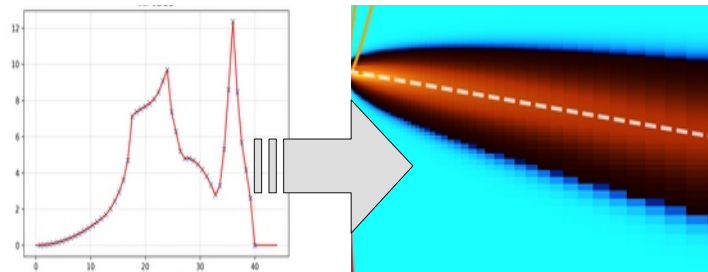
Energy transfer distributions:
Debye model misses details, but
much better than free gas model.

BeO, Scattering angle in scatterings at 1.8 Å (NCrystal v2.0.0)



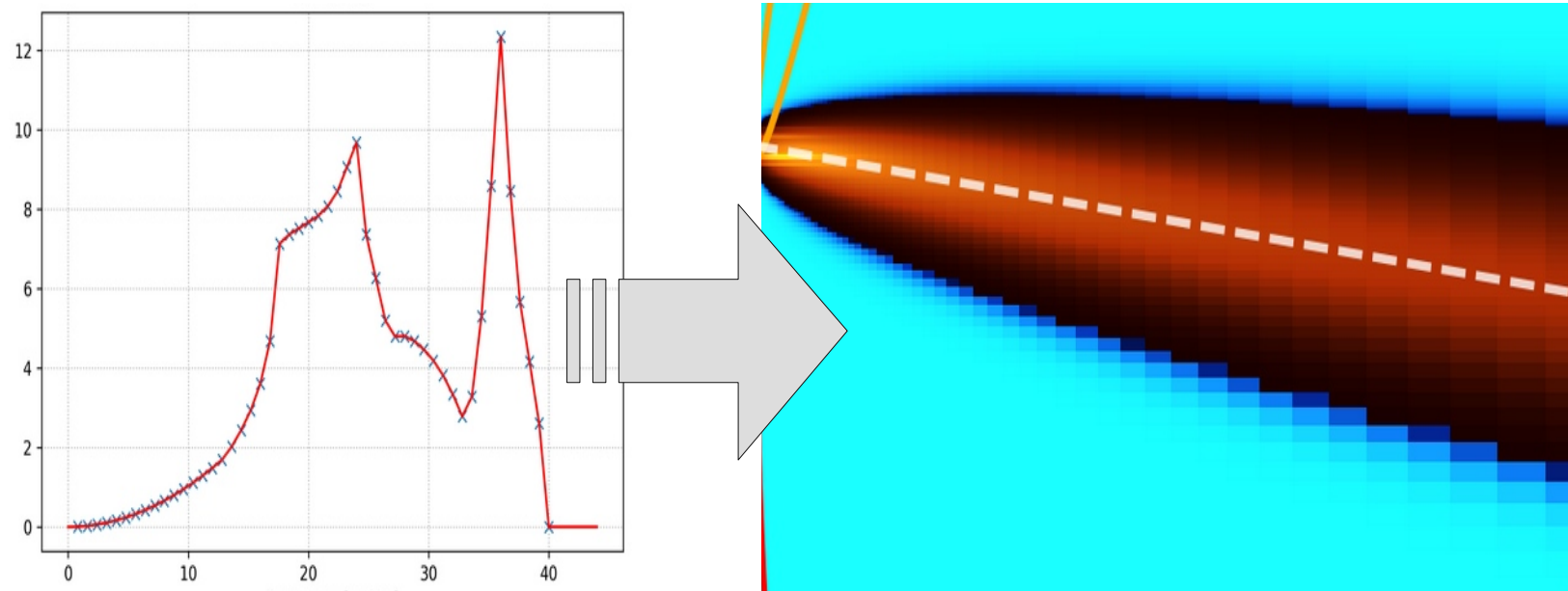
Angular distributions:
Debye model surprisingly good,
free gas model very wrong.

VDOS $\rightarrow$ Scattering kernel (the Sjölander method)



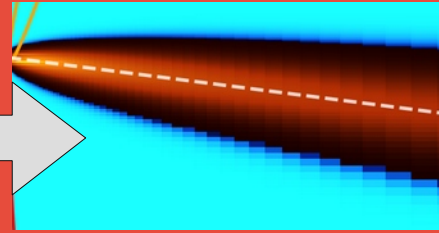
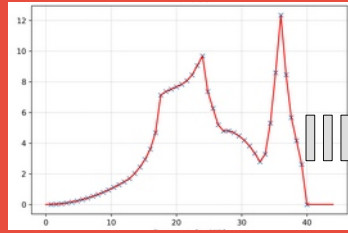
Connecting scattering kernels to phonons

Unlike scat. kernels, phonon spectra widely available



Captures material structure info relevant for inelastic neutron scatterings (isotropic materials, incoherent approximation)

VDOS $\rightarrow S(\alpha, \beta)$



NCrystal
Thermal Neutron Transport

ARKIV FÖR FYSIK Band 14 nr 21

Communicated 14 May 1958 by IVAR WALLER and ERIK RUDBERG

Multi-phonon processes in slow neutron scattering by crystals

By ALF SJÖLANDER

With 12 figures in the text

ABSTRACT

The multi-phonon processes in incoherent scattering of slow neutrons by crystals are discussed, assuming the harmonic approximation for the crystal vibrations. The differential scattering cross section is expanded in the Hermite orthogonal functions and approximate expressions for the cross section are derived. Extensive numerical calculations have been carried out to illustrate the accuracy of the approximations made. An approximation for the total cross section (the mass-ratio expansion) suggested by Placzek is discussed and in some respects generalized. The approximations for the differential cross section mentioned above are also used to derive approximate formulae for the total cross section valid for cold neutrons but arbitrary temperatures and mass ratios.

Introduction

The basic ideas of the theory of slow neutron scattering by crystals were developed by Wick [1], Pomeranchuk [2], Seeger and Teller [3] and Akhiezer and Pomeranchuk [4]. A quantitative account was given by Weinstock [5], who discussed the temperature dependence of the total scattering. Afterwards the formal treatment was completed especially by Fröman [6]. He separated the scattering into phonon processes and consistently used the analogies with X-ray diffraction. An alternative method, very convenient for calculating the total scattering cross section, was later suggested by Placzek [7]. Recently the theory was reformulated by Glauber [8] and Van Hove [9] making it more surveyable. They derived closed expressions for the differential scattering cross section, which seem to be a convenient starting point for quantitative discussions. Van Hove also generalized the theory to general systems of nuclei, as for instance liquids and magnetic materials. A large number of experiments have been performed and these mainly confirm the basis of the theoretical treatment [10, 10].

Well-established method!

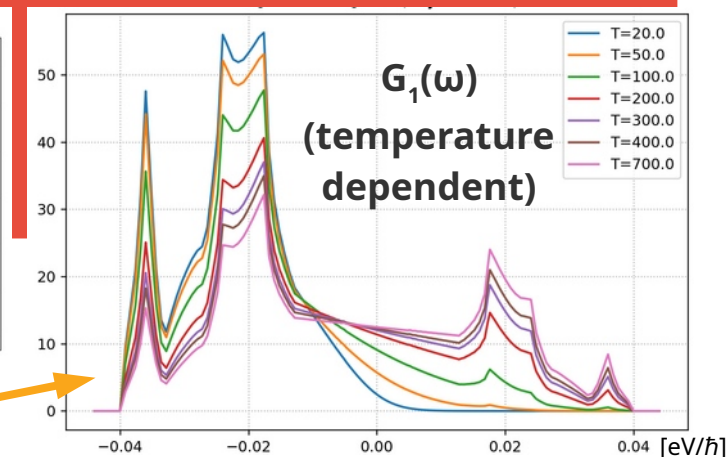
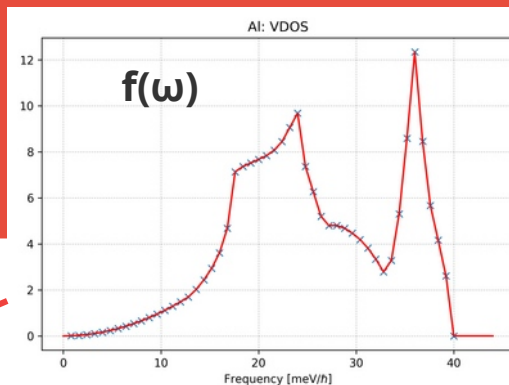
Used in NJOY/LEAPR. Most ENDF $S(\alpha, \beta)$ kernels were created this way.

Key feature: Capability is built into NCrystal
And so fast (<0.1s) it can be invoked on-the-fly.

Gives us:

- **Flexibility.** Work directly from VDOS input, avoid usage of non-trivial third-party SW.
- **More materials.** VDOS are much more easily obtained than full kernels.
- **Small data files!**
Can easily include everywhere.
- **Temperature dependency built in:**
Unlike static $S(\alpha, \beta)$ which is only valid for a specific temperature.

V DOS $\rightarrow S(\alpha, \beta)$ Sjölander's recipe



$$G_1(\omega) = g(\omega) = \frac{f(\omega)}{\omega \gamma(0)} \frac{\coth\left(\frac{\hbar\omega}{2kT}\right) - 1}{2},$$

$$G_2(\omega) = \int_{-\infty}^{\infty} g(\omega - \omega') G_1(\omega') d\omega',$$

...

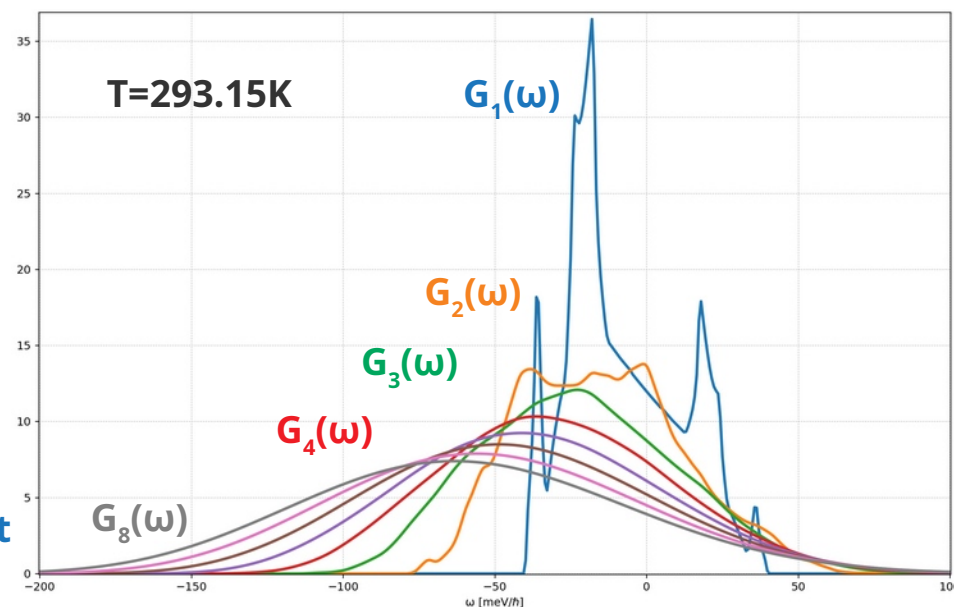
$$G_{n+1}(\omega) = \int_{-\infty}^{\infty} g(\omega - \omega') G_n(\omega') d\omega'.$$

$2W = \delta^2 q^2$, δ = atomic displacement

$$\frac{d^2 \sigma_{\text{inel}}}{d\Omega d\omega} = A \frac{k}{k_0} e^{-2W} \sum_{n=1}^{\infty} \frac{(2W)^n}{n!} G_n(\omega - \omega_0).$$

$$= \sqrt{\frac{E_f}{E}} \frac{\sigma_b}{4\pi} \frac{S(\alpha, \beta)}{k_b T}$$

$$-\hbar(\omega - \omega_0)/kT = \Delta E/kT = \beta$$

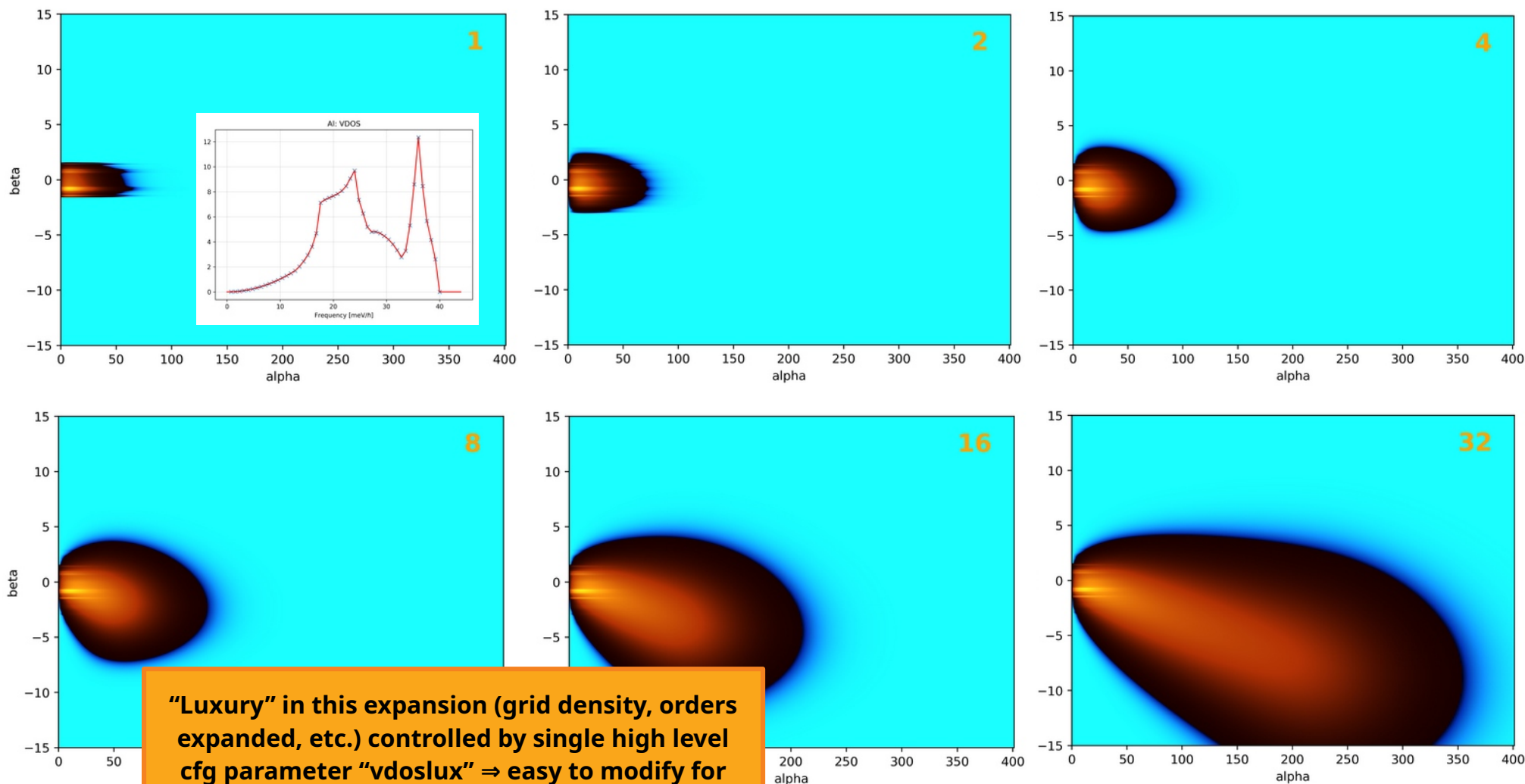


VDOS $\rightarrow$ $S(\alpha, \beta)$: Aluminium

<https://github.com/mctools/ncrystal/wiki/VDOSAnimations>



NCrystal
Thermal Neutron Transport



**“Luxury” in this expansion (grid density, orders expanded, etc.) controlled by single high level cfg parameter “vdoslux” $\Rightarrow$ easy to modify for any user (see backup slide for details).
Default value is of course sensible.**

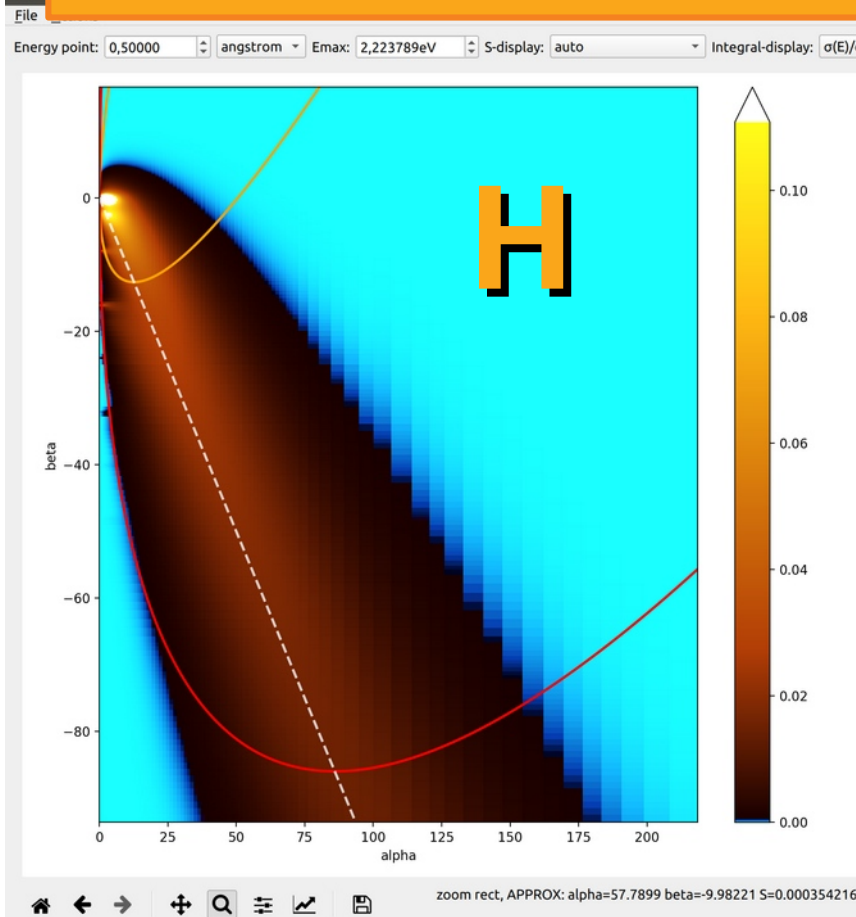
Control DOS→scat. kernel expansion through cfg parameter “vdoslux”

- Controls all aspects of DOS→kernel expansion with only one high level user-accessible parameter.
- Exposing the underlying multitude of parameters to end-users would do no good in practice since no-one would understand how to change them in a self-consistent way.
- vdoslux=0**: Extremely crude, 100x50 grid, Emax=0.5eV (costs 0.1MB mem, 0.02s init time) ← **The default if using Debye model instead of actual input data (vdoslux gets reduced by 3 for these mats.)**
- vdoslux=1**: Crude, 200x100 grid, Emax=1eV (costs 0.5MB mem, 0.04s init time)
- vdoslux=2**: Decent, 400x200 grid, Emax=3eV (costs 2MB mem, 0.08s init time) ← **Typical level in ENDF kernels**
- vdoslux=3**: Good, 800x400 grid, Emax=5eV (costs 8MB mem, 0.2s init time) ← **The default!**
- vdoslux=4**: Very good, 1600x800 grid, Emax=8eV (costs 30MB mem, 0.8s init time)
- vdoslux=5**: Extremely good, 3200x1600 grid, Emax=12eV (costs 125MB mem, 5s init time) ← **Overkill, exists for validation purpose**

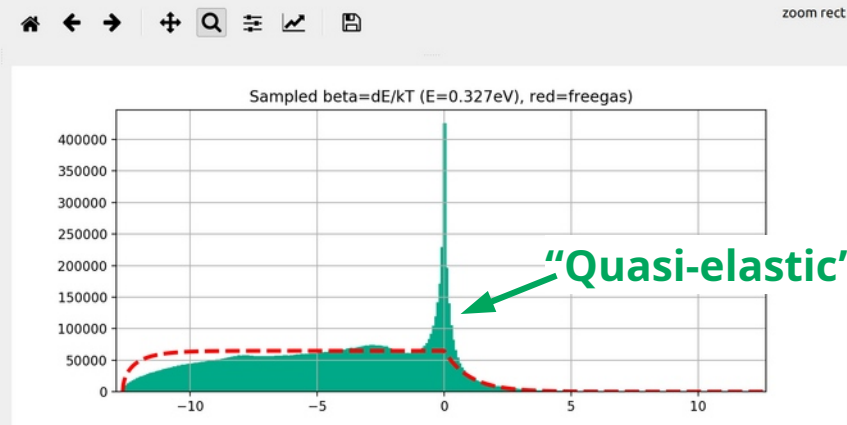
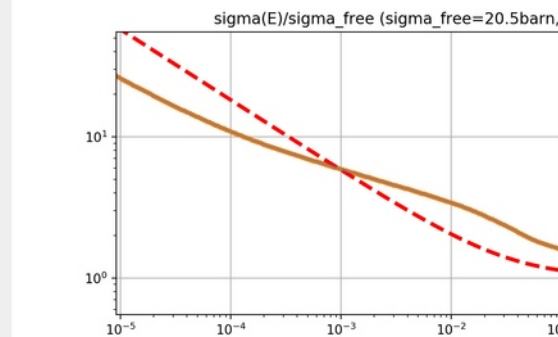
**Users adviced to leave at default (3),
or change with ± 1 to 2 or 4.**

Support for liquids rely on externally provided kernels, here water (converted from ENDF8)

VDOS→ $S(\alpha, \beta)$ does not work directly for liquids!
NCrystal for now relies on kernel converted from e.g. ENDF



Showing tsl-HinH2O_300.0K.ncmat [H, A=1.00794, fraction=66.67%, T=300K, kT=0.02585eV, 1.1MB]



Annals of Nuclear Energy 65 (2014) 280–289

Contents lists available at ScienceDirect

Annals of Nuclear Energy

journal homepage: www.elsevier.com/locate/anucene



CAB models for water: A new evaluation of the thermal neutron scattering laws for light and heavy water in ENDF-6 format

J.I. Márquez Damián^{a,*}, J.R. Granada^a, D.C. Malaspina^b

^aNeutron Physics Department and Instituto Balseiro, Centro Atómico Bariloche, CNEA, Argentina

^bDepartment of Biomedical Engineering and Chemistry of Life Processes Institute, Northwestern University, 2145 Sheridan Road, Evanston, IL

ARTICLE INFO

Article history:
Received 29 August 2013
Received in revised form 5 November 2013
Accepted 7 November 2013
Available online 6 December 2013

ABSTRACT

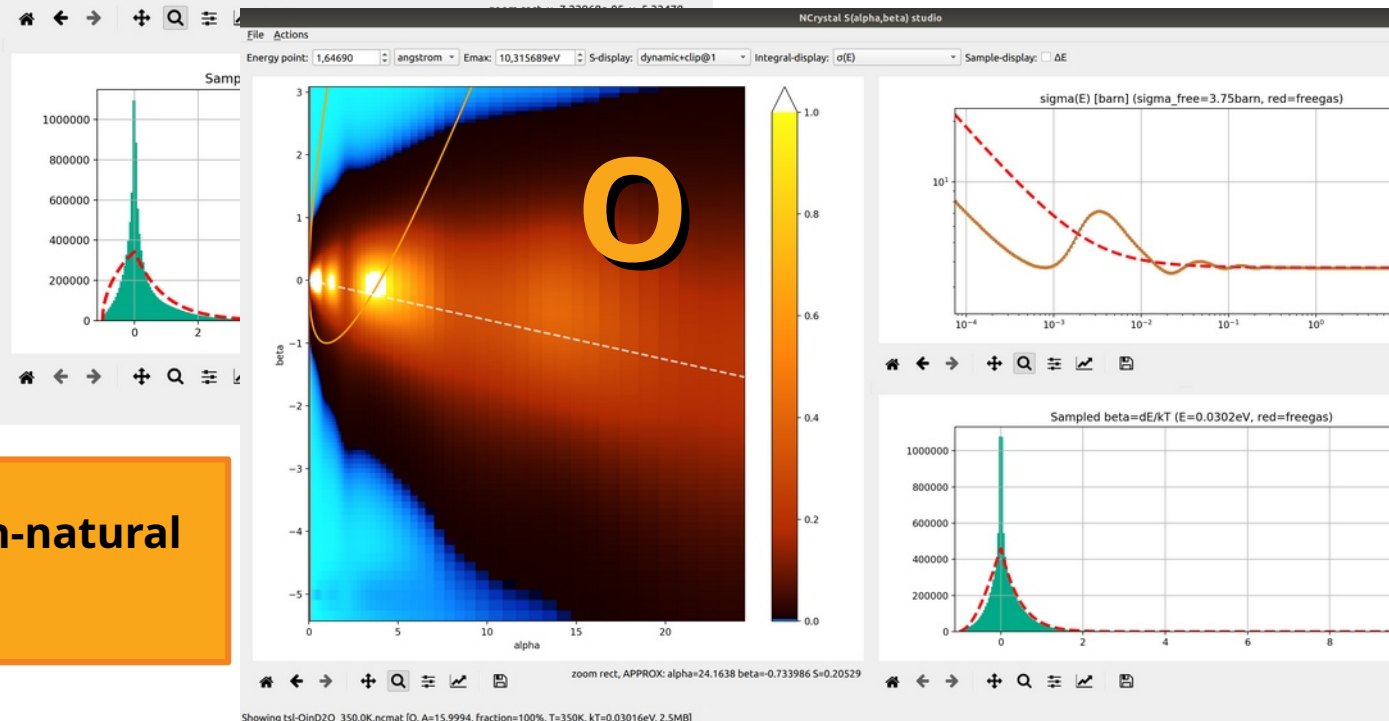
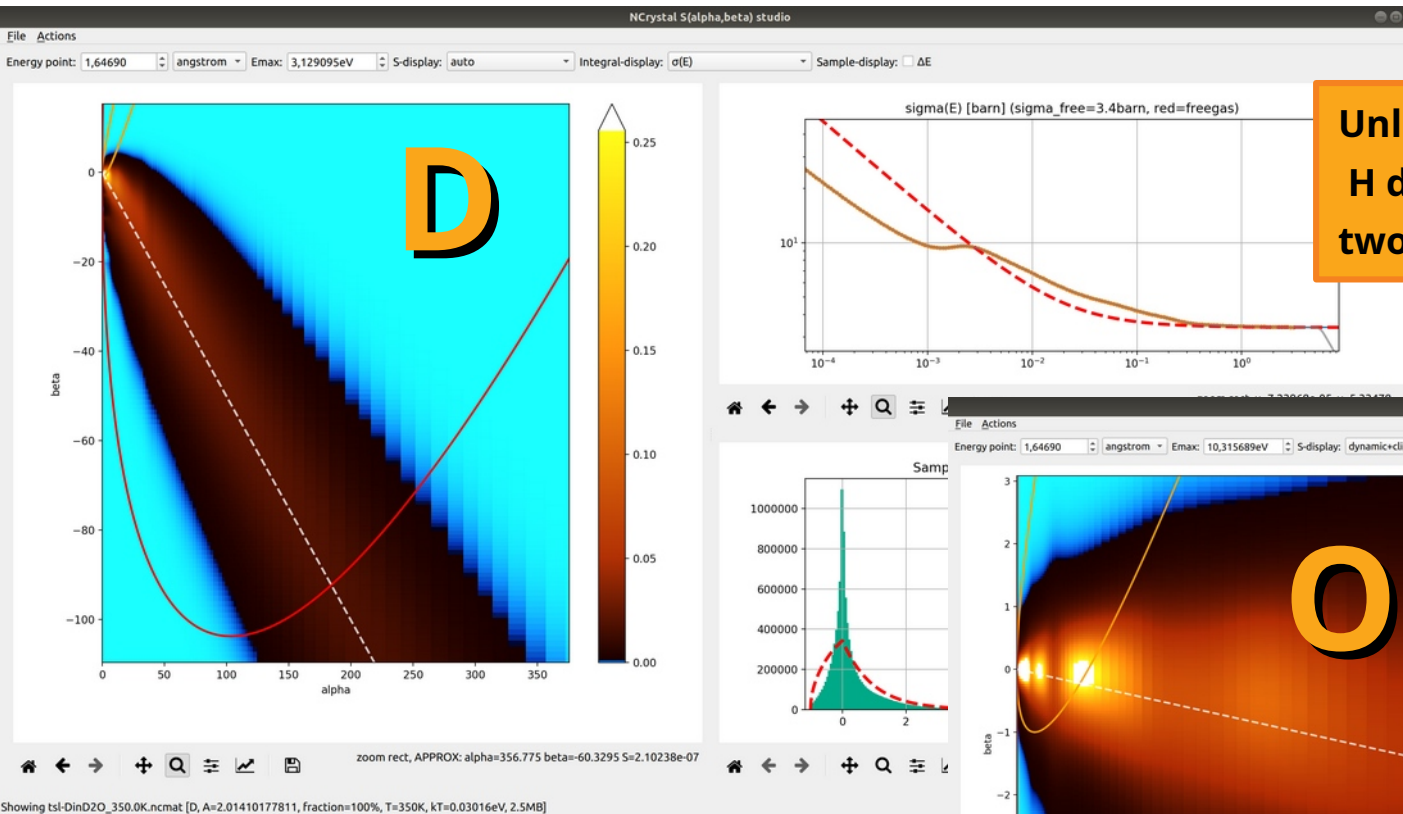
In this work we present the CAB models for water: a set of new models for neutron scattering laws for light and heavy water in ENDF-6 format, used in these models are based on experimental structure data and frequency spectra dynamics simulations. The calculations show a significant improvement when compared with measurements of differential and integral scattering

Oxygen is modelled as free gas

... and heavy water (also converted from ENDF8)

Unlike liquid H_2O (where scattering on H dominates completely), D_2O needs two kernels

NCrystal of course supports non-natural
“enrichments” like $\text{D} = {}^2\text{H}$



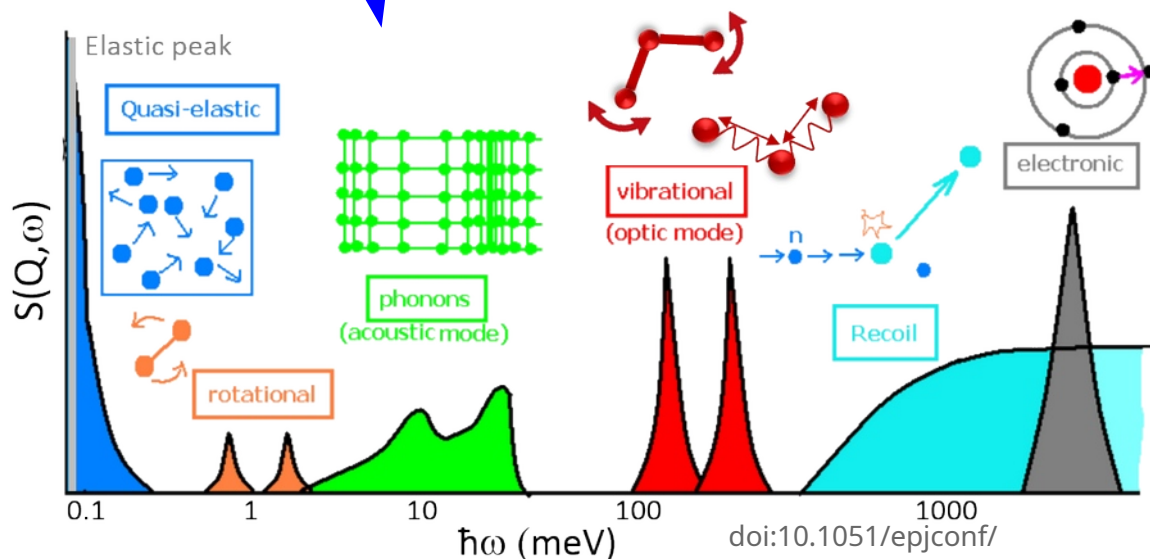
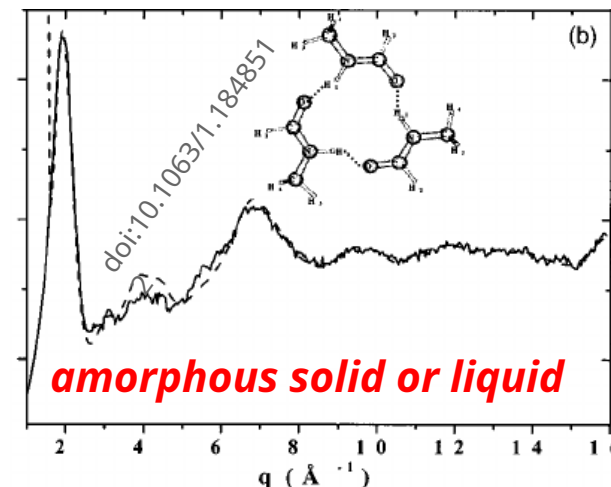
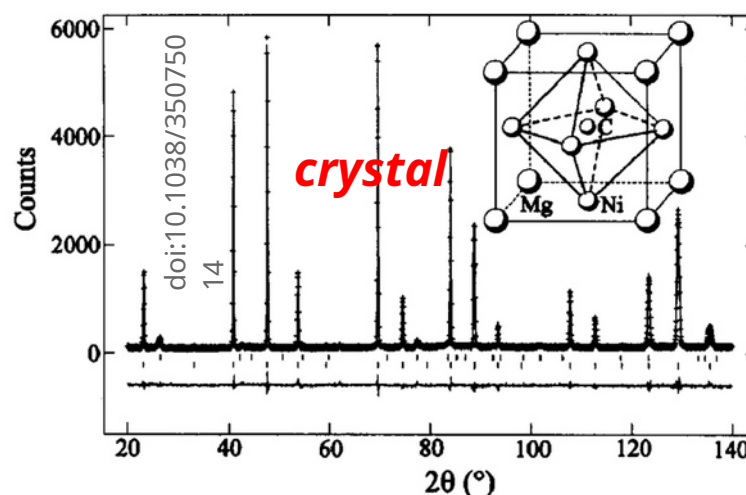
A small recap of Thermal Neutron scattering theory



Thermal neutron scattering: Rich connection to material structure

Thermal neutron
wavelength
 $\approx$ interatomic distances.
 $\Rightarrow$ **Sensitive to atomic
positions**

Thermal neutron energy
 $\approx$ material energy levels
 $\Rightarrow$ **sensitive to material
dynamics**



*Thermal neutron scattering a
uniquely useful tool for under-
standing material structure*

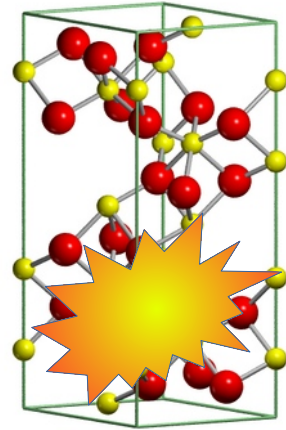
*On the flipside it is impossible
to ignore material structure
in MC simulations!*

Thermal neutron scattering (ignoring polarisation)

~~POLARISATION~~



$\vec{k}_i, E_i$



$\vec{k}_f, E_f$

Normally OK to assume neutrons travel as free waves between interactions $\Rightarrow$ simplifies analysis and Monte Carlo simulations!

$$\vec{k} = \vec{p}/\hbar, \quad \vec{q} \equiv \vec{k}_f - \vec{k}_i, \quad \Delta E \equiv E_f - E_i \equiv -\hbar\omega$$

$$\lambda = 2\pi/k, \quad 2E = m_n v^2 = p^2/m_n = \hbar^2 k^2/m_n$$

Probability to scatter to given $\vec{k}_f$ given by differential cross section:

$$\frac{d^2\sigma_{\vec{k}_i \Rightarrow \vec{k}_f}}{d\Omega_f dE_f} = \frac{k_f}{k_i} \left[\frac{1}{2\pi\hbar} \sum_{j,j'=1}^N b_j b_{j'} \int_{-\infty}^{\infty} dt \langle e^{-i\vec{q} \cdot \vec{R}_{j'}(0)} e^{i\vec{q} \cdot \vec{R}_j(t)} \rangle e^{-i\omega t} \right] S(\vec{q}, \omega)$$

N nuclei in target system (points to the sum over j, j')

Scattering length of jth nucleus (depends on isotope & spin state) (points to b_j)

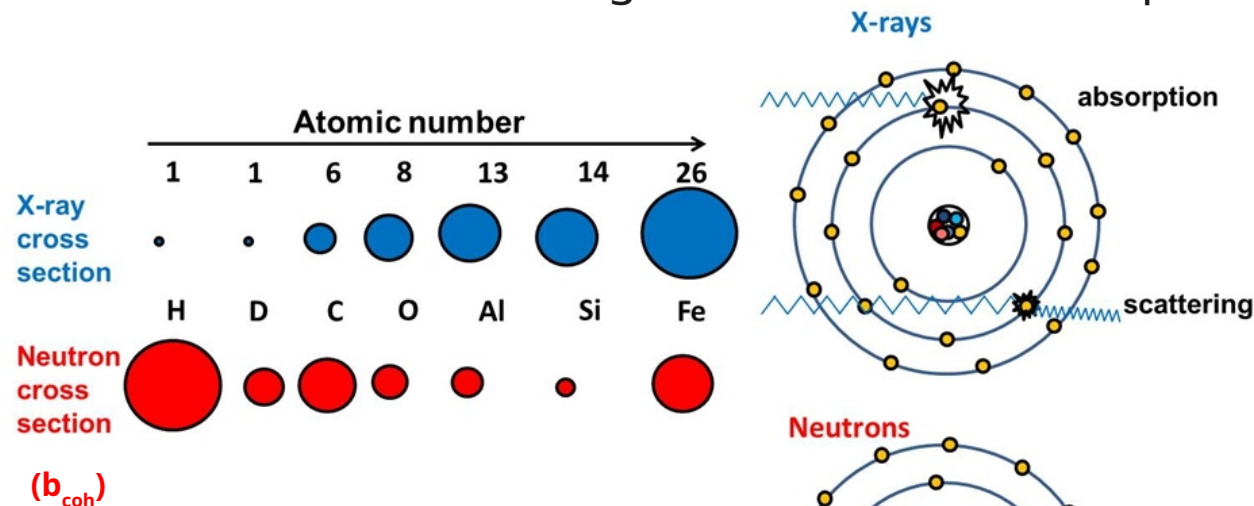
Correlate position of nucleus j at time 0 with position of nucleus j' at time t. $\Rightarrow$ Neutron X.S. depends on material structure!! (points to the time correlation function)

Scattering function
Depends on layout and dynamics of target particles. Does *not* depend on state of incident neutron. (points to $S(\vec{q}, \omega)$)

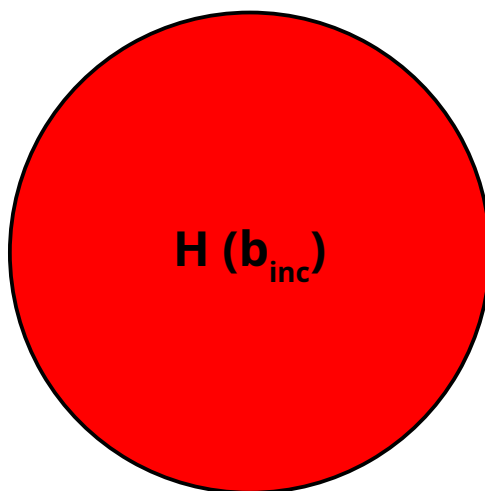
Scattering cross section depends on interference from scattering on different atoms, not just sum of 1-atom scatterings!!

Coherent scat. len. (b_{coh}) for select elements

X-ray strength increases with Z , neutron strength various across isotopes:



Note: hydrogen has huge *incoherent* scat. len. (b_{inc}):



- Neutron scattering sensitive to structure which is otherwise inaccessible to x-rays.
- Sample deuteration ($H \rightarrow D$) allows tuning hydrogen scattering strength

Split in coherent / incoherent

NB: This is unique to neutrons, absent for x-ray scattering!

Most target systems can be split into statistically equivalent subsystems (e.g. unit cells for crystals). Average contribution per sub-system:

$$S(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j,j'=1}^N \overline{b_j b_{j'}} \int_{-\infty}^{\infty} dt \langle j', j \rangle e^{-i\omega t}$$

Average

shorthand for $\langle e^{-i\vec{Q}\cdot\vec{R}_{j'}(0)} e^{i\vec{Q}\cdot\vec{R}_j(t)} \rangle$

(only nuclear charge (Z) important for structure, not isotope/spin state)

Using $\overline{b_j b_{j'}} = \begin{cases} \overline{b_j} \cdot \overline{b_{j'}}, & \text{for } j \neq j' \\ \overline{b_j^2}, & \text{for } j = j' \end{cases}$

and reordering terms:

$$S(\vec{Q}, \omega) = S_{\text{coh}}(\vec{Q}, \omega) + S_{\text{inc}}(\vec{Q}, \omega)$$

$$S_{\text{coh}}(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j,j'=1}^N \overline{b_j} \cdot \overline{b_{j'}} \int_{-\infty}^{\infty} dt \langle j', j \rangle e^{-i\omega t}$$

$\overline{b_j} \cdot \overline{b_{j'}} \stackrel{\text{def}}{=} \mathbf{b}_{\text{coh}}$

$$S_{\text{inc}}(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j=1}^N \left(\overline{b_j^2} - (\overline{b_j})^2 \right) \int_{-\infty}^{\infty} dt \langle j, j \rangle e^{-i\omega t}$$

$\overline{b_j^2} - (\overline{b_j})^2 \stackrel{\text{def}}{=} (\mathbf{b}_{\text{inc}})^2$

Coherent:

- Keep pair-correlations as in full $S(\mathbf{q}, \omega)$
- Plug in per-element scat. lengths, which are the *averages* of isotopic/spin scatlen

Incoherent:

- No pair-correlation, no interference! (but still *indirect* dep. on mat. structure!)
- Just sum up separate contributions.
- Plug in per-element scat. lengths which are the *variances* of isotopic/spin scatlen

Always 0 in X-ray scattering!

Elastic versus inelastic scatterings

(in the sense of $\Delta E=0$ and $\Delta E \neq 0$)

- Mathematically speaking, elastic scattering means factors of delta-functions: $\delta(\Delta E)$
- $\delta(\Delta E)$ appears as result of static (i.e. time-independent) correlations between atomic positions:

$$S(\vec{Q}, \omega) \equiv \frac{1}{2\pi\hbar} \sum_{j,j'=1}^N \overline{b_j b_{j'}} \int_{-\infty}^{\infty} dt \langle j', j \rangle e^{-i\omega t}$$

$\langle e^{-i\vec{Q} \cdot \vec{R}_{j'}(0)} e^{i\vec{Q} \cdot \vec{R}_j(t)} \rangle$
 (positional pair-correlations)

$\int_{-\infty}^{\infty} dt e^{-i\omega t} = 2\pi\hbar \delta(\hbar\omega) = 2\pi\hbar \delta(E_f - E_i)$

Fourier transform of static term
 gives δ -function in energy

Static correlations between atomic positions, and thus $\Delta E=0$ scatterings is a feature of solid materials (crystalline or amorphous).

Does not strictly speaking occur in liquids or gasses, although correlations in such might give rise to "quasi-elastic" scatterings peaking around $\Delta E=0$.

In summary: four $S(\bar{q}, \omega)$ components

$$S(\bar{q}, \omega) = S_{\text{coh,inel}}(\bar{q}, \omega) + S_{\text{inc,inel}}(\bar{q}, \omega) + S_{\text{coh,el}}(\bar{q}, \omega) + S_{\text{inc,el}}(\bar{q}, \omega)$$

solid systems only

Depending on material and neutron energy, any of these four components can dominate!

For isotropic materials: $S(\bar{q}, \omega) \rightarrow S(q, \omega)$, allowing to treat scattering via 2D function (*scattering kernels*).

In practice, only inelastic scattering is modelled via 2D scattering kernels. Elastic scattering involves delta-functions and is best described by dedicated algorithms.